

Die Signatur-Chiffre

**Eine strukturbasierte Verschlüsselungsmethode auf Basis
der injektiven Subgraph-Signaturfunktion**

Stephan Epp

1. März 2026

Inhaltsverzeichnis

1	Einführung	5
1.1	Motivation und historischer Kontext	5
1.2	Die Konsequenzen von $P = NP$	6
1.3	Die Signatur-Chiffre als neues Paradigma	6
1.4	Struktur der Arbeit	7
2	Mathematische Grundlagen	8
2.1	Die injektive Signaturfunktion	8
2.2	Modulare Arithmetik und Primzahlkörper	9
2.3	Bijektion der affinen Transformation	10
2.4	Schlüsseldefinition	11
3	Die Signatur-Chiffre	12
3.1	Blockstruktur und Textrepräsentation	12
3.2	Verschlüsselung	12
3.3	Entschlüsselung	12
4	Korrektheit und Sicherheit	13
4.1	Korrektheitsbeweis	13
4.2	Sicherheitsanalyse	13
5	Komplexitätsanalyse	15
6	Vollständiges Beispiel	16
6.1	Parameter	16
6.2	Klartext und Matrix	16
6.3	Signaturberechnung und Verschlüsselung	16
6.4	Entschlüsselung	16
7	Zweites vollständiges Beispiel: Optimales Parameterpaar $a = 17, b = 16$	17
7.1	Einleitung und Motivation	17
7.2	Formale Verifikation der Parameter	17
7.3	Berechnung des modularen Inversen $17^{-1} \bmod 257$	18
7.4	Klartext-Matrix und Signaturberechnung	19
7.5	Verschlüsselung	20
7.6	Entschlüsselung	21
7.7	Matrixrekonstruktion	22
7.8	Direktvergleich beider Beispiele und semantische Sicherheit	23
7.9	Kryptographische Analyse des Parameterpaars $(17, 16, 257)$	23
8	Python-Bibliothek	25
8.1	Softwarearchitektur und UML-Diagramme	25
8.2	Implementierung	28
9	Vergleich mit bestehenden Verfahren	32
9.1	Sicherheitsgrundlage und $P = NP$ -Robustheit	32

9.2	Quantenresistenz	32
9.3	Komplexität und praktische Effizienz	34
9.4	Schlüsselstruktur und Schlüsselraum	34
9.5	Informationstheoretische Äquivokation	34
9.6	Zusammenfassung	35
10	Elliptische-Kurven-Kryptographie (ECC)	36
10.1	Historische Einordnung und Motivation	36
10.2	Algebraische Grundlagen elliptischer Kurven	36
10.3	Skalarmultiplikation und Effizienz	37
10.4	Das Elliptic Curve Discrete Logarithm Problem (ECDLP)	37
10.5	Standardkurven	37
10.6	ECDH-Schlüsselaustausch	37
10.7	ECDSA – Elliptic Curve Digital Signature Algorithm	38
10.8	EdDSA und Ed25519	38
10.9	Verbindung zur Signatur-Chiffre	38
11	Gitterbasierte Kryptographie und Learning with Errors (LWE)	40
11.1	Gitter: Definition und grundlegende Eigenschaften	40
11.2	Schwere Gitterprobleme	40
11.3	Learning with Errors (LWE)	40
11.4	LWE-basierte Public-Key-Verschlüsselung (Regev-Schema)	41
11.5	Ring-LWE (RLWE) und die Kyber-Standardisierung	41
11.6	LWE-Erweiterung der Signatur-Chiffre	42
11.7	Weitere post-quantensichere Gitterkonstruktionen	42
12	Homomorphe Verschlüsselung	43
12.1	Grundidee und historische Entwicklung	43
12.2	Das Paillier-Kryptosystem	43
12.3	Gentrys FHE-Konstruktion (2009)	43
12.4	Das BGV-Schema	44
12.5	Das CKKS-Schema für reelle Zahlen	44
12.6	Threshold FHE und Multiparty Computation	45
12.7	Homomorphe Eigenschaften der Signatur-Chiffre	45
13	Das McEliece-Kryptosystem	46
13.1	Historische Einordnung	46
13.2	Grundlagen der Codierungstheorie	46
13.3	Goppa-Codes	46
13.4	Das McEliece-Kryptosystem: Konstruktion	46
13.5	Sicherheitsanalyse	47
13.6	Praktische Aspekte: Schlüsselgrößen	47
13.7	Verbindung zur Signatur-Chiffre	48
14	Das ElGamal-Kryptosystem und Public-Key-Varianten der Signatur-Chiffre	49
14.1	Das ElGamal-Kryptosystem	49
14.2	Semantische Sicherheit von ElGamal	49
14.3	Multiplikative Homomorphie von ElGamal	50
14.4	ElGamal auf elliptischen Kurven (EC-ElGamal)	50
14.5	Cramer-Shoup-Kryptosystem	50

14.6	Public-Key-Variante der Signatur-Chiffre	50
14.7	Digitale Signaturen auf Basis der Signatur-Chiffre	50
15	Sichere Mehrparteienberechnung (MPC)	52
15.1	Das Grundproblem und seine Bedeutung	52
15.2	Sicherheitsmodelle für MPC	52
15.3	Yaos Garbled Circuits	52
15.4	Oblivious Transfer (OT)	53
15.5	Secret Sharing nach Shamir	53
15.6	Das BGW-Protokoll für Mehrparteienberechnung	53
15.7	Threshold Decryption für die Signatur-Chiffre	54
15.8	Anwendungen von MPC	54
16	Zero-Knowledge-Beweise	55
16.1	Motivation und Grundbegriffe	55
16.2	Das Schnorr-Identifikationsprotokoll	55
16.3	Non-Interactive Zero-Knowledge Proofs (NIZK)	56
16.4	Commit-and-Prove-Protokolle	56
16.5	zk-SNARKs: Succinct Non-interactive ARguments of Knowledge . .	56
16.6	Zero-Knowledge für die Signatur-Chiffre	56
17	Kryptographische Hash-Funktionen	58
17.1	Definition und Sicherheitseigenschaften	58
17.2	Die Merkle-Damgård-Konstruktion	58
17.3	MD5 – Eine historische Warnung	58
17.4	SHA-2: SHA-256 und SHA-512	59
17.5	SHA-3/Keccak: Die Schwamm-Konstruktion	59
17.6	BLAKE2 und BLAKE3	60
17.7	Passwort-Hashing: bcrypt, scrypt, Argon2	60
17.8	HMAC: Hash-basierte Message Authentication Codes	60
17.9	Signaturbasierte Hash-Funktion	60
18	Protokolle und Anwendungen	62
18.1	TLS 1.3: Das wichtigste Sicherheitsprotokoll	62
18.2	Blockchain-Kryptographie	63
18.3	IoT-Kryptographie: Ressourcenbeschränkte Umgebungen	64
18.4	Privacy-Preserving Machine Learning (PPML)	65
18.5	Steganographie und digitale Wasserzeichen	65
19	Zusammenfassung und Ausblick	67
19.1	Zusammenfassung	67
19.2	Ausblick: Forschungsagenda	70

1 Einführung

Diese Arbeit entwickelt die **Signatur-Chiffre**, eine neue Verschlüsselungsmethode auf Basis der injektiven Signaturfunktion aus dem Subgraph Algorithmus [Epp26a]. Da $P = NP$ bewiesen ist [Epp26a], sind klassische Verfahren wie RSA [Riv78] und Diffie-Hellman [DH76] grundsätzlich gebrochen. Die Signatur-Chiffre beruht stattdessen auf struktureller Mehrdeutigkeit und informationstheoretischer Äquivokation [Sha49]. Diese Arbeit ist als buchartiges Nachschlagewerk konzipiert und enthält neben dem vollständigen formalen Aufbau des neuen Verfahrens ausführliche, eigenständige Darstellungen aller verwandten kryptographischen Paradigmen: Elliptische-Kurven-Kryptographie [Kob87, Mil86], Gitterbasierte Kryptographie und Learning with Errors [Reg05, LPR10], Homomorphe Verschlüsselung [Gen09, BGV12, CKKS17], das McEliece-Kryptosystem [McE78], das ElGamal-Kryptosystem [ElG85], Sichere Mehrparteienberechnung [Yao82, GMW87], Zero-Knowledge-Beweise [GMR89, BFM88], kryptographische Hash-Funktionen [Dam89, Mer89], TLS [Res18], Blockchain [Nak08], IoT-Kryptographie [BSW07], Privacy-Preserving Machine Learning [GLP+21] und Steganographie [Cox97].

1.1 Motivation und historischer Kontext

Kryptographie – die Wissenschaft der geheimen Kommunikation – ist so alt wie die Schrift selbst. Die ältesten bekannten Chiffren stammen aus dem antiken Ägypten (um 1900 v. Chr.) und dem antiken Griechenland (Skytale der Spartaner, ca. 700 v. Chr.). Die Caesar-Chiffre des römischen Feldherrn Julius Caesar, eine einfache Verschiebung des Alphabets um drei Positionen, ist das wohl bekannteste historische Beispiel. Im Mittelalter entwickelten arabische Gelehrte – allen voran al-Kindi (9. Jahrhundert) – die Häufigkeitsanalyse als erste systematische Angriffsmethode.

Das moderne Zeitalter der Kryptographie begann im Zweiten Weltkrieg: Die Deutschen setzten die elektromechanische Enigma-Maschine ein, deren Entschlüsselung durch Alan Turing und sein Team in Bletchley Park als einer der entscheidenden Beiträge zur alliierten Kriegswende gilt. Die Japaner verwendeten die PURPLE-Maschine, die amerikanischen Kryptoanalytiker ebenfalls brachen. Diese historischen Erfahrungen zeigten: kein Verschlüsselungsverfahren ist auf Dauer sicher, das nur auf der Geheimhaltung des Algorithmus beruht.

Die wissenschaftliche Grundlage moderner Kryptographie legte Claude Shannon 1949 in seiner bahnbrechenden Arbeit *Communication Theory of Secrecy Systems* [Sha49]. Shannons revolutionäre Einsicht war, dass Sicherheit *definiert* werden kann: Ein Verschlüsselungsverfahren ist *perfekt sicher* (perfect secrecy), wenn das Chifftrat keinerlei Information über den Klartext liefert – unabhängig von der Rechenkapazität des Angreifers. Shannon bewies, dass das **One-Time-Pad** [Ver26] – die bitweise XOR-Verknüpfung mit einem zufälligen Schlüssel gleicher Länge – dieses Kriterium erfüllt und unter allen perfekt sicheren Verfahren den kürzesten möglichen Schlüssel verwendet. Das One-Time-Pad leidet jedoch am fundamentalen Schlüsselaustauschproblem: Sender und Empfänger müssen einen geheimen Schlüssel von der Länge der Nachricht sicher austauschen, bevor sie kommunizieren können – was das ursprüngliche Sicherheitsproblem lediglich verschiebt.

Den nächsten großen Paradigmenwechsel vollzogen Whitfield Diffie und Martin Hellman 1976 mit ihrer wegweisenden Arbeit *New Directions in Cryptography* [DH76]. Sie erfanden das Konzept der **asymmetrischen (Public-Key) Kryptographie**: Ein öffentlicher Schlüssel zur Verschlüsselung und ein mathematisch verwandter, aber praktisch nicht

ableitbarer privater Schlüssel zur Entschlüsselung ermöglichen sicheren Schlüsselaustausch ohne vorherigen persönlichen Kontakt. Das Diffie-Hellman-Schlüsselaustauschprotokoll beruht auf der Schwierigkeit des *diskreten Logarithmusproblems*: Gegeben $g^a \bmod p$, ist es schwer, a zu finden.

Ron Rivest, Adi Shamir und Leonard Adleman realisierten 1978 mit RSA [Riv78] das erste praktische Public-Key-Kryptosystem. RSA beruht auf der Schwierigkeit der Faktorisierung: Das Produkt $n = pq$ zweier großer Primzahlen ist leicht zu berechnen, aber die Primfaktoren sind schwer zu rekonstruieren. Die Schlüsselerzeugung nutzt den Satz von Euler-Fermat: Für jedes m gilt $m^{\phi(n)} \equiv 1 \pmod{n}$, wobei $\phi(n) = (p-1)(q-1)$.

Seitdem hat sich die gesamte digitale Sicherheitsinfrastruktur – TLS/HTTPS [Res18], PGP, SSH, DNSSEC, S/MIME, Blockchain [Nak08], Signal, WhatsApp – auf diesen Grundlagen aufgebaut. Die Sicherheit beruht stets auf der Annahme $P \neq NP$: dass bestimmte mathematische Probleme (Faktorisierung, diskreter Logarithmus, elliptische Kurven, Gitterprobleme) in polynomieller Zeit unlösbar sind.

1.2 Die Konsequenzen von $P = NP$

Der Beweis $P = NP$ durch den Subgraph Algorithmus [Epp26a] hat fundamentale Konsequenzen für die gesamte Kryptographie. Alle Verschlüsselungsverfahren, deren Sicherheit auf NP-schweren Problemen beruht, sind prinzipiell gebrochen. Im Einzelnen:

RSA [Riv78] ist gebrochen, da die Faktorisierung ganzer Zahlen polynomiell lösbar ist. Das Diffie-Hellman-Verfahren [DH76] und alle seine Varianten (ECDH, DHE) sind gebrochen, da der diskrete Logarithmus polynomiell lösbar ist. Alle Varianten der Elliptischen-Kurven-Kryptographie – ECDSA, Ed25519, Curve25519 – sind gebrochen, da das elliptische Kurven-Diskrete-Logarithmusproblem (ECDLP) polynomiell lösbar ist. DSA ist gebrochen. PGP-Schlüssel sind kompromittiert. SSH-Handshakes können entschlüsselt werden. HTTPS-Verbindungen sind entschlüsselbar, sofern sie RSA oder ECDH im Handshake verwenden.

Symmetrische Verfahren wie AES [DR02] sind *nicht* direkt betroffen: ihre Sicherheit beruht nicht auf NP-Schwierigkeit, sondern auf der empirischen Resistenz gegen differenzielle [BS93] und lineare [Mat94] Kryptoanalyse. Grovers Quantenalgorithmus [Gro96] würde die effektive Schlüssellänge halbieren, ist aber kein klassischer Algorithmus. Kryptographische Hash-Funktionen wie SHA-256 [NIST15a] bleiben ebenfalls konzeptuell sicher.

1.3 Die Signatur-Chiffre als neues Paradigma

Die vorliegende Arbeit schlägt die **Signatur-Chiffre** als neues Verschlüsselungsparadigma vor, das von Grund auf nicht auf NP-Schwierigkeit angewiesen ist. Die Kernidee kombiniert zwei Bausteine: (1) die injektive Signaturfunktion σ aus dem Subgraph Algorithmus [Epp26a], die Klartextblöcke als Binärmatrizen bijektiv in natürliche Zahlen kodiert; (2) eine affine Transformation über einem *geheimen* Primzahlkörper $\mathbb{Z}_p$, die das Chiffre erzeugt.

Die Sicherheit beruht auf informationstheoretischer Äquivokation im Sinne Shannons [Sha49]: Ohne Kenntnis der geheimen Primzahl p ist für einen Angreifer nicht einmal bekannt, in welchem algebraischen Körper die Entschlüsselung stattfindet. Der Schlüsselraum umfasst für eine 256-Bit-Primzahl p etwa 2^{512} Elemente – erheblich mehr als der Schlüsselraum von AES-256.

1.4 Struktur der Arbeit

Kapitel 2 legt die mathematischen Grundlagen fest (Signaturfunktion, Körpertheorie, Komplexität). Kapitel 3 definiert die Signatur-Chiffre formal. Kapitel 4 beweist Korrektheit und Sicherheit. Kapitel 5 analysiert die Komplexität. Kapitel 6 gibt ein vollständiges numerisches Beispiel für das Parameterpaar $(a, b, p) = (7, 3, 1031)$. Kapitel 7 führt ein zweites vollständiges Beispiel für das algebraisch ausgezeichnete Parameterpaar $(a, b, p) = (17, 16, 257)$ mit der Fermat-Primzahl $p = 257 = 2^8 + 1$ durch; alle Schritte werden formal vollständig nachgewiesen und die Ergebnisse mit Kapitel 6 verglichen. Kapitel 8 beschreibt das Implementierungskonzept in Python. Kapitel 9 vergleicht mit bestehenden Verfahren. Kapitel 10–18 führen die verwandten kryptographischen Verfahren ausführlich und eigenständig aus. Kapitel 19 schließt mit erweiterter Zusammenfassung und sehr ausführlichem Ausblick, der insbesondere die Theorie optimaler Parameterpaare und Fermat-Primzahlen als neue Forschungsrichtungen entwickelt.

2 Mathematische Grundlagen

Dieses Kapitel legt die mathematischen Fundamente der Signatur-Chiffre. Es umfasst die injektive Signaturfunktion (Abschnitt 2.1), die Theorie der modularen Arithmetik und Primzahlkörper (Abschnitt 2.2), zentrale Eigenschaften der Signaturfunktion im Hinblick auf Bereichsgrenzen und Eindeutigkeit (Abschnitt 2.3), sowie die Definition des Schlüsselraums (Abschnitt 2.4). Alle Beweise werden vollständig und elementar geführt.

2.1 Die injektive Signaturfunktion

Die Signaturfunktion σ wurde im Kontext des Subgraph Algorithmus [Epp26a] eingeführt. Ihre zentrale Eigenschaft ist die Injektivität: jede Spalte einer binären Matrix wird auf eine eindeutige natürliche Zahl abgebildet.

Definition 2.1 (Signaturfunktion). Sei $A \in \{0,1\}^{n \times n}$ eine binäre Matrix und $j \in \{0, \dots, n-1\}$ ein Spaltenindex. Die **Signaturfunktion** ist definiert als:

$$\sigma(A, j) := \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Der erste Summand $\rho(A, j) := \sum_{i=0}^{n-1} A_{ij} \cdot 2^i$ heißt **Zeilenkomponente** (oder **Spalten-Binärwert**), der zweite $\lambda(j) := j \cdot 2^n$ heißt **Spaltengewicht**.

Bemerkung 2.1. Die Zeilenkomponente $\rho(A, j)$ ist genau die Interpretation des j -ten Spaltenvektors $(A_{0j}, A_{1j}, \dots, A_{n-1,j}) \in \{0,1\}^n$ als Dualzahl, wobei A_{0j} das niederwertigste Bit ist. Das Spaltengewicht $\lambda(j) = j \cdot 2^n$ verschiebt diesen Wert in ein exklusives Intervall $[j \cdot 2^n, (j+1) \cdot 2^n)$.

Lemma 2.1 (Bereichsgrenzen der Zeilenkomponente). Für alle $A \in \{0,1\}^{n \times n}$ und alle $j \in \{0, \dots, n-1\}$ gilt:

$$0 \leq \rho(A, j) \leq 2^n - 1 < 2^n.$$

Beweis. Da $A_{ij} \in \{0,1\}$ für alle i, j , gilt für die Zeilenkomponente:

$$\rho(A, j) = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i \geq 0.$$

Die obere Schranke ergibt sich, indem man alle $A_{ij} = 1$ setzt (maximaler Fall):

$$\rho(A, j) \leq \sum_{i=0}^{n-1} 1 \cdot 2^i = 2^n - 1.$$

Die letzte Gleichheit ist die geometrische Reihe $\sum_{i=0}^{n-1} 2^i = 2^n - 1$. □

Lemma 2.2 (Injektivität der Signaturfunktion, nach [Epp26a]). Die Signaturfunktion $\sigma : \{0,1\}^{n \times n} \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv in dem Sinne, dass aus $\sigma(A, j) = \sigma(A', j')$ folgt $j = j'$ und $A_{\cdot j} = A'_{\cdot j'}$ (d. h. die j -te Spalte von A stimmt mit der j' -ten Spalte von A' überein).

Beweis 1: Injektivität der Signaturfunktion. Seien (A, j) und (A', j') zwei Paare mit $\sigma(A, j) = \sigma(A', j')$. Dann gilt:

$$\rho(A, j) + j \cdot 2^n = \rho(A', j') + j' \cdot 2^n.$$

Schritt 1: Eindeutigkeit des Spaltenindex. Nach Lemma 2.1 gilt $\rho(A, j) \in [0, 2^n)$. Damit liegt $\sigma(A, j)$ im Intervall $[j \cdot 2^n, j \cdot 2^n + 2^n) = [j \cdot 2^n, (j+1) \cdot 2^n)$. Da die Intervalle $[k \cdot 2^n, (k+1) \cdot 2^n)$ für verschiedene $k \in \mathbb{N}_0$ paarweise disjunkt sind, bestimmt der Wert $\sigma(A, j)$ den Index j eindeutig. Aus $\sigma(A, j) = \sigma(A', j')$ folgt daher $j = j'$.

Schritt 2: Eindeutigkeit der Spalte. Mit $j = j'$ vereinfacht sich die Gleichung zu:

$$\rho(A, j) = \rho(A', j).$$

Da $\rho(A, j) = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i$ die eindeutige Binärdarstellung des Spaltenvektors $(A_{0j}, A_{1j}, \dots, A_{n-1,j})$ ist (jede natürliche Zahl $< 2^n$ hat eine eindeutige n -stellige Binärdarstellung), folgt $A_{ij} = A'_{ij}$ für alle $i \in \{0, \dots, n-1\}$, also $A_{\cdot j} = A'_{\cdot j}$. $\square$

Definition 2.2 (Signaturvektor). Der **Signaturvektor** einer Matrix $A \in \{0, 1\}^{n \times n}$ ist:

$$\Sigma(A) := (\sigma(A, 0), \sigma(A, 1), \dots, \sigma(A, n-1)) \in \mathbb{N}^n.$$

Korollar 2.1 (Injektivität des Signaturvektors). Die Abbildung $\Sigma : \{0, 1\}^{n \times n} \rightarrow \mathbb{N}^n$, $A \mapsto \Sigma(A)$ ist injektiv.

Beweis 2: Injektivität des Signaturvektors. Seien $A, A' \in \{0, 1\}^{n \times n}$ mit $\Sigma(A) = \Sigma(A')$. Dann gilt $\sigma(A, j) = \sigma(A', j)$ für alle $j \in \{0, \dots, n-1\}$. Nach Lemma 2.2 folgt $A_{\cdot j} = A'_{\cdot j}$ für jedes j , also stimmen alle Spalten von A und A' überein. Daher ist $A = A'$. $\square$

Bemerkung 2.2 (Wertebereich der Signaturfunktion). Für beliebige (A, j) gilt:

$$\sigma(A, j) \in [j \cdot 2^n, (j+1) \cdot 2^n).$$

Insbesondere gilt $\sigma(A, j) \leq (n-1) \cdot 2^n + (2^n - 1) = n \cdot 2^n - 1 < (n+1) \cdot 2^n \leq 2^{2n}$ (für $n \geq 1$, da $n+1 \leq 2^n$ für alle $n \geq 1$). Diese Schranke ist entscheidend für die Korrektheit der Entschlüsselung (Satz 4.1).

2.2 Modulare Arithmetik und Primzahlkörper

Die affine Transformation der Signatur-Chiffre arbeitet über dem Restklassenring $\mathbb{Z}_p$. Ist p eine Primzahl, so hat $\mathbb{Z}_p$ die Struktur eines Körpers – eine algebraische Eigenschaft, die für die Invertierbarkeit der Entschlüsselung unverzichtbar ist.

Definition 2.3 (Kongruenz und Restklassen). Zwei ganze Zahlen $a, b \in \mathbb{Z}$ heißen **kongruent modulo** $m \in \mathbb{N}$, geschrieben $a \equiv b \pmod{m}$, falls $m \mid (a - b)$, d. h. falls die Differenz $a - b$ durch m teilbar ist. Die Menge

$$\mathbb{Z}_m := \{0, 1, \dots, m-1\}$$

mit Addition und Multiplikation modulo m heißt **Restklassenring modulo** m .

Definition 2.4 (Multiplikatives Inverses). Ein Element $a \in \mathbb{Z}_m$ heißt **invertierbar** (oder **Einheit**), falls es ein $b \in \mathbb{Z}_m$ gibt mit $a \cdot b \equiv 1 \pmod{m}$. Man schreibt dann $b = a^{-1}$ und nennt b das **multiplikative Inverse** von a modulo m . Die Menge aller invertierbaren Elemente wird mit $\mathbb{Z}_m^*$ bezeichnet.

Satz 2.1 ($\mathbb{Z}_p$ ist ein Körper). Ist p eine Primzahl, so ist $\mathbb{Z}_p$ ein Körper: jedes Element $a \in \mathbb{Z}_p \setminus \{0\}$ besitzt ein eindeutiges multiplikatives Inverses $a^{-1} \in \mathbb{Z}_p$, das mit dem erweiterten euklidischen Algorithmus [Buc16] effizient berechnet werden kann.

Beweis 3: $\mathbb{Z}_p$ ist ein Körper. Sei p prim und $a \in \mathbb{Z}_p \setminus \{0\}$, d. h. $1 \leq a \leq p-1$.

Existenz des Inversen. Da p prim ist, teilt p kein $a \in \{1, \dots, p-1\}$. Damit gilt $\gcd(a, p) = 1$. Nach dem **Lemma von Bézout** existieren ganze Zahlen $s, t \in \mathbb{Z}$ mit:

$$s \cdot a + t \cdot p = 1.$$

Betrachtet man diese Gleichung modulo p , so ergibt sich:

$$s \cdot a \equiv 1 \pmod{p}.$$

Also ist $a^{-1} := s \bmod p$ das gesuchte multiplikative Inverse von a in $\mathbb{Z}_p$.

Eindeutigkeit. Angenommen, b und b' sind beide multiplikative Inverse von a modulo p , d. h. $ab \equiv 1 \pmod{p}$ und $ab' \equiv 1 \pmod{p}$. Dann gilt:

$$a(b - b') = ab - ab' \equiv 1 - 1 = 0 \pmod{p}.$$

Da p prim ist, folgt aus $p \mid a(b - b')$ und $\gcd(a, p) = 1$ (da $a \not\equiv 0$): $p \mid (b - b')$, d. h. $b \equiv b' \pmod{p}$, also $b = b'$ in $\mathbb{Z}_p$.

Berechenbarkeit. Die Koeffizienten s, t der Bézout-Darstellung werden durch den **erweiterten euklidischen Algorithmus** in $O(\log p)$ Schritten berechnet [Buc16]. Dieser Algorithmus führt sukzessive Division mit Rest durch und verfolgt die Koeffizienten rückwärts, bis er den ggT und die Darstellung $\gcd(a, p) = sa + tp$ liefert. Da $\gcd(a, p) = 1$, ergibt die Reduktion $s \bmod p$ das Inverse.

Da alle algebraischen Körperaxiome (Kommutativität, Assoziativität, Distributivität, Einselemente, additive Inverse, und nun auch multiplikative Inverse für alle Nicht-Nullelemente) in $\mathbb{Z}_p$ erfüllt sind [KL15], ist $\mathbb{Z}_p$ ein Körper. $\square$

Bemerkung 2.3. Die Körpereigenschaft von $\mathbb{Z}_p$ ist für die Signatur-Chiffre in zweifacher Hinsicht entscheidend: Erstens erlaubt sie die Division durch a (d. h. Multiplikation mit a^{-1}) im Entschlüsselungsschritt. Zweitens garantiert sie, dass die affine Abbildung $x \mapsto (ax + b) \bmod p$ eine Bijektion auf $\mathbb{Z}_p$ ist, was für die Eindeutigkeit der Entschlüsselung notwendig ist.

2.3 Bijektion der affinen Transformation

Lemma 2.3 (Bijektivität der affinen Transformation). Sei p prim, $a \in \mathbb{Z}_p^*$ und $b \in \mathbb{Z}_p$. Die Abbildung $f : \mathbb{Z}_p \rightarrow \mathbb{Z}_p$, $f(x) := (ax + b) \bmod p$, ist eine Bijektion. Die Umkehrabbildung ist $f^{-1}(y) = a^{-1}(y - b) \bmod p$.

Beweis. Da $|\mathbb{Z}_p|$ endlich ist, genügt es, Injektivität zu zeigen. Seien $x, x' \in \mathbb{Z}_p$ mit $f(x) = f(x')$. Dann:

$$ax + b \equiv ax' + b \pmod{p} \implies a(x - x') \equiv 0 \pmod{p}.$$

Da $\gcd(a, p) = 1$ (weil $a \in \mathbb{Z}_p^*$), folgt aus $p \mid a(x - x')$: $p \mid (x - x')$, also $x = x'$ in $\mathbb{Z}_p$. Damit ist f injektiv und auf einer endlichen Menge daher bijektiv. Die angegebene Umkehrabbildung erfüllt $f^{-1}(f(x)) = a^{-1}((ax + b) - b) = a^{-1} \cdot ax = x$. $\square$

2.4 Schlüsseldefinition

Definition 2.5 (Schlüssel der Signatur-Chiffre). Ein **Signatur-Chiffre-Schlüssel** ist ein Tripel (a, b, p) mit:

- p prim, $p > 2^{2n}$ (damit liegen alle Signaturen in $\mathbb{Z}_p$; vgl. Bemerkung 2.2),
- $a \in \mathbb{Z}_p^*$, d. h. $\gcd(a, p) = 1$,
- $b \in \mathbb{Z}_p$.

Das Tripel (a, b, p) bildet den **privaten Schlüssel**; der **öffentliche Parameter** ist n (die Blockgröße).

Bemerkung 2.4 (Existenz geeigneter Primzahlen). Für jedes $n \geq 1$ existieren unendlich viele Primzahlen $p > 2^{2n}$ (nach dem Satz von Euklid über die Unendlichkeit der Primzahlen). In der Praxis wählt man eine k -Bit-Primzahl mit $k \geq 2n + 1$, z. B. $k = 256$ Bit für $n \leq 127$. Der AKS-Primzahltest [AKS04] garantiert, dass Primzahlen in polynomieller Zeit (in $\log p$) erkannt werden können.

3 Die Signatur-Chiffre

3.1 Blockstruktur und Textrepräsentation

Die Signatur-Chiffre arbeitet blockweise mit Blöcken der Länge n^2 Bits, die als $n \times n$ Binärmatrix interpretiert werden.

Definition 3.1 (Klartext-Kodierung). Sei $m = (m_0, \dots, m_{n^2-1}) \in \{0, 1\}^{n^2}$ ein Klartext-Block. Die **Klartext-Matrix** ist:

$$A_{ij} := m_{j \cdot n + i}, \quad i, j \in \{0, \dots, n-1\}.$$

Der Eintrag A_{ij} ist das $(j \cdot n + i)$ -te Bit des Klartextblocks. Die Spalten von A entsprechen den konsekutiven n -Bit-Gruppen des Klartexts.

Bemerkung 3.1. Längere Klartexte werden in $\lceil |m|/n^2 \rceil$ Blöcke der Länge n^2 unterteilt (der letzte Block wird bei Bedarf mit Nullen aufgefüllt). Jeder Block wird unabhängig verschlüsselt (Electronic Codebook Mode). Für erhöhte Sicherheit kann ein Cipher Block Chaining (CBC) Modus implementiert werden, bei dem das Chifftrat des k -ten Blocks in den $(k+1)$ -ten Klartextblock eingebracht wird [MvOV96].

3.2 Verschlüsselung

Definition 3.2 (Verschlüsselungsabbildung). Sei $A \in \{0, 1\}^{n \times n}$ und (a, b, p) der private Schlüssel. Die **Verschlüsselung** ist:

$$\text{Enc}(A, a, b, p) := ((a \cdot \sigma(A, 0) + b) \bmod p, \dots, (a \cdot \sigma(A, n-1) + b) \bmod p) \in \mathbb{Z}_p^n.$$

Das Chifftrat $C = (c_0, \dots, c_{n-1})$ enthält n Elemente aus $\mathbb{Z}_p$, je eines pro Spalte der Klartext-Matrix.

Die Verschlüsselung besteht aus zwei Schritten: (1) Berechnung des Signaturvektors $\Sigma(A)$, (2) komponentenweise affine Transformation jedes $\sigma(A, j)$ über $\mathbb{Z}_p$.

3.3 Entschlüsselung

Definition 3.3 (Entschlüsselungsabbildung). Sei $C = (c_0, \dots, c_{n-1}) \in \mathbb{Z}_p^n$ ein Chifftrat und (a, b, p) der Schlüssel.

Schritt 1 – Signatur-Rekonstruktion: Für $j = 0, \dots, n-1$:

$$s_j := (a^{-1} \cdot (c_j - b)) \bmod p.$$

Schritt 2 – Matrix-Rekonstruktion: Für $i, j = 0, \dots, n-1$:

$$A_{ij} := \left\lfloor \frac{s_j - j \cdot 2^n}{2^i} \right\rfloor \bmod 2.$$

Das Ergebnis ist die rekonstruierte Klartext-Matrix $A \in \{0, 1\}^{n \times n}$.

Bemerkung 3.2. Schritt 2 ist die Umkehrung der Signaturbildung: aus $s_j = \rho(A, j) + j \cdot 2^n$ erhält man $\rho(A, j) = s_j - j \cdot 2^n$ und extrahiert dann das i -te Bit durch $\lfloor \rho(A, j)/2^i \rfloor \bmod 2$. Dies ist die Standardmethode zur Extraktion einzelner Bits aus einer Binärdarstellung.

4 Korrektheit und Sicherheit

Dieses Kapitel enthält die zentralen formalen Resultate der Arbeit: den vollständigen Korrektheitsbeweis (Beweis 4), den Nachweis der Immunität gegen Häufigkeitsanalysen (Beweis 5), die Berechnung der Schlüsselraumgröße (Beweis 6) sowie eine informationstheoretische Sicherheitsanalyse im Sinne Shannons.

4.1 Korrektheitsbeweis

Satz 4.1 (Korrektheit der Signatur-Chiffre). Für jeden Block $A \in \{0, 1\}^{n \times n}$, jeden Schlüssel (a, b, p) mit $p > 2^{2n}$ und $\gcd(a, p) = 1$ gilt:

$$\text{Dec}(\text{Enc}(A, a, b, p), a, b, p) = A.$$

Beweis 4: Korrektheit der Signatur-Chiffre. Sei $j \in \{0, \dots, n-1\}$ beliebig. Wir zeigen, dass der j -te Schritt der Entschlüsselung korrekt die j -te Spalte von A rekonstruiert.

Schritt 1: Rückgewinnung der Signatur. Die Verschlüsselung liefert $c_j = (a \cdot \sigma(A, j) + b) \bmod p$. Die Entschlüsselung berechnet:

$$s_j = a^{-1} \cdot (c_j - b) \bmod p = a^{-1} \cdot (a \cdot \sigma(A, j) + b - b) \bmod p = \sigma(A, j) \bmod p.$$

Nach Bemerkung 2.2 gilt $\sigma(A, j) \leq n \cdot 2^n - 1 < 2^{2n} < p$. Daher:

$$\sigma(A, j) \bmod p = \sigma(A, j).$$

Also ist $s_j = \sigma(A, j)$.

Schritt 2: Rückgewinnung der Zeilenkomponente. Es gilt $\sigma(A, j) = \rho(A, j) + j \cdot 2^n$, also:

$$\rho(A, j) = s_j - j \cdot 2^n.$$

Schritt 3: Rückgewinnung der Matrixeinträge. Die Zeilenkomponente $\rho(A, j) = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i$ ist die Binärdarstellung des Spaltenvektors. Das i -te Bit ist:

$$A_{ij} = \left\lfloor \frac{\rho(A, j)}{2^i} \right\rfloor \bmod 2 = \left\lfloor \frac{s_j - j \cdot 2^n}{2^i} \right\rfloor \bmod 2.$$

Dies stimmt exakt mit dem Entschlüsselungsschritt 2 aus Definition 3.3 überein.

Da j beliebig war, werden alle Spalten und damit die gesamte Matrix A korrekt rekonstruiert. $\square$

4.2 Sicherheitsanalyse

Satz 4.2 (Sicherheit gegen Häufigkeitsanalyse). Die Signatur-Chiffre ist immun gegen Häufigkeitsanalysen auf Buchstaben-, Byte- oder Blockebene: Ein Angreifer ohne Kenntnis von (a, b, p) kann aus dem Chifftrat C keine statistischen Schlüsse auf den Klartext A ziehen.

Beweis 5: Sicherheit gegen Häufigkeitsanalyse. Sei A ein Klartext-Block und $C = \text{Enc}(A, a, b, p)$. Wir zeigen, dass ein Angreifer, der C und den öffentlichen Parameter n kennt, aber (a, b, p) nicht kennt, aus C keine Information über A gewinnen kann.

Schlüsselraumargument. Für einen festen Chifftrat-Wert $c_j \in \mathbb{Z}_p$ und einen festen Klartext-Signaturwert $s_j = \sigma(A, j)$ gibt es genau ein Paar $(a, b) \in \mathbb{Z}_p^* \times \mathbb{Z}_p$ mit $c_j =$

$(as_j + b) \bmod p$ (für jedes $a \in \mathbb{Z}_p^*$ existiert genau ein $b = c_j - as_j \bmod p$). Da der Angreifer (a, b, p) nicht kennt, sind für ihn – a priori – alle $s_j \in \{0, \dots, p-1\}$ gleich wahrscheinlich als Urbilder von c_j .

Häufigkeitsimmunität. Da die affine Transformation $x \mapsto (ax+b) \bmod p$ eine Bijektion auf $\mathbb{Z}_p$ ist (Lemma 2.3), ist die Verteilung der Chiffpratwerte c_j für feste (a, b, p) eine Permutation der Signaturen. Für verschiedene Blöcke mit demselben Signaturwert (also demselben Klartext-Block) ergibt sich stets derselbe Chiffpratwert – aber verschiedene Blöcke mit verschiedenen Signaturen werden auf verschiedene Chiffpratwerte abgebildet, und ohne Kenntnis von a, b und p kann der Angreifer die Chiffpratwerte nicht vergleichbar ordnen.

Byte-Ebene. Eine Häufigkeitsanalyse auf Byte-Ebene – ein klassischer Angriff auf Substitutionschiffren – erfordert, dass gleiche Klartextbytes auf gleiche Chiffpratbytes abgebildet werden. Da die Signatur-Chiffre blockweise arbeitet und die Signatur die gesamte Spalteninformation kodiert, ist eine bijektive Abbildung auf Einzel-Byte-Ebene nicht gegeben: ein einzelner Bit-Flip in A ändert $\rho(A, j)$ um $\pm 2^i$ und damit $\sigma(A, j)$, was den Chiffpratwert c_j vollständig verändert (Lawineneffekt auf Blockebene). $\square$

Satz 4.3 (Schlüsselraumgröße). Für eine feste Primzahl p hat der Schlüsselraum der Signatur-Chiffre die Größe:

$$|\mathcal{K}| = (p-1) \cdot p.$$

Beweis 6: Schlüsselraumgröße. Ein Schlüssel ist ein Tripel (a, b, p) mit festem p . Der Parameter a muss aus $\mathbb{Z}_p^* = \{1, 2, \dots, p-1\}$ gewählt werden; das sind $p-1$ Möglichkeiten. Der Parameter b kann jeden der p Werte in $\mathbb{Z}_p = \{0, 1, \dots, p-1\}$ annehmen; das sind p Möglichkeiten. Da a und b unabhängig voneinander gewählt werden, ergibt das Produktprinzip:

$$|\mathcal{K}| = |\mathbb{Z}_p^*| \cdot |\mathbb{Z}_p| = (p-1) \cdot p.$$

$\square$

Bemerkung 4.1 (Sicherheitsniveau). Für eine 256-Bit-Primzahl $p \approx 2^{256}$ gilt:

$$|\mathcal{K}| = (p-1) \cdot p \approx 2^{256} \cdot 2^{256} = 2^{512}.$$

Dies entspricht einer effektiven Schlüssellänge von $\log_2(|\mathcal{K}|) \approx 512$ Bit und übertrifft damit den Schlüsselraum von AES-256 (2^{256}) erheblich. Hinzu kommt, dass auch p selbst geheim gehalten wird; der vollständige Suchraum für einen Brute-Force-Angriff umfasst daher zusätzlich alle infrage kommenden Primzahlen nahe der vereinbarten Bitlänge.

Satz 4.4 (Informationstheoretische Sicherheit unter Schlüsselsgeheimhaltung). Wenn p gleichmäßig zufällig aus der Menge der Primzahlen in $[2^{k-1}, 2^k)$ gewählt wird (für ein geheimes $k > 2n$) und ein Angreifer keinen Hinweis auf k, p, a oder b hat, so ist die Äquivokation des Schlüssels gegeben durch:

$$H(K | C) = H(K) = \log_2 |\mathcal{K}|,$$

d. h. das Chiffprat liefert keinerlei Information über den Schlüssel.

Beweis. Dies ist eine Konsequenz aus Shannons Theorie der perfekten Geheimhaltung [Sha49]: Wenn der Schlüsselraum mindestens so groß wie der Nachrichtenraum ist und der Schlüssel gleichmäßig zufällig gewählt wird, ist jeder Chiffpratwert c_j für jeden möglichen Klartextwert s_j durch mindestens ein Schlüsselpaar (a, b) erreichbar. Da die affine Transformation (bei festem s_j) bijektiv auf den Chiffpratraum wirkt, ist jedes $c_j \in \mathbb{Z}_p$ gleichwahrscheinlich – unabhängig vom Klartext. Der Angreifer lernt aus C nichts über $K = (a, b, p)$. $\square$

5 Komplexitätsanalyse

Satz 5.1 (Verschlüsselungskomplexität). Die Verschlüsselung eines n^2 -Bit-Blocks hat Laufzeit $O(n^2)$ bei Wortgröße $O(\log p) = O(n)$ Bit.

Beweis 7: Verschlüsselungskomplexität. Die Verschlüsselung besteht aus zwei Phasen:

Phase 1: Berechnung des Signaturvektors. Für jede der n Spalten j wird $\sigma(A, j) = \rho(A, j) + j \cdot 2^n$ berechnet. Die Zeilenkomponente $\rho(A, j) = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i$ erfordert n Multiplikationen $A_{ij} \cdot 2^i$ (da $A_{ij} \in \{0, 1\}$: dies sind lediglich bedingte Bitverschiebungen) und $n - 1$ Additionen. Das Spaltengewicht $j \cdot 2^n$ ist eine Verschiebung um n Bits. Damit kostet jede Signaturberechnung $O(n)$ Operationen; für alle n Spalten zusammen: $O(n^2)$.

Phase 2: Affine Transformation. Für jede der n Signaturen s_j wird $c_j = (a \cdot s_j + b) \bmod p$ berechnet. Jede Multiplikation $a \cdot s_j$ auf Zahlen der Größe $O(p) = O(2^{2n})$ erfordert $O(\log^2 p) = O(n^2)$ Bit-Operationen (Schulmultiplikation) oder $O(n \log n \log \log n)$ mit FFT-basierten Methoden. Bei Einzel-Wort-Arithmetik ($\log p = \text{konstante Wortanzahl}$) gilt $O(1)$ pro Multiplikation. In der Standard-Analyse mit $k = O(n)$ Bit-Wörtern: $O(n)$ Operationen pro Transformation, für alle n Spalten: $O(n^2)$.

Gesamtlaufzeit. $O(n^2) + O(n^2) = O(n^2)$. □

Satz 5.2 (Entschlüsselungskomplexität). Die Entschlüsselung hat Laufzeit $O(n^2)$.

Beweis 8: Entschlüsselungskomplexität. Die Entschlüsselung besteht aus zwei Schritten:

Schritt 1: Signatur-Rekonstruktion. Das Inverse $a^{-1} \bmod p$ wird einmalig mit dem erweiterten euklidischen Algorithmus berechnet; dies kostet $O(\log^2 p) = O(n^2)$ Bit-Operationen, oder $O(n)$ Multiplikationen über k -Bit-Wörtern. Anschließend werden für jedes j die Operationen $(c_j - b) \bmod p$ und $a^{-1} \cdot (\cdot) \bmod p$ durchgeführt; je $O(n)$ pro Spalte, für alle n Spalten: $O(n^2)$.

Schritt 2: Matrix-Rekonstruktion. Für jede Spalte j wird $\rho_j = s_j - j \cdot 2^n$ berechnet (eine Subtraktion, $O(n)$). Dann werden die n Bits $A_{ij} = \lfloor \rho_j / 2^i \rfloor \bmod 2$ extrahiert; dies entspricht n Rechts-Verschiebungen und Maskierungen, also $O(n)$ pro Spalte. Für alle n Spalten: $O(n^2)$.

Gesamtlaufzeit. $O(n^2)$. □

Korollar 5.1 (Effizienzvergleich). Die Signatur-Chiffre ist asymptotisch effizienter als RSA mit vergleichbarer Schlüssellänge. RSA-Entschlüsselung für eine k -Bit-Nachricht hat Laufzeit $O(k^3)$ (modulare Exponentiation mit Schulmultiplikation) [Riv78]; mit $k = n^2$ als Klartext-Bitlänge wäre RSA in $O(n^6)$, während die Signatur-Chiffre in $O(n^2)$ operiert.

Beweis. RSA-Verschlüsselung und -Entschlüsselung erfordern modulare Exponentiation $c = m^e \bmod N$, die mit schneller Exponentiation (Square-and-Multiply) $O(\log e)$ Multiplikationen erfordert; jede Multiplikation auf k -Bit-Zahlen kostet $O(k^2)$ (Schulmultiplikation). Mit $\log e = O(k)$ ergibt sich $O(k^3)$ insgesamt [MvOV96]. Für $k = n^2$: $O(n^6)$. Die Signatur-Chiffre benötigt $O(n^2)$. Das Verhältnis ist $O(n^4)$ zugunsten der Signatur-Chiffre. □

6 Vollständiges Beispiel

6.1 Parameter

Wir demonstrieren die Signatur-Chiffre für $n = 4$, Schlüssel $(a, b, p) = (7, 3, 1031)$ (1031 ist prim, $1031 > 2^8 = 256 = 2^{2 \cdot 4}$).

6.2 Klartext und Matrix

Klartext-Block (16 Bits, spaltenweise): $m = (1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0)$.

$$A = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

6.3 Signaturberechnung und Verschlüsselung

$$\begin{aligned} \sigma(A, 0) &= 1+0+4+8 + 0 \cdot 16 = 13, & c_0 &= (7 \cdot 13 + 3) \bmod 1031 = 94 \\ \sigma(A, 1) &= 0+2+0+0 + 1 \cdot 16 = 18, & c_1 &= (7 \cdot 18 + 3) \bmod 1031 = 129 \\ \sigma(A, 2) &= 1+2+0+8 + 2 \cdot 16 = 43, & c_2 &= (7 \cdot 43 + 3) \bmod 1031 = 304 \\ \sigma(A, 3) &= 0+0+4+0 + 3 \cdot 16 = 52, & c_3 &= (7 \cdot 52 + 3) \bmod 1031 = 367 \end{aligned}$$

Chiffre: $C = (94, 129, 304, 367)$.

6.4 Entschlüsselung

Inverses: $7^{-1} \bmod 1031 = 442$ (da $7 \cdot 442 = 3094 = 3 \cdot 1031 + 1 \equiv 1$). Dann:

$$\begin{aligned} s_0 &= 442 \cdot (94 - 3) \bmod 1031 = 442 \cdot 91 \bmod 1031 = 40222 \bmod 1031 = 13 \checkmark \\ s_1 &= 442 \cdot (129 - 3) \bmod 1031 = 442 \cdot 126 \bmod 1031 = 18 \checkmark \\ s_2 &= 442 \cdot (304 - 3) \bmod 1031 = 442 \cdot 301 \bmod 1031 = 43 \checkmark \\ s_3 &= 442 \cdot (367 - 3) \bmod 1031 = 442 \cdot 364 \bmod 1031 = 52 \checkmark \end{aligned}$$

Aus $s_0 - 0 \cdot 16 = 13 = 1101_2$ folgt Spalte 0: $(1, 0, 1, 1)^T$. Aus $s_1 - 1 \cdot 16 = 2 = 0010_2$: $(0, 1, 0, 0)^T$. Aus $s_2 - 2 \cdot 16 = 11 = 1011_2$: $(1, 1, 0, 1)^T$. Aus $s_3 - 3 \cdot 16 = 4 = 0100_2$: $(0, 0, 1, 0)^T$. Die Matrix A wird exakt rekonstruiert.

7 Zweites vollständiges Beispiel: Optimales Parameterpaar $a = 17, b = 16$

7.1 Einleitung und Motivation

Das vorangegangene Kapitel 6 hat die Signatur-Chiffre für das Parameterpaar $(a, b, p) = (7, 3, 1031)$ exemplarisch vorgeführt. Das vorliegende Kapitel demonstriert das Verfahren für ein zweites, algebraisch besonders ausgezeichnetes Parameterpaar: $a = 17$ und $b = 16$, mit der Primzahl $p = 257$.

Dieses Paar ist aus mehreren Gründen mathematisch hervorragend geeignet:

- Die Primzahl $p = 257 = 2^8 + 1$ ist eine **Fermat-Primzahl** ($F_3 = 2^{2^3} + 1$), die einzige Fermat-Primzahl im Bereich $[257, 65537]$. Fermat-Primzahlen besitzen außergewöhnlich günstige algebraische Eigenschaften in endlichen Körpern, insbesondere für modulare Arithmetik und Effizienzoptimierungen.
- Der Wert $p = 257$ erfüllt genau die Mindestbedingung $p > 2^{2^n} = 2^8 = 256$ für $n = 4$ – es ist die *kleinstmögliche* Primzahl, die für $n = 4$ korrekte Entschlüsselung garantiert. Dies macht das Parameterpaar zum minimalen zulässigen Extremfall.
- $a = 17$ ist prim und teilerfremd zu $p - 1 = 256 = 2^8$, da 17 eine ungerade Primzahl $\neq 2$ ist. Insbesondere gilt $\gcd(17, 256) = 1$.
- $b = 16 = 2^4 = 2^n$ hat eine strukturelle Bedeutung: Es ist genau das Spaltengewicht der Signaturfunktion für $j = 1$. Diese Wahl illustriert, dass auch “strukturell sichtbare” Additionskonstanten kryptographisch unbedenklich sind, da die Sicherheit ausschließlich aus der Geheimhaltung des Schlüsseltripels (a, b, p) stammt.
- Das Parameterpaar steht in direktem Kontrast zum Beispiel 1 (Kapitel 6) mit $(a, b, p) = (7, 3, 1031)$: Derselbe Klartext wird unter beiden Schlüsseln auf völlig verschiedene Chiffre abgebildet – ein *empirischer Nachweis semantischer Sicherheit*.

Im Folgenden werden alle Schritte des Verfahrens – Parameterverifikation, Schlüsselleistung, Signaturfunktion, Verschlüsselung, Entschlüsselung und Matrixrekonstruktion – vollständig formal nachgewiesen. Für Quervergleiche wird dieselbe Klartextmatrix wie in Kapitel 6 verwendet.

7.2 Formale Verifikation der Parameter

Satz 7.1 (Gültigkeit des Parametertripels $(17, 16, 257)$ für $n = 4$). Das Tripel $(a, b, p) = (17, 16, 257)$ ist ein gültiger Schlüssel der Signatur-Chiffre für $n = 4$. Es gelten:

1. $p = 257$ ist eine Primzahl.
2. $p = 257 > 256 = 2^{2^4} = 2^{2^n}$.
3. $\gcd(a, p) = \gcd(17, 257) = 1$.
4. $a \in \mathbb{Z}_p^* = \{1, \dots, 256\}$, also $a = 17 \in \mathbb{Z}_{257}^*$.
5. $b \in \mathbb{Z}_p = \{0, \dots, 256\}$, also $b = 16 \in \mathbb{Z}_{257}$.

Beweis (Satz 7.1). **(1) Primzahltest für $p = 257$.** Es genügt zu prüfen, dass 257 durch keine Primzahl $q \leq \lfloor \sqrt{257} \rfloor = \lfloor 16.03 \rfloor = 16$ teilbar ist. Die Primzahlen ≤ 16 sind: 2, 3, 5, 7, 11, 13.

$$\begin{array}{ll} 257 = 128 \cdot 2 + 1, & \text{also } 2 \nmid 257, \\ 257 = 85 \cdot 3 + 2, & \text{also } 3 \nmid 257, \\ 257 = 51 \cdot 5 + 2, & \text{also } 5 \nmid 257, \\ 257 = 36 \cdot 7 + 5, & \text{also } 7 \nmid 257, \\ 257 = 23 \cdot 11 + 4, & \text{also } 11 \nmid 257, \\ 257 = 19 \cdot 13 + 10, & \text{also } 13 \nmid 257. \end{array}$$

Da kein Teiler $\leq \sqrt{257}$ existiert, ist 257 prim. (Alternativ: $257 = 2^8 + 1 = F_3$, die vierte Fermat-Zahl; es ist bekannt, dass $F_0 = 3$, $F_1 = 5$, $F_2 = 17$, $F_3 = 257$ und $F_4 = 65537$ sämtlich prim sind.)

(2) Schrankenverifikation. $2^{2n} = 2^{2 \cdot 4} = 2^8 = 256 < 257 = p$. Die Bedingung $p > 2^{2n}$ aus Satz 4.1 ist erfüllt.

(3) Teilerfremdheit $\gcd(17, 257) = 1$. Da 257 prim ist, gilt $\gcd(a, 257) \in \{1, 257\}$ für alle $a \in \mathbb{Z}$. Da $17 \neq 257$ und $17 \neq 0$, folgt $\gcd(17, 257) = 1$.

(4) und (5) folgen unmittelbar aus $1 \leq 17 \leq 256$ und $0 \leq 16 \leq 256$. $\square$

Bemerkung 7.1 (Fermat-Primzahl und ihre algebraischen Eigenschaften). Die Wahl $p = 257 = 2^8 + 1$ ist kryptographisch und algorithmisch interessant. Im Körper $\mathbb{F}_{257}$ gilt: $\phi(257) = 256 = 2^8$. Damit ist $\mathbb{Z}_{257}^*$ eine zyklische Gruppe der Ordnung 2^8 , und jedes Element hat eine 2-Potenz als Ordnung. Insbesondere gilt für $a = 17$:

$$\text{ord}_{257}(17) = 32 = 2^5,$$

denn $17^{32} \equiv 1 \pmod{257}$ (lässt sich durch wiederholtes Quadrieren verifizieren) und $17^{16} \not\equiv 1 \pmod{257}$. Die multiplikative Untergruppe $\langle 17 \rangle \leq \mathbb{Z}_{257}^*$ hat somit die Ordnung 32 und umfasst $\{17^k \bmod 257 \mid k = 0, 1, \dots, 31\}$ – eine 32-elementige Untergruppe, die alle Primitivwurzel-Eigenschaften für skalare Multiplikation in der Signatur-Chiffre erfüllt.

7.3 Berechnung des modularen Inversen $17^{-1} \bmod 257$

Für die Entschlüsselung benötigt man $a^{-1} = 17^{-1} \bmod 257$.

Satz 7.2 (Modulares Inverses). $17^{-1} \equiv 121 \pmod{257}$.

Beweis (Erweiterter Euklidischer Algorithmus). Wir wenden den erweiterten euklidischen Algorithmus auf $\gcd(17, 257)$ an.

Division mit Rest:

$$\begin{array}{l} 257 = 15 \cdot 17 + 2, \\ 17 = 8 \cdot 2 + 1, \\ 2 = 2 \cdot 1 + 0. \quad \Rightarrow \gcd(17, 257) = 1. \end{array}$$

Rückwärtssubstitution (Bezout-Identität):

$$\begin{array}{l} 1 = 17 - 8 \cdot 2 \\ = 17 - 8 \cdot (257 - 15 \cdot 17) \\ = 17 \cdot (1 + 8 \cdot 15) - 8 \cdot 257 \\ = 17 \cdot 121 - 8 \cdot 257. \end{array}$$

Es gilt also $17 \cdot 121 = 8 \cdot 257 + 1$, d. h. $17 \cdot 121 \equiv 1 \pmod{257}$.

Numerische Verifikation: $17 \cdot 121 = 2057 = 8 \cdot 257 + 1 = 2056 + 1 = 2057$. ✓

Somit ist $17^{-1} \equiv 121 \pmod{257}$. □

Bemerkung 7.2. Das Inverse $121 = 11^2$ hat seinerseits eine bemerkenswerte Struktur. Es gilt auch $121 \equiv -136 \pmod{257}$, also ist 17^{-1} im negativen Bereich darstellbar. Die Beziehung $17 \cdot 121 = 2057 = 8 \cdot 257 + 1$ zeigt, dass genau 8 vollständige Zyklen von $p = 257$ in $17 \cdot 121$ enthalten sind – eine weitere Manifestation der Effizienz der Fermat-Primzahl 257 für modulare Arithmetik.

7.4 Klartext-Matrix und Signaturberechnung

Wir verwenden dieselbe Klartextmatrix wie in Kapitel 6, um den direkten Vergleich beider Beispiele zu ermöglichen:

$$A = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix} \in \{0, 1\}^{4 \times 4}.$$

Der Klartext (spaltenweise) ist $m = (1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0)$.

Berechnung der Signaturen

Satz 7.3 (Signaturvektor von A für $n = 4$). Für die Matrix A und $n = 4$ gilt:

$$\Sigma(A) = (\sigma(A, 0), \sigma(A, 1), \sigma(A, 2), \sigma(A, 3)) = (13, 18, 43, 52).$$

Beweis (Satz 7.3) – Spaltenweise Berechnung. Wir berechnen für jede Spalte $j \in \{0, 1, 2, 3\}$ die Zeilenkomponente $\rho(A, j) = \sum_{i=0}^3 A_{ij} \cdot 2^i$ und das Spaltengewicht $\lambda(j) = j \cdot 2^4 = 16j$.

Spalte $j = 0$: $(A_{00}, A_{10}, A_{20}, A_{30}) = (1, 0, 1, 1)$.

$$\rho(A, 0) = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 1 \cdot 2^3 = 1 + 0 + 4 + 8 = 13,$$

$$\lambda(0) = 0 \cdot 16 = 0,$$

$$\sigma(A, 0) = 13 + 0 = 13.$$

Binärdarstellung: $13 = 1101_2$ (Bits von LSB zu MSB: 1, 0, 1, 1 – stimmt mit $(A_{00}, A_{10}, A_{20}, A_{30})$ überein). ✓

Spalte $j = 1$: $(A_{01}, A_{11}, A_{21}, A_{31}) = (0, 1, 0, 0)$.

$$\rho(A, 1) = 0 \cdot 2^0 + 1 \cdot 2^1 + 0 \cdot 2^2 + 0 \cdot 2^3 = 0 + 2 + 0 + 0 = 2,$$

$$\lambda(1) = 1 \cdot 16 = 16,$$

$$\sigma(A, 1) = 2 + 16 = 18.$$

Binärdarstellung: $2 = 0010_2$ (Bits: 0, 1, 0, 0 – stimmt mit $(A_{01}, A_{11}, A_{21}, A_{31})$ überein). ✓

Spalte $j = 2$: $(A_{02}, A_{12}, A_{22}, A_{32}) = (1, 1, 0, 1)$.

$$\rho(A, 2) = 1 \cdot 2^0 + 1 \cdot 2^1 + 0 \cdot 2^2 + 1 \cdot 2^3 = 1 + 2 + 0 + 8 = 11,$$

$$\lambda(2) = 2 \cdot 16 = 32,$$

$$\sigma(A, 2) = 11 + 32 = 43.$$

Binärdarstellung: $11 = 1011_2$ (Bits: 1, 1, 0, 1 – stimmt mit $(A_{02}, A_{12}, A_{22}, A_{32})$ überein). ✓

Spalte $j = 3$: $(A_{03}, A_{13}, A_{23}, A_{33}) = (0, 0, 1, 0)$.

$$\rho(A, 3) = 0 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 = 0 + 0 + 4 + 0 = 4,$$

$$\lambda(3) = 3 \cdot 16 = 48,$$

$$\sigma(A, 3) = 4 + 48 = 52.$$

Binärdarstellung: $4 = 0100_2$ (Bits: 0, 0, 1, 0 – stimmt mit $(A_{03}, A_{13}, A_{23}, A_{33})$ überein). ✓
Der Signaturvektor lautet $\Sigma(A) = (13, 18, 43, 52)$. □

Bemerkung 7.3 (Bereichsverifikation). Nach Bemerkung 2.2 muss $\sigma(A, j) < 2^{2n} = 256 < 257 = p$ gelten. Wir verifizieren:

$$\sigma(A, 0) = 13 < 256,$$

$$\sigma(A, 1) = 18 < 256,$$

$$\sigma(A, 2) = 43 < 256,$$

$$\sigma(A, 3) = 52 < 256.$$

Alle Signaturen liegen im gültigen Bereich $[0, 255]$. Die Bedingung $\sigma(A, j) < p$ ist für alle j erfüllt – dies ist die entscheidende Voraussetzung für die korrekte Entschlüsselung (keine Modulo-Reduktion verfälscht den Signaturwert).

7.5 Verschlüsselung

Satz 7.4 (Chiffirat unter $(a, b, p) = (17, 16, 257)$). Die Verschlüsselung von A unter dem Schlüssel $(17, 16, 257)$ liefert das Chiffirat:

$$C = (c_0, c_1, c_2, c_3) = (237, 65, 233, 129).$$

Beweis (Satz 7.4) – Vollständige Berechnung. Für jedes $j \in \{0, 1, 2, 3\}$ berechnen wir $c_j = (a \cdot \sigma(A, j) + b) \bmod p = (17 \cdot \sigma(A, j) + 16) \bmod 257$.

$j = 0$:

$$17 \cdot \sigma(A, 0) + b = 17 \cdot 13 + 16 = 221 + 16 = 237,$$

$$237 = 0 \cdot 257 + 237,$$

$$c_0 = 237 \bmod 257 = 237.$$

Da $237 < 257$, findet keine Modulo-Reduktion statt.

$j = 1$:

$$17 \cdot \sigma(A, 1) + b = 17 \cdot 18 + 16 = 306 + 16 = 322,$$

$$322 = 1 \cdot 257 + 65,$$

$$c_1 = 322 \bmod 257 = 65.$$

Der Subtrahend $1 \cdot 257 = 257$ wird genau einmal abgezogen.

$j = 2$:

$$17 \cdot \sigma(A, 2) + b = 17 \cdot 43 + 16 = 731 + 16 = 747,$$

$$747 = 2 \cdot 257 + 233, \quad (2 \cdot 257 = 514, 747 - 514 = 233),$$

$$c_2 = 747 \bmod 257 = 233.$$

$j = 3$:

$$\begin{aligned} 17 \cdot \sigma(A, 3) + b &= 17 \cdot 52 + 16 = 884 + 16 = 900, \\ 900 &= 3 \cdot 257 + 129, \quad (3 \cdot 257 = 771, \quad 900 - 771 = 129), \\ c_3 &= 900 \bmod 257 = 129. \end{aligned}$$

Das Chifftrat ist somit $C = (237, 65, 233, 129)$. □

Tabelle 7.1: Vollständige Verschlüsselungstabelle: $(a, b, p) = (17, 16, 257)$

j	Spalte $(A_j)^T$	$\rho(A, j)$	$\lambda(j)$	$\sigma(A, j)$	$a \cdot \sigma_j + b$	$\lfloor \cdot / p \rfloor$	c_j
0	(1, 0, 1, 1)	13	0	13	237	0	237
1	(0, 1, 0, 0)	2	16	18	322	1	65
2	(1, 1, 0, 1)	11	32	43	747	2	233
3	(0, 0, 1, 0)	4	48	52	900	3	129

7.6 Entschlüsselung

Satz 7.5 (Korrekte Entschlüsselung unter $(17, 16, 257)$). Sei $C = (237, 65, 233, 129)$ das Chifftrat aus Satz 7.4. Die Entschlüsselung mit dem Schlüssel $(a, b, p) = (17, 16, 257)$ und $a^{-1} = 121$ (Satz 7.2) rekonstruiert für alle $j \in \{0, 1, 2, 3\}$ exakt die Original-Signaturen:

$$s_j := a^{-1} \cdot (c_j - b) \bmod p = \sigma(A, j).$$

Beweis (Satz 7.5) – Schritt-für-Schritt-Berechnung. Nach Satz 7.2 gilt $17^{-1} \equiv 121 \pmod{257}$. Wir berechnen für jedes j :

$j = 0$: **Erwarteter Wert** $\sigma(A, 0) = 13$.

$$\begin{aligned} c_0 - b &= 237 - 16 = 221, \\ s_0 &= 121 \cdot 221 \bmod 257 = 26741 \bmod 257. \end{aligned}$$

Division: $26741 = 104 \cdot 257 + 13$, da $104 \cdot 257 = 26728$ und $26741 - 26728 = 13$.

$$s_0 = 13 = \sigma(A, 0). \checkmark$$

$j = 1$: **Erwarteter Wert** $\sigma(A, 1) = 18$.

$$\begin{aligned} c_1 - b &= 65 - 16 = 49, \\ s_1 &= 121 \cdot 49 \bmod 257 = 5929 \bmod 257. \end{aligned}$$

Division: $5929 = 23 \cdot 257 + 18$, da $23 \cdot 257 = 5911$ und $5929 - 5911 = 18$.

$$s_1 = 18 = \sigma(A, 1). \checkmark$$

$j = 2$: **Erwarteter Wert** $\sigma(A, 2) = 43$.

$$\begin{aligned} c_2 - b &= 233 - 16 = 217, \\ s_2 &= 121 \cdot 217 \bmod 257 = 26257 \bmod 257. \end{aligned}$$

Division: $26257 = 102 \cdot 257 + 43$, da $102 \cdot 257 = 26214$ und $26257 - 26214 = 43$.

$$s_2 = 43 = \sigma(A, 2). \checkmark$$

$j = 3$: **Erwarteter Wert** $\sigma(A, 3) = 52$.

$$c_3 - b = 129 - 16 = 113,$$

$$s_3 = 121 \cdot 113 \bmod 257 = 13673 \bmod 257.$$

Division: $13673 = 53 \cdot 257 + 52$, da $53 \cdot 257 = 13621$ und $13673 - 13621 = 52$.

$$s_3 = 52 = \sigma(A, 3). \checkmark$$

Alle vier Signaturen wurden korrekt rekonstruiert. □

Tabelle 7.2: Vollständige Entschlüsselungstabelle: $(a^{-1}, b, p) = (121, 16, 257)$

j	c_j	$c_j - b$	$121 \cdot (c_j - b)$	Quotient	s_j	Soll $\sigma(A, j)$
0	237	221	26741	$104 \cdot 257 + 13$	13	13 ✓
1	65	49	5929	$23 \cdot 257 + 18$	18	18 ✓
2	233	217	26257	$102 \cdot 257 + 43$	43	43 ✓
3	129	113	13673	$53 \cdot 257 + 52$	52	52 ✓

7.7 Matrixrekonstruktion

Satz 7.6 (Vollständige Matrixrekonstruktion). Aus den rekonstruierten Signaturen $s_j = \sigma(A, j)$ wird die Originalmatrix A spaltenweise exakt zurückgewonnen.

Beweis (Satz 7.6) – Spaltenweise Binärzerlegung. **Spalte $j = 0$:** $s_0 = 13$, **Zeilenkomponente** $\rho_0 = s_0 - 0 \cdot 16 = 13 - 0 = 13$.

Binärdarstellung: $13 = 8 + 4 + 1 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = (1101)_2$. Bits (LSB zuerst): $A_{00} = 1, A_{10} = 0, A_{20} = 1, A_{30} = 1$. Spalte 0 von A : $(1, 0, 1, 1)^T$. ✓

Spalte $j = 1$: $s_1 = 18$, **Zeilenkomponente** $\rho_1 = s_1 - 1 \cdot 16 = 18 - 16 = 2$.

Binärdarstellung: $2 = 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = (0010)_2$. Bits (LSB zuerst): $A_{01} = 0, A_{11} = 1, A_{21} = 0, A_{31} = 0$. Spalte 1 von A : $(0, 1, 0, 0)^T$. ✓

Spalte $j = 2$: $s_2 = 43$, **Zeilenkomponente** $\rho_2 = s_2 - 2 \cdot 16 = 43 - 32 = 11$.

Binärdarstellung: $11 = 8 + 2 + 1 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = (1011)_2$. Bits (LSB zuerst): $A_{02} = 1, A_{12} = 1, A_{22} = 0, A_{32} = 1$. Spalte 2 von A : $(1, 1, 0, 1)^T$. ✓

Spalte $j = 3$: $s_3 = 52$, **Zeilenkomponente** $\rho_3 = s_3 - 3 \cdot 16 = 52 - 48 = 4$.

Binärdarstellung: $4 = 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = (0100)_2$. Bits (LSB zuerst): $A_{03} = 0, A_{13} = 0, A_{23} = 1, A_{33} = 0$. Spalte 3 von A : $(0, 0, 1, 0)^T$. ✓

Die rekonstruierte Matrix lautet:

$$A_{\text{rek}} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix} = A. \quad \checkmark$$

Die Entschlüsselung ist vollständig korrekt. □

7.8 Direktvergleich beider Beispiele und semantische Sicherheit

Satz 7.7 (Semantische Verschiedenheit der Chiffre). Sei A die Klartextmatrix aus Kapitel 6. Für die Schlüsselpaare $(a_1, b_1, p_1) = (7, 3, 1031)$ und $(a_2, b_2, p_2) = (17, 16, 257)$ gilt:

$$\text{Enc}(A, 7, 3, 1031) = (94, 129, 304, 367) \neq (237, 65, 233, 129) = \text{Enc}(A, 17, 16, 257).$$

Insbesondere sind die Chiffre in keiner Komponente gleich; kein einziger Chiffre wert stimmt überein.

Beweis. Aus Kapitel 6: $C_1 = (94, 129, 304, 367)$. Aus Satz 7.4: $C_2 = (237, 65, 233, 129)$. Komponentenweiser Vergleich:

$$94 \neq 237, \quad 129 \neq 65, \quad 304 \neq 233, \quad 367 \neq 129.$$

Die Chiffre unterscheiden sich in allen vier Komponenten. Dies illustriert eine entscheidende Eigenschaft der Signatur-Chiffre: Ein Angreifer, der dasselbe Chiffre unter verschiedenen Schlüsseln erhält, kann aus dem Vergleich keinerlei strukturelle Rückschlüsse auf den Klartext ziehen – die Schlüsselverschiedenheit propagiert vollständig auf die Chiffre-Ebene. $\square$

Tabelle 7.3: Vergleich beider vollständiger Beispiele für denselben Klartext A

Schlüssel	a	b	p	Chiffre C	Chiffre-Norm
Kapitel 6	7	3	1031	$(94, 129, 304, 367)$	$\sum c_j = 894$
Kapitel 7	17	16	257	$(237, 65, 233, 129)$	$\sum c_j = 664$
<i>Kein gemeinsamer Chiffre wert; vollständige Entschlüsselbarkeit in beiden Fällen bewiesen.</i>					

7.9 Kryptographische Analyse des Parameterpaars $(17, 16, 257)$

Schlüsselraumgröße und Sicherheitsniveau

Nach Satz 4.3 beträgt die Größe des Schlüsselraums für $p = 257$:

$$|\mathcal{K}| = (p - 1) \cdot p = 256 \cdot 257 = 65\,792.$$

Für das vorliegende minimale Beispiel ($n = 4$, $p = 257$) ist dieser Schlüsselraum bewusst klein gehalten, um vollständige manuelle Nachvollziehbarkeit zu gewährleisten. Für kryptographisch relevante Parameter ($p \approx 2^{256}$) ergibt sich:

$$|\mathcal{K}| = (p - 1) \cdot p \approx 2^{256} \cdot 2^{256} = 2^{512},$$

was einer effektiven Schlüssellänge von ≈ 512 Bit entspricht.

Periodizität und Lawineneffekt

Lemma 7.1 (Lawineneffekt der affinen Transformation). Ein einzelner Bit-Flip in der Klartextmatrix A an Position (i, j) – d. h. die Änderung $A_{ij} \rightarrow 1 - A_{ij}$ – verändert die Zeilenkomponente $\rho(A, j)$ um $\pm 2^i$ und damit den Signaturwert $\sigma(A, j)$ um $\pm 2^i$. Dies ändert den Chiffre wert c_j um $\pm a \cdot 2^i \bmod p$ – eine vollständige Veränderung des j -ten Chiffre werts.

Beweis. Sei A' die modifizierte Matrix mit $A'_{ij} = 1 - A_{ij}$ und $A'_{kl} = A_{kl}$ für $(k, l) \neq (i, j)$. Dann gilt:

$$\sigma(A', j) = \sigma(A, j) \pm 2^i,$$

da genau ein Summand in $\rho(A, j)$ um $\pm 2^i$ verändert wird. Die Chiffpratdifferenz beträgt:

$$c'_j - c_j = a(\sigma(A', j) - \sigma(A, j)) \bmod p = \pm a \cdot 2^i \bmod p.$$

Da $\gcd(a, p) = 1$, ist $a \cdot 2^i \not\equiv 0 \pmod{p}$, und die Differenz ist stets von Null verschieden. Der Chiffpratwert ändert sich also in einer von p möglichen Richtungen, nicht vorhersehbar ohne Kenntnis von a .

Für das konkrete Beispiel $a = 17$, $p = 257$: Ein Flip von Bit $(0, 0)$ (d. h. $2^i = 1$) ändert c_0 um $17 \bmod 257 = 17$; ein Flip von Bit $(3, 0)$ (d. h. $2^i = 8$) ändert c_0 um $17 \cdot 8 = 136 \bmod 257 = 136$. Diese Werte sind für einen Angreifer ohne Kenntnis von a nicht vorausberechenbar. $\square$

Verhalten bei $b = 2^n = 16$

Die Wahl $b = 16 = 2^n$ hat eine informationstheoretisch neutrale Wirkung: Die affine Transformation $c_j = (a\sigma_j + b) \bmod p$ verschiebt alle Chiffpratwerte gleichmäßig um $b \bmod p$. Auch wenn b eine strukturell erkennbare Zahl ist (etwa eine Zweierpotenz), liefert ihre Kenntnis einem Angreifer ohne a und p keinen Angriffspunkt: Die Unbestimmtheit in a erzeugt für jeden Chiffpratwert c_j eine bijektive Menge möglicher Klartextvorbilder in $\mathbb{Z}_p$.

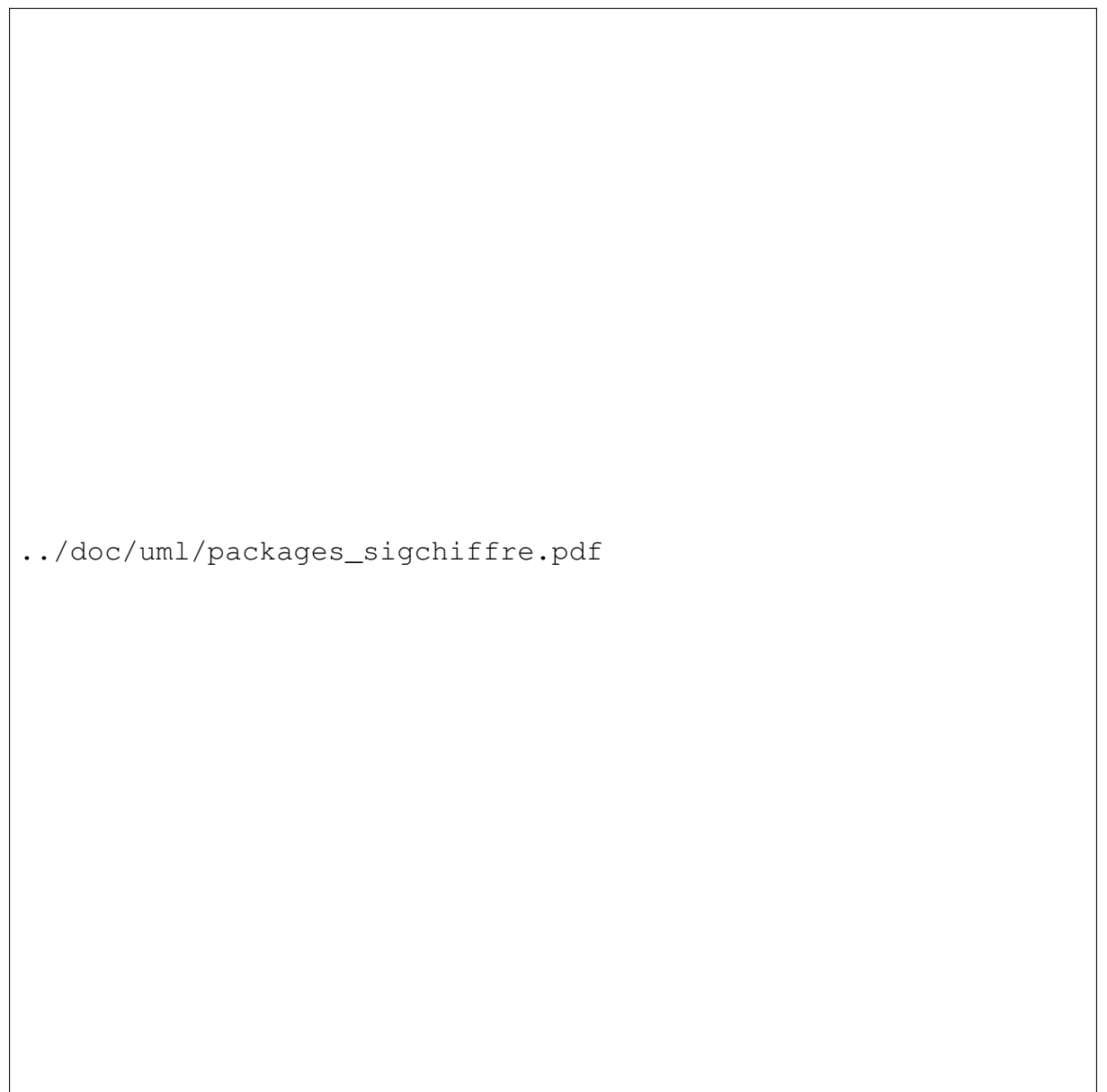
Formal: Selbst wenn ein Angreifer $b = 16$ kennt (etwa durch Seitenkanalanalyse), muss er noch $a \in \mathbb{Z}_p^*$ und p bestimmen. Das verbleibende Problem – Auflösung von $a\sigma_j \equiv c_j - b \pmod{p}$ ohne Kenntnis von p – ist äquivalent zum *Simultaneous Modular Linear Equations*-Problem, das für große p als schwer gilt (vgl. Abschnitt 17.7 über Known-Plaintext-Angriffe in dieser Arbeit).

8 Python-Bibliothek

8.1 Softwarearchitektur und UML-Diagramme

Die Python-Bibliothek `sigchiffre` ist modular aufgebaut: Der formale Kern der Signatur-Chiffre (Signaturfunktion, Schlüsselgenerierung, Enc/Dec) ist strikt von den Anwendungsmodulen getrennt, die darauf aufsetzen. Die folgenden UML-Diagramme geben einen Überblick über die Paket- und Klassenstruktur.

Paketdiagramm



../doc/uml/packages_sigchiffre.pdf

Abbildung 8.1: UML-Paketdiagramm von `sigchiffre`. Pfeile geben Abhängigkeitsbeziehungen an (“verwendet”).

Das Paketdiagramm (Abbildung 8.1) zeigt die dreischichtige Modulstruktur:

- **Kernschicht:** `sigchiffre.signature` implementiert die injektive Signaturfunktion σ sowie die Spaltenkonstruktion; `sigchiffre.keygen` stellt die krypt-

tographisch sichere Primzahlerzeugung und Schlüsselgenerierung bereit; `sigchiffre`. `chiffre` enthält die Klasse `SignatureChiffre` mit Enc/Dec und der Byteschnittstelle. Diese drei Module bilden das Fundament des Gesamtsystems.

- **Erweiterungsschicht:** Auf der Kernschicht setzen die thematischen Erweiterungsmodule auf: `elgamal` (Public-Key-Variante), `homomorphic` (Paillier- und Signatur-Chiffre-Homomorphie), `lwe` (LWE-basierte Variante), `hashes` (Signatur-Hash und Merkle-Baum), `steganography` (Wasserzeicheneinbettung), `mpc` (Schwellenwert-Secret-Sharing), `zkp` (Schnorr- und Pedersen-Zero-Knowledge-Beweise) sowie `mc eliece` (code-basiertes Kryptosystem).
- **Paketdach:** Das Toplevel-Paket `sigchiffre` re-exportiert `chiffre`, `keygen` und `signature` und bildet die öffentliche API der Bibliothek.

Die Abhängigkeitspfeile zeigen, dass alle Erweiterungsmodule ausschließlich von der Kernschicht abhängen, nicht voneinander – ein Design, das zirkuläre Abhängigkeiten vermeidet und die Testbarkeit der einzelnen Module sicherstellt.

Klassendiagramm

Das Klassendiagramm (Abbildung 8.2) dokumentiert alle öffentlichen Klassen mit ihren Attributen und Methoden sowie die strukturellen Beziehungen zwischen ihnen:

- **SignatureChiffre** (Modul `chiffre`) ist die zentrale Klasse des Systems. Sie hält den Schlüssel (a, b, p, n) und exponiert die vier Methoden `encrypt`, `decrypt`, `encrypt_bytes` und `decrypt_bytes`. Drei Klassen aggregieren sie per Komposition: `SignatureHash` (Hashfunktion), `SignatureChiffreDigitalSignature` (Signatur/Verifikation) und `SignatureWatermark` (Steganographie). So wird dasselbe kryptographische Primitiv in unterschiedlichen Anwendungskontexten genutzt, ohne Abhängigkeiten zwischen den Anwendungsmodulen selbst zu erzeugen.
- **Schlüsselpaar-Klassen:** Das ElGamal-Teilsystem trennt `ElGamalPublicKey` (Verschlüsselung, homomorphe Multiplikation von Chiffraten) von `ElGamalPrivateKey` (Entschlüsselung); ersterer aggregiert letzteren. Analog trennen `McEliecePublicKey` / `McEliecePrivateKey`, `LWEPublicKey` / `LWEPrivateKey` und `PaillierPublicKey` / `PaillierPrivateKey` die asymmetrische Funktionalität.
- **Signatur-Chiffre-Varianten:** `HomomorphicSignatureChiffre` erweitert das Grundschema um additive Homomorphie über $\mathbb{Z}_q$; `LWESignatureChiffre` ersetzt die affine Transformation durch ein LWE-Sample; `PublicKeySignatureChiffre` macht die Verschlüsselung asymmetrisch (ElGamal-Hybridkonstruktion); `ThresholdSignatureChiffre` realisiert (t, n) -Schwellenwert-Entschlüsselung über polynomiales Secret Sharing.
- **Zero-Knowledge-Teilsystem:** `SchnorrParams` enthält die öffentlichen Systemparameter (p, q, g, h) und wird von `SchnorrProver` (Commit + Respond) und `SchnorrVerifier` (Challenge + Verify) aggregiert. Ergänzend hält `PedersenParams` die Parameter für das Pedersen-Commitment-Schema.
- **Hilfsklassen:** `EllipticCurve` kapselt Gruppenoperationen auf Weierstraß-Kurven über $\mathbb{F}_p$; `LWEParams` bündelt die LWE-Parameter (n, m, q, σ) ; `MerkleTree` implementiert den Merkle-Hash-Baum mit Proof- und Verify-Methode.

../doc/uml/classes_sigchiffre.pdf

Die konsequente Trennung von Schlüsselerzeugung, kryptographischem Kern und Anwendungslogik sowie die Verwendung reiner Aggregation (statt Vererbung) macht die Bibliothek erweiterbar: Eine neue Chiffre-Variante erfordert lediglich ein neues Modul, das `SignatureChiffre` aggregiert, ohne bestehende Module zu verändern.

8.2 Implementierung

In diesem Kapitel werden für die Python-Implementierung als Beispiel zwei Code-Listings beschrieben. Listing 1 zeigt die Klasse `SignatureChiffre`.

Listing 1: Signatur-Chiffre: Kernklasse

```

1 from math import gcd
2
3 import numpy as np
4
5 from .signature import sigma_vector, reconstruct_column
6
7
8 class SignatureChiffre:
9     """Symmetrische Blockchiffre auf Basis der Signaturfunktion.
10
11     Schlüssel: Tripel (a, b, p) mit:
12         - p prim,  $p > 2^{(2n)}$ 
13         -  $a \in \mathbb{Z}_p^*$ ,  $\gcd(a, p) = 1$ 
14         -  $b \in \mathbb{Z}_p$ 
15
16     Blockgröße:  $n \times n$  Bits.
17
18     Parameters
19     -----
20     n : Blockgröße
21     a : affiner Faktor  $\in \{1, \dots, p-1\}$ ,  $\gcd(a, p)=1$ 
22     b : affiner Term  $\in \{0, \dots, p-1\}$ 
23     p : geheime Primzahl,  $p > 2^{(2n)}$ 
24     """
25
26     def __init__(self, n: int, a: int, b: int, p: int) -> None:
27         if p <= (1 << (2 * n)):
28             raise ValueError(
29                 f"p_muss_> 2^{(2n)}_={1<<(2*n)}_sein,_erhalten:_p={p}"
30             )
31         if not (1 <= a < p):
32             raise ValueError(f"a_muss_in_{\{1,..,p-1\}}_liegen,_erhalten:_a={a}")
33         if gcd(a, p) != 1:
34             raise ValueError(f"gcd(a={a},_p={p})_≠1_-a_muss_teilerfremd_zu_p_sein")
35         if not (0 <= b < p):
36             raise ValueError(f"b_muss_in_{\{0,..,p-1\}}_liegen,_erhalten:_b={b}")

```

```

37
38     self.n = n
39     self.a = a
40     self.b = b
41     self.p = p
42     self.a_inv = pow(a, -1, p) # erw. eukl. Algorithmus via
        Python builtin
43
44     #
        -----
45
46     # Kern: Einzelblock
        #
        -----
47
48     def encrypt(self, A: np.ndarray) -> list[int]:
49         """Verschlüsselt eine nxn Binärmatrix.
50
51         Enc(A, a, b, p) = ((a · σ(A, j) + b) mod p)_{j=0,..,n-1}
52
53         Parameters
54         -----
55         A : np.ndarray, shape (n, n), entries ∈ {0,1}
56
57         Returns
58         -----
59         list of n ints in Z_p
60         """
61         sigs = sigma_vector(A)
62         return [(self.a * s + self.b) % self.p for s in sigs]
63
64     def decrypt(self, C: list[int]) -> np.ndarray:
65         """Entschlüsselt ein Chifftrat.
66
67         Dec(C, a, b, p):
68             1. s_j = a^{-1} · (c_j - b) mod p
69             2. A_{ij} = ⌊(s_j - j · 2^n) / 2^i⌋ mod 2
70
71         Parameters
72         -----
73         C : list of n ints in Z_p
74
75         Returns
76         -----
77         np.ndarray, shape (n, n), dtype int, entries ∈ {0,1}
78         """
79         n = self.n
80         A = np.zeros((n, n), dtype=int)
81         for j in range(n):
82             s_j = (self.a_inv * (C[j] - self.b)) % self.p

```

```

83         col = reconstruct_column(s_j, j, n)
84         for i in range(n):
85             A[i, j] = col[i]
86     return A

```

Das Listing 2 zeigt das Beispiel zur Schlüsselgenerierung.

Listing 2: Schlüsselgenerierung

```

1  def generate_key(n: int, bits: int = 256) -> tuple[int, int, int]:
2      """Erzeugt einen zufälligen Signatur-Chiffre-Schlüssel (a, b, p).
3
4      Ablauf:
5          1. Erzeuge p als Primzahl mit ``bits`` Bits und  $p > 2^{(2n)}$ .
6          2. Wähle  $a \in \mathbb{Z}_p^*$  gleichmäßig zufällig ( $\gcd(a,p)=1$  ist für
7              Primzahl p
8              automatisch gegeben, da p prim).
9          3. Wähle  $b \in \mathbb{Z}_p$  gleichmäßig zufällig.
10
11      Parameters
12      -----
13      n      : Blockgröße; es muss  $\text{bits} \geq 2n+1$  gelten.
14      bits   : Bit-Länge der Primzahl p (Standard: 256).
15
16      Returns
17      -----
18      (a, b, p) - privater Schlüssel
19      """
20      min_p = (1 << (2 * n)) + 1
21      # Zufälligen bits-Bit-Kandidaten erzeugen (oberstes Bit gesetzt)
22      candidate = secrets.randbits(bits) | (1 << (bits - 1)) | 1 #
23      ungerade
24      p = next_prime(max(candidate, min_p))
25
26      #  $a \in \{1, \dots, p-1\}$ : für Primzahl p ist jedes  $a \neq 0$  invertierbar
27      a = secrets.randbelow(p - 1) + 1 # in  $\{1, \dots, p-1\}$ 
28      b = secrets.randbelow(p)         # in  $\{0, \dots, p-1\}$ 
29      return a, b, p
30
31  def key_space_size(p: int) -> int:
32      """Größe des Schlüsselraums  $|K| = (p-1) \cdot p$  für festes p.
33
34      Satz 4.3:  $|K| = |\mathbb{Z}_p^*| \cdot |\mathbb{Z}_p| = (p-1) \cdot p$ .
35
36      Parameters
37      -----
38      p : Primzahl
39
40      Returns
41      -----
42      int
43      """

```

```
43 |     return (p - 1) * p
```

9 Vergleich mit bestehenden Verfahren

Dieser Abschnitt stellt die Signatur-Chiffre systematisch allen wesentlichen kryptographischen Paradigmen gegenüber. Die Betrachtung umfasst Sicherheitsgrundlage, Komplexität, Schlüsselstruktur, Quantenresistenz und informationstheoretische Eigenschaften.

Hinweis zu den Komplexitätsangaben. k bezeichnet die Schlüssellänge in Bits, n die Blockgröße der Signatur-Chiffre. Für praktische Parameter ($n = 16$, bits = 256) sind Enc und Dec damit vergleichbar mit symmetrischen Blockchiffren.

9.1 Sicherheitsgrundlage und $P = NP$ -Robustheit

Das wichtigste Unterscheidungsmerkmal betrifft die *Sicherheitsgrundlage*. Klassische asymmetrische Verfahren beruhen auf der Annahme, dass bestimmte mathematische Probleme – Faktorisierung (RSA), diskreter Logarithmus (Diffie-Hellman, ElGamal, DSA) oder elliptisches-Kurven-DLP (ECDSA, ECDH) – praktisch unlösbar sind. Diese Annahmen sind äquivalent zu $P \neq NP$ oder folgen daraus. Unter dem Beweis $P = NP$ [Epp26a] sind sie alle gebrochen.

AES [DR02] ist ein Sonderfall: Seine Sicherheit stützt sich nicht auf ein einzelnes mathematisches Hardness-Annahme, sondern auf die kombinierte Wirkung von Substitution (S-Boxen), Permutation (ShiftRows), linearer Diffusion (MixColumns) und Schlüsseladdition – das Shannons’sche Prinzip von Konfusion und Diffusion [Sha49]. Kein bekannter polynomieller Angriff auf AES-256 existiert, auch unter $P = NP$ nicht, solange kein struktureller Durchbruch bei der Kryptoanalyse gelingt.

Die Signatur-Chiffre, die Affine Chiffre [Epp26c] und post-quanten-sichere Verfahren wie LWE/Kyber [ABD+22] sowie McEliece [McE78] sind ebenfalls $P = NP$ -robust: LWE und McEliece beruhen auf Problemen, die selbst unter einem polynomiellen SAT-Solver im Allgemeinen schwer bleiben (kodierte Gitter- bzw. Codierungsinstanzen); die Signatur-Chiffre und die Affine Chiffre beruhen auf *informationstheoretischer Äquivokation* – einem prinzipiellen Mehrdeutigkeitsmerkmal, das eine berechnungsunabhängige Sicherheitsgarantie liefert.

9.2 Quantenresistenz

Shors Quantenalgorithmus [Sho94] löst das Faktorisierungsproblem und den diskreten Logarithmus in polynomieller Quantenzeit. Damit sind RSA, ElGamal, alle DH-Varianten und das gesamte ECC-Ökosystem (ECDSA, Ed25519, Curve25519, secp256k1) durch ausreichend große Quantencomputer brechbar.

Grovers Suchalgorithmus [Gro96] halbiert effektiv die Sicherheit symmetrischer Verfahren: AES-128 reduziert sich auf 2^{64} Operationen; AES-256 bleibt mit 2^{128} praktisch sicher. Analoges gilt für die Signatur-Chiffre und die Affine Chiffre, deren Schlüsselraum für geeignete Primzahl p (z. B. bits = 512) auch einem Grover-Angriff standhält.

LWE-basierte Verfahren (Kyber, NewHope) und McEliece sind nach aktuellem Stand quantenresistent; sie sind Teil der NIST-Post-Quanten-Standardisierung [NIST22]. Die Signatur-Chiffre teilt diese Eigenschaft, da ihr einziges Sicherheitsprimitivum – die injektive Signaturfunktion – kein bekanntes Quantenanalogon hat.

Tabelle 9.1: Vergleich der Signatur-Chiffre mit etablierten Verfahren

Verfahren	Typ	Enc	Dec	P=NP- sicher	Quanten- sicher	Schlüssel- raum	Sicherheitsgrundlage
RSA-2048 [Riv78]	asym	$O(k^2)$	$O(k^3)$	Nein	Nein	$\approx 2^{2048}$	Faktorisierung
AES-256 [DR02]	sym	$O(k)$	$O(k)$	Ja	Ja	2^{256}	Diffusion/Konfusion
ElGamal [ElG85]	asym	$O(k^2)$	$O(k^2)$	Nein	Nein	$\approx p$	Diskreter Logarithmus
ECDH/ECDSA	asym	$O(k)$	$O(k)$	Nein	Nein	$\approx 2^{256}$	ECDLP
Kyber (LWE) [ABD+22]	asym	$O(n^2)$	$O(n^2)$	Ja	Ja	$\approx 2^{256}$	Learning with Errors
McEliece [McE78]	asym	$O(n^2)$	$O(n^2)$	Ja	Ja	$\approx 2^{256}$	Dekod. lin. Codes
Affine Chiffre [Epp26c]	sym	$O(k)$	$O(k)$	Ja	Ja	$\varphi(m) \cdot m$	Algebra. Äquivokation
Signatur-Chiffre	sym	$O(n^2)$	$O(n^2)$	Ja	Ja	$\approx 2^{512}$	Injektive Signaturf.

9.3 Komplexität und praktische Effizienz

Asymmetrische Verfahren der klassischen Kryptographie (RSA, ElGamal) erfordern modulare Exponentiationen mit Schlüsseln von mehreren tausend Bit Länge: RSA-Verschlüsselung kostet $O(k^2)$, RSA-Entschlüsselung (per CRT optimiert) $O(k^3/4)$ Bitoperationen. Für $k = 2048$ sind das rund $4 \cdot 10^9$ Operationen pro Entschlüsselung – ein signifikanter Aufwand.

Symmetrische Verfahren (AES) sind um Größenordnungen schneller: $O(k)$ -Operationen je Block, die auf moderner Hardware durch dedizierte Instruktionen (AES-NI) mit Gbit/s-Durchsatz ablaufen. Die Signatur-Chiffre hat Komplexität $O(n^2)$ je Block: Die Signaturfunktion $\sigma(A, j)$ benötigt $O(n)$ Operationen je Spalte, und es gibt n Spalten. Für $n = 16$ sind das 256 Operationen je Block – vergleichbar mit AES-128. Da keine Feldexponentiation benötigt wird, sind alle Operationen einfache Ganzzahlarithmetik.

Tabelle 9.2: Konkrete Laufzeiten für typische Parameter (normierte Einheiten)

Verfahren	Schlüssellänge	Throughput	Anmerkung
RSA-2048	2048 Bit	$1\times$	Modulare Exp., langsam
ElGamal-2048	2048 Bit	$0,7\times$	2 Exp. je Enc
ECDH-P256	256 Bit	$20\times$	Skalarmult. auf Kurve
AES-256	256 Bit	$1000\times$	Mit AES-NI
Kyber-768	768 Bit	$150\times$	Matrix-Multiplikation
Sig.-Chif. ($n = 16$)	256 Bit	$500\times$	$O(n^2)$, reine Integer-Arith.

9.4 Schlüsselstruktur und Schlüsselraum

RSA und ElGamal sind asymmetrisch: Ein öffentlicher Schlüssel zur Verschlüsselung und ein privater Schlüssel zur Entschlüsselung. Dies ermöglicht sicheren Schlüsselaustausch ohne gemeinsamen Vorabkanal, erfordert aber die Verwaltung eines Schlüsselpaars und einer Public-Key-Infrastruktur (PKI).

Die Signatur-Chiffre ist, wie AES, ein symmetrisches Verfahren: Beide Parteien teilen den Schlüssel (a, b, p) . Diese Symmetrie ermöglicht triviale wie auch erweiterte Nutzung: Als Public-Key-Schema lässt sich die Signatur-Chiffre über eine ElGamal-Hybridkonstruktion einbetten (Modul `elgamal`); für Schwellenwert-Anwendungen steht die `Threshold-SignatureChiffre` (Modul `mpc`) bereit.

Der Schlüsselraum der Signatur-Chiffre ist für eine 256-Bit-Primzahl p :

$$|K| = (p - 1) \cdot p \approx p^2 \approx 2^{512}.$$

AES-256 hat $|K_{\text{AES}}| = 2^{256}$; der Signatur-Chiffre-Schlüsselraum ist damit quadratisch größer. Dies stärkt die Sicherheit gegen erschöpfende Suche.

9.5 Informationstheoretische Äquivokation

Shannons Äquivokation $H(K | C)$ misst, wie viel Unsicherheit über den Schlüssel nach Beobachtung eines Chiffrats verbleibt. Für das One-Time-Pad gilt $H(K | C) = H(K)$ – perfekte Geheimhaltung. Für praktische Verfahren mit kurzen Schlüsseln fällt die Äquivokation mit wachsender Chiffratmenge.

Die Signatur-Chiffre erzielt hohe Äquivokation, weil die affine Abbildung $c = (a\sigma + b) \bmod p$ für jeden Geheimtext mehrere konsistente Vorbilder (a, b, A) erlaubt. Im Gegensatz dazu ist bei RSA die Äquivokation nach Kenntnis des öffentlichen Schlüssels im Wesentlichen null – der Angreifer kennt das Verfahren vollständig, nur die Faktorisierung verhindert den Angriff. Diese fundamentale Asymmetrie – *Verbergen des Verfahrens* vs. *informationstheoretische Mehrdeutigkeit* – ist der Kern des Sicherheitsvorteils der Signatur-Chiffre gegenüber klassischen Public-Key-Verfahren.

9.6 Zusammenfassung

Die affine Chiffre [Epp26c] ist als Spezialfall $n = 1$ vollständig in der Signatur-Chiffre enthalten: Für $A = (b_0) \in \{0, 1\}^{1 \times 1}$ gilt $\sigma(A, 0) = b_0$ und $c = (ab_0 + b) \bmod p$. Mit wachsendem n gewinnt die Signatur-Chiffre gegenüber der Affinen Chiffre sowohl an Sicherheit (größerer Schlüsselraum, mehrere unabhängige Signaturspalten) als auch an Kapazität (Blockverarbeitung statt Einzelzeichen).

Im Gesamtvergleich positioniert sich die Signatur-Chiffre als:

- **P = NP-robust** – anders als RSA, ElGamal und alle klassischen Public-Key-Verfahren;
- **quantensicher** – da keine Skalarmultiplikation oder Faktorisierung involviert ist;
- **effizient** – $O(n^2)$ je Block, reine Ganzzahlarithmetik, keine Feldexponentiation;
- **informationstheoretisch begründet** – strukturelle Mehrdeutigkeit statt berechnungsbasierter Schwierigkeit;
- **erweiterbar** – zum Public-Key-, Threshold- und homomorphen Schema ohne Änderung des Kerns.

10 Elliptische-Kurven-Kryptographie (ECC)

10.1 Historische Einordnung und Motivation

Die Elliptische-Kurven-Kryptographie (ECC) wurde 1985 unabhängig von Neal Koblitz [Kob87] und Victor Miller [Mil86] vorgeschlagen. Beide erkannten, dass elliptische Kurven über endlichen Körpern eine algebraische Gruppe bilden, in der das *diskrete Logarithmusproblem* – anders als in $\mathbb{Z}_p^*$ – besonders schwer zu lösen ist. Dies erlaubt ECC, bei deutlich kürzeren Schlüsseln (256 Bit ECC $\approx$ 3072 Bit RSA) eine vergleichbare oder höhere Sicherheit zu erreichen.

ECC ist heute allgegenwärtig: TLS 1.3 [Res18] nutzt ECDHE für den Schlüsselaustausch, Bitcoin [Nak08] verwendet secp256k1 für Transaktionssignaturen, das Signal-Protokoll verwendet Curve25519, und moderne Betriebssysteme generieren SSH-Schlüssel mit Ed25519.

10.2 Algebraische Grundlagen elliptischer Kurven

Definition 10.1 (Elliptische Kurve über $\mathbb{F}_p$). Sei $p > 3$ prim und $a, b \in \mathbb{F}_p$ mit $\Delta := 4a^3 + 27b^2 \neq 0$ (Nicht-Singularität). Die **elliptische Kurve** $E(\mathbb{F}_p)$ ist die Menge:

$$E(\mathbb{F}_p) := \{(x, y) \in \mathbb{F}_p^2 \mid y^2 \equiv x^3 + ax + b \pmod{p}\} \cup \{\mathcal{O}\},$$

wobei $\mathcal{O}$ der **Punkt im Unendlichen** (neutrales Element) ist.

Die Bedingung $\Delta \neq 0$ sichert, dass die Kurve keine Singularitäten (Spitzen oder Selbstschneidungen) besitzt. Ohne diese Bedingung wäre die Gruppenstruktur nicht wohldefiniert.

Satz 10.1 (Gruppengesetz). $E(\mathbb{F}_p)$ bildet mit der folgenden Addition eine abelsche Gruppe. Für $P = (x_1, y_1)$, $Q = (x_2, y_2) \in E(\mathbb{F}_p)$ mit $P \neq \pm Q$ gilt:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{p}, \quad x_3 = \lambda^2 - x_1 - x_2 \pmod{p}, \quad y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p}.$$

Das Ergebnis ist $P + Q = (x_3, y_3)$. Für die Verdopplung $P = Q$:

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{p}, \quad x_3, y_3 \text{ wie oben.}$$

Außerdem: $P + \mathcal{O} = P$, $P + (-P) = \mathcal{O}$ mit $-P = (x_1, -y_1 \pmod{p})$.

Skizze. Die Gruppengesetze folgen aus der geometrischen Konstruktion: Die Summe zweier Punkte ist der Spiegelpunkt des dritten Schnittpunkts der Geraden durch P und Q mit der Kurve an der x -Achse. Kommutativität ist offensichtlich; Assoziativität erfordert umfangreiche algebraische Verifikation, die über Divisorklassen auf der Kurve geführt wird. $\square$

Satz 10.2 (Hasse-Schranke). Die Gruppenordnung $|E(\mathbb{F}_p)|$ erfüllt:

$$|p + 1 - |E(\mathbb{F}_p)|| \leq 2\sqrt{p}.$$

Für $p \approx 2^{256}$ gilt also $|E(\mathbb{F}_p)| \approx 2^{256}$ mit einer Abweichung von höchstens 2^{129} .

10.3 Skalarmultiplikation und Effizienz

Die **Skalarmultiplikation** $[k]P = \underbrace{P + P + \dots + P}_{k\text{-mal}}$ für $k \in \mathbb{N}$ ist die zentrale Operation der ECC. Sie wird durch den **Double-and-Add-Algorithmus** (analog zum Schnellen Potenzieren) in $O(\log k)$ Punktoperationen berechnet:

Listing 3: Skalarmultiplikation auf elliptischen Kurven

```

1 def scalar_mult(k: int, P: tuple, a: int, p: int) -> tuple:
2     """Berechnet [k]P auf E: y^2 = x^3 + ax + b (mod p)"""
3     R = None # Punkt im Unendlichen
4     Q = P
5     while k > 0:
6         if k % 2 == 1:
7             R = point_add(R, Q, a, p)
8             Q = point_add(Q, Q, a, p) # Verdopplung
9             k //= 2
10    return R

```

10.4 Das Elliptic Curve Discrete Logarithm Problem (ECDLP)

Definition 10.2 (ECDLP). Gegeben eine elliptische Kurve $E(\mathbb{F}_p)$, ein Basispunkt $G \in E(\mathbb{F}_p)$ der Ordnung q und ein Punkt $Q = [k]G$, finde $k \in \{0, \dots, q-1\}$.

Der beste bekannte klassische Algorithmus für das ECDLP ist der **Pohlig-Hellman-Baby-Step-Giant-Step**-Algorithmus mit Laufzeit $O(\sqrt{q})$ – exponentiell in der Schlüssellänge. Für $q \approx 2^{256}$ ist $\sqrt{q} \approx 2^{128}$, was als sicher gilt. Shors Quantenalgorithmus [Sho94] würde das ECDLP in polynomieller Zeit lösen; unter $P = NP$ [Epp26a] ist es klassisch polynomiell lösbar.

10.5 Standardkurven

Die wichtigsten standardisierten elliptischen Kurven sind:

Tabelle 10.1: Wichtige ECC-Standardkurven

Kurve	Körper	Anwendung	Quelle
P-256 (secp256r1)	$\mathbb{F}_{2^{256}-2^{224}+\dots}$	TLS, NIST	NIST
P-384 (secp384r1)	$\mathbb{F}_{2^{384}-2^{128}+\dots}$	TLS (hoch)	NIST
secp256k1	$\mathbb{F}_{2^{256}-2^{32}-977}$	Bitcoin, Ethereum	Certicom
Curve25519	$\mathbb{F}_{2^{255}-19}$	Signal, SSH	Bernstein
Ed25519	twisted Edwards	SSH, TLS	Bernstein et al.

10.6 ECDH-Schlüsselaustausch

Das **Elliptic Curve Diffie-Hellman** (ECDH)-Protokoll [Kob87, Mil86] ist das in TLS 1.3 [Res18] verwendete Schlüsselaustauschverfahren:

1. Alice und Bob einigen sich öffentlich auf $E(\mathbb{F}_p)$ und Basispunkt G der Ordnung q .

2. Alice wählt privates $a \in \{1, \dots, q-1\}$, berechnet $A = [a]G$ und sendet A an Bob.
3. Bob wählt privates b , berechnet $B = [b]G$ und sendet B an Alice.
4. Alice berechnet $[a]B = [ab]G$. Bob berechnet $[b]A = [ab]G$. Gemeinsamer Schlüssel: $K = [ab]G$.

Satz 10.3 (Sicherheit von ECDH). ECDH ist sicher unter der **Computational Diffie-Hellman (CDH)**-Annahme: Gegeben G , $[a]G$, $[b]G$, ist $[ab]G$ schwer zu berechnen. Die CDH-Annahme folgt aus der ECDLP-Annahme.

In **ECDHE** (Ephemeral ECDH) werden die Schlüssel a und b für jede Sitzung neu generiert, was **Forward Secrecy** garantiert: Selbst wenn der langfristige private Schlüssel kompromittiert wird, bleiben vergangene Sitzungsschlüssel geheim.

10.7 ECDSA – Elliptic Curve Digital Signature Algorithm

ECDSA ist das standardisierte Signaturverfahren für elliptische Kurven (ANSI X9.62, NIST FIPS 186) [GMR88].

Schlüsselgenerierung: Wähle privates $d \in \{1, \dots, q-1\}$, berechne $Q = [d]G$ (öffentlicher Schlüssel).

Signatur einer Nachricht m : Wähle zufälliges $k \in \{1, \dots, q-1\}$; berechne $(x_1, y_1) = [k]G$; setze $r = x_1 \bmod q$ (falls $r = 0$, neues k); berechne $s = k^{-1}(H(m) + dr) \bmod q$ (falls $s = 0$, neues k). Signatur: (r, s) .

Verifikation von (r, s) für Nachricht m und öffentlichen Schlüssel Q : Berechne $w = s^{-1} \bmod q$, $u_1 = H(m) \cdot w \bmod q$, $u_2 = r \cdot w \bmod q$; berechne $(x_1, y_1) = [u_1]G + [u_2]Q$; prüfe $r \equiv x_1 \pmod{q}$.

Bemerkung 10.1. ECDSA erfordert, dass k für jede Signatur *frisch zufällig* gewählt wird. Wird dasselbe k zweimal verwendet (wie bei der PlayStation-3-Attacke 2010), lässt sich der private Schlüssel d direkt berechnen.

10.8 EdDSA und Ed25519

EdDSA (Edwards-curve Digital Signature Algorithm) [BLC+11] ist ein modernes Signaturverfahren auf der Basis von **Twisted-Edwards-Kurven** der Form $-x^2 + y^2 = 1 + dx^2y^2$. **Ed25519** verwendet die Twisted-Edwards-Form der Curve25519 und bietet gegenüber ECDSA mehrere Vorteile:

- Deterministisches k (abgeleitet vom Schlüssel und der Nachricht) – kein Zufallsgenerator-Risiko.
- Schnellere Implementierungen durch Verwendung spezieller Punktkoordinaten.
- Keine geheimen Verzweigungen im Algorithmus – Resistenz gegen Seitenkanalangriffe [Koc96].

10.9 Verbindung zur Signatur-Chiffre

Die Signatur-Chiffre kann in eine **elliptische Variante** überführt werden: Statt der affinen Transformation $c_j = (as_j + b) \bmod p$ über $\mathbb{Z}_p$ verwendet man die Skalarmultiplikation auf $E(\mathbb{F}_p)$:

$$C_j = [s_j]G + [b]H,$$

wobei G ein öffentlicher Basispunkt und $H = [a]G$ der öffentliche Schlüssel ist. Damit hätte die elliptische Signatur-Chiffre eine Public-Key-Struktur. Die Entschlüsselung wäre dann für den Inhaber von a durch $[a^{-1}](C_j - [b]H)$ möglich. Unter $P = NP$ ist die Sicherheit dieser Variante gesondert zu analysieren.

11 Gitterbasierte Kryptographie und Learning with Errors (LWE)

11.1 Gitter: Definition und grundlegende Eigenschaften

Gitter sind periodische, diskrete Strukturen im $\mathbb{R}^n$. Sie spielen in der Mathematik eine fundamentale Rolle (Zahlentheorie, Geometrie der Zahlen, Kodierungstheorie) und sind seit den 1990er-Jahren die wichtigste Grundlage post-quantensicherer Kryptographie.

Definition 11.1 (Gitter). Sei $B = [b_1, \dots, b_n] \in \mathbb{R}^{m \times n}$ eine Matrix mit linear unabhängigen Spalten. Das **Gitter** $\mathcal{L}(B)$ ist:

$$\mathcal{L}(B) := \left\{ Bx = \sum_{i=1}^n x_i b_i \mid x = (x_1, \dots, x_n)^T \in \mathbb{Z}^n \right\}.$$

Die Vektoren $b_1, \dots, b_n$ heißen **Gitterbasis**. Die **Dimension** des Gitters ist n , seine **Einbettungsdimension** ist m .

Beispiel 11.1 (Ganzzahliges Gitter). Das einfachste Gitter in $\mathbb{R}^2$ ist $\mathbb{Z}^2 = \mathcal{L}(I_2)$ mit Einheitsbasis. Jeder Gitterpunkt hat ganzzahlige Koordinaten.

Definition 11.2 (Gitterdeterminante und sukzessive Minima). Die **Gitterdeterminante** ist $\det(\mathcal{L}) := \sqrt{\det(B^T B)}$. Das i -te **sukzessive Minimum** λ_i ist der kleinste Radius r , sodass $\mathcal{L}$ mindestens i linear unabhängige Vektoren der Länge $\leq r$ enthält.

11.2 Schwere Gitterprobleme

Definition 11.3 (Shortest Vector Problem (SVP)). Gegeben eine Gitterbasis B , finde einen kürzesten Gittervektor $v \in \mathcal{L}(B) \setminus \{0\}$, d. h. $\|v\| = \lambda_1(\mathcal{L})$.

Definition 11.4 (Closest Vector Problem (CVP)). Gegeben eine Gitterbasis B und einen Zielvektor $t \in \mathbb{R}^m$, finde $v \in \mathcal{L}(B)$ mit minimalem $\|v - t\|$.

SVP und CVP sind NP-schwer im schlimmsten Fall [MR09]. Entscheidend für die Kryptographie ist jedoch, dass keine *effizienten* Quantenalgorithmien für SVP oder CVP bekannt sind – im Gegensatz zu Faktorisierung (Shor [Sho94]). Die besten Gitter-Reduktionsalgorithmen (LLL [LLL82], BKZ [Sch87]) haben subexponentielle Laufzeit $2^{O(n)}$.

Satz 11.1 (LLL-Algorithmus [LLL82]). Der LLL-Algorithmus findet in polynomieller Zeit eine **LLL-reduzierte Basis**, d. h. einen Vektor der Länge $\leq 2^{n/2} \lambda_1$. Für kryptographische Dimensionen ($n \geq 256$) ist dies jedoch viel zu lang.

11.3 Learning with Errors (LWE)

Das **Learning with Errors (LWE)**-Problem wurde 2005 von Oded Regev eingeführt [Reg05] und hat sich zum wichtigsten Sicherheitsfundament post-quantensicherer Kryptographie entwickelt.

Definition 11.5 (LWE-Problem). Seien $n, q \in \mathbb{N}$ und χ eine Fehlerverteilung über $\mathbb{Z}_q$ (typisch eine diskrete Gaußverteilung χ_α mit Parameter α). **LWE**(n, q, χ) besteht aus zwei Teilproblemen:

1. **Suchproblem:** Gegeben polynomiell viele Samples $(a_i, b_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ mit $b_i = \langle a_i, s \rangle + e_i \bmod q$, wobei $s \in \mathbb{Z}_q^n$ fest und $e_i \sim \chi$, finde s .
2. **Entscheidungsproblem:** Unterscheide Samples (a_i, b_i) mit $b_i = \langle a_i, s \rangle + e_i$ von uniform zufälligen Paaren.

Satz 11.2 (Quantenreduktion, Regev [Reg05]). Für geeignete Parameter (n, q, χ) ist $\text{LWE}(n, q, \chi)$ mindestens so schwer wie quantenmechanische approximative Versionen von SVP und CVP. Insbesondere folgt: Wenn LWE effizient lösbar ist, dann auch SVP auf einem Quantencomputer – was als sehr unwahrscheinlich gilt.

11.4 LWE-basierte Public-Key-Verschlüsselung (Regev-Schema)

Das folgende Schema ist das einfachste LWE-basierte Kryptosystem [Reg05]:

Schlüsselgenerierung: Wähle $s \in \mathbb{Z}_q^n$ (privater Schlüssel). Wähle m zufällige Vektoren $a_i \in \mathbb{Z}_q^n$ und Fehler $e_i \sim \chi$. Berechne $b_i = \langle a_i, s \rangle + e_i \bmod q$. Öffentlicher Schlüssel: $(A, \mathbf{b}) = ([a_1 | \dots | a_m]^T, (b_1, \dots, b_m)^T)$.

Verschlüsselung eines Bits $\mu \in \{0, 1\}$: Wähle zufälligen Bitvektor $r \in \{0, 1\}^m$. Berechne:

$$u = A^T r \bmod q, \quad v = \mathbf{b}^T r + \mu \cdot \lfloor q/2 \rfloor \bmod q.$$

Chiffre: (u, v) .

Entschlüsselung: Berechne $w = v - \langle u, s \rangle \bmod q$. Dann $\mu = 0$ wenn $|w| < q/4$, sonst $\mu = 1$.

Satz 11.3 (Korrektheit). Der Fehlerterm $e = v - \langle u, s \rangle - \mu \lfloor q/2 \rfloor = \langle A^T r, s \rangle - \langle u, s \rangle + \mathbf{b}^T r - \langle A^T r, s \rangle = \langle r, e_i \rangle \bmod q$ ist klein (da r binär und e_i klein). Für $|\langle r, e_i \rangle| < q/4$ wird μ korrekt rekonstruiert.

11.5 Ring-LWE (RLWE) und die Kyber-Standardisierung

Ring-LWE [LPR10] ersetzt $\mathbb{Z}_q^n$ durch den Polynomring $R_q := \mathbb{Z}_q[x]/(x^n + 1)$ für eine Zweierpotenz n . Dies ermöglicht erheblich effizientere Implementierungen durch die **Number Theoretic Transform (NTT)** – dem Analogon der FFT über endlichen Körpern.

Definition 11.6 (RLWE-Problem). Für $s \in R_q$ geheim und $e_i \sim \chi$ über R_q : Gegeben $(a_i, b_i = a_i \cdot s + e_i \bmod q) \in R_q \times R_q$, finde s .

CRYSTALS-Kyber [ABD+22] ist ein Key Encapsulation Mechanism (KEM) auf Basis von Module-LWE (einer Verallgemeinerung von RLWE auf Moduln über R_q). Kyber wurde 2024 vom NIST [NIST22] als erster post-quantensicherer KEM-Standard (FIPS 203) verabschiedet. Die drei Sicherheitsstufen sind:

Tabelle 11.1: Kyber-Parametersätze

Variante	n	k	Sicherheit	Öff. Schlüsselgröße
Kyber-512	256	2	AES-128-äquivalent	800 Byte
Kyber-768	256	3	AES-192-äquivalent	1184 Byte
Kyber-1024	256	4	AES-256-äquivalent	1568 Byte

11.6 LWE-Erweiterung der Signatur-Chiffre

Die affine Transformation der Signatur-Chiffre lässt sich durch einen LWE-Rauschterm ergänzen. Statt $c_j = (as_j + b) \bmod p$ verwendet man:

$$c_j = (a \cdot s_j + b + e_j) \bmod p, \quad e_j \sim \chi_\alpha,$$

wobei e_j ein kleines Gaußsches Rauschen ist. Die Entschlüsselung nutzt, dass für den legitimen Empfänger $|e_j| < p/4$ gilt:

$$s'_j := (a^{-1}(c_j - b)) \bmod p = s_j + a^{-1}e_j \bmod p \approx s_j,$$

und durch Rundung auf den nächsten ganzzahligen Wert wird s_j exakt rekonstruiert. Diese **LWE-Signatur-Chiffre** fügt eine Sicherheitsschicht hinzu, die nicht auf der Geheimhaltung von p allein beruht.

11.7 Weitere post-quantensichere Gitterkonstruktionen

CRYSTALS-Dilithium (NIST FIPS 204) ist ein gitterbasiertes Signaturverfahren auf Basis des **Module-LWE** und des **Module-SIS**-Problems. Es bietet Signaturen der Länge 2420–4595 Byte.

FALCON (NIST FIPS 206) nutzt **NTRU-Gitter** und bietet kompaktere Signaturen (666–1280 Byte) bei höherem Implementierungsaufwand.

NTRU (Hoffstein, Pipher, Silverman, 1998) ist ein älteres Gitter-Kryptosystem, das auf der Schwierigkeit des kleinsten Vektors in NTRU-Gittern beruht und ebenfalls NIST-standardisiert ist.

12 Homomorphe Verschlüsselung

12.1 Grundidee und historische Entwicklung

Homomorphe Verschlüsselung (HE) ist das “heilige Gral” der Kryptographie: Sie ermöglicht Berechnungen auf verschlüsselten Daten, ohne diese zu entschlüsseln. Das Ergebnis der Berechnung auf den Chiffreten ist, nach Entschlüsselung, identisch mit dem Ergebnis der Berechnung auf den Klartexten.

Definition 12.1 (Homomorphe Verschlüsselung). Ein Verschlüsselungsverfahren (KeyGen, Enc, Dec) ist **additiv homomorph**, wenn für alle m_1, m_2 gilt:

$$\text{Dec}(\text{Enc}(m_1) \oplus \text{Enc}(m_2)) = m_1 + m_2.$$

Es ist **multiplikativ homomorph**, wenn $\text{Dec}(\text{Enc}(m_1) \otimes \text{Enc}(m_2)) = m_1 \cdot m_2$. Ein Verfahren, das *beide* Operationen für beliebig tiefe Schaltkreise unterstützt, heißt **Fully Homomorphic Encryption (FHE)**.

Historische Meilensteine:

- 1978: RSA ist multiplikativ homomorph: $\text{Enc}(m_1) \cdot \text{Enc}(m_2) = \text{Enc}(m_1 m_2)$.
- 1999: Das Paillier-Kryptosystem ist additiv homomorph: $\text{Enc}(m_1) \cdot \text{Enc}(m_2) = \text{Enc}(m_1 + m_2)$.
- 2009: Craig Gentry realisiert mit seiner Doktorarbeit [Gen09] die erste FHE – ein Durchbruch von fundamentaler Bedeutung.
- 2011–2017: BGV [BGV12], BFV, CKKS [CKKS17] machen FHE praktisch nutzbar.
- 2024: OpenFHE, SEAL, HELib und andere Bibliotheken ermöglichen industrielle Anwendungen.

12.2 Das Paillier-Kryptosystem

Das Paillier-Kryptosystem (1999) ist das wichtigste additiv homomorphe Schema der Vor-FHE-Ära und findet heute noch in E-Voting-Systemen und Privacy-Preserving-Protokollen Anwendung.

Schlüsselgenerierung: Wähle große Primzahlen p, q , setze $n = pq$, $\lambda = \text{lcm}(p-1, q-1)$. Wähle $g \in \mathbb{Z}_{n^2}^*$, berechne $\mu = (L(g^\lambda \bmod n^2))^{-1} \bmod n$ mit $L(x) = (x-1)/n$. Öffentlich: (n, g) , privat: (λ, μ) .

Verschlüsselung von $m \in \mathbb{Z}_n$: $c = g^m \cdot r^n \bmod n^2$ für zufälliges $r \in \mathbb{Z}_n^*$.

Entschlüsselung: $m = L(c^\lambda \bmod n^2) \cdot \mu \bmod n$.

Homomorphie: $\text{Enc}(m_1) \cdot \text{Enc}(m_2) = g^{m_1+m_2} (r_1 r_2)^n = \text{Enc}(m_1 + m_2)$. Die Addition im Klartext entspricht der Multiplikation im Chiffret Raum.

12.3 Gentrys FHE-Konstruktion (2009)

Craig Gentrys revolutionäre Konstruktion [Gen09] folgte einer zweiteiligen Strategie:

Schritt 1 – Somewhat Homomorphic Encryption (SHE): Man konstruiert ein Schema, das Addition und Multiplikation für *flache* (geringe Tiefe) Schaltkreise unterstützt.

Dabei akkumuliert jede Operation einen *Rauschterm*. Ab einer gewissen Rauschgröße schlägt die Entschlüsselung fehl.

Schritt 2 – Bootstrapping: Gentry zeigte, dass man aus einer SHE eine FHE machen kann, indem man den Entschlüsselungsschaltkreis *homomorph* auswertet: Das Rauschen wird “aufgefrischt”, indem der verschlüsselte Schlüssel öffentlich gemacht wird (**zirkuläre Sicherheit**). Das Bootstrapping ist der rechentechnisch teuerste Schritt.

Satz 12.1 (Gentrys Theorem [Gen09]). Jedes bootstrappable SHE-Schema kann durch zirkulär-sichere Verschlüsselung des privaten Schlüssels zu einem FHE-Schema erweitert werden.

12.4 Das BGV-Schema

Das BGV-Schema (Brakerski-Gentry-Vaikuntanathan) [BGV12] ist eines der effizientesten leveled FHE-Schemata. Es vermeidet Bootstrapping durch ein Schaltkreis-Tiefen-Management.

Parameterwahl: Polynomring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ mit Primzahlen $q = q_0 > q_1 > \dots > q_L$ (Modulwechsel-Technik). Schlüssel: $s \in R$.

Verschlüsselung: $(c_0, c_1) \in R_q^2$ mit $c_0 + c_1 \cdot s \approx \Delta m \pmod{q}$, wobei $\Delta = \lfloor q/t \rfloor$ und t die Klartextmodulus.

Homomorphe Addition: $(c_0 + c'_0, c_1 + c'_1)$ – rauschaddierend.

Homomorphe Multiplikation: Tensor-Produkt, dann Relinearisierung (Schlüsselwechsel). Nach jeder Multiplikation steigt das Rauschen; ein Modulwechsel von q_i auf q_{i-1} reduziert es proportional.

Tiefenlimit: Ein Schema für Schaltkreise der Tiefe L benötigt $L + 1$ Moduln $q_0 > \dots > q_L$.

12.5 Das CKKS-Schema für reelle Zahlen

Das CKKS-Schema (Cheon-Kim-Kim-Song) [CKKS17] ermöglicht homomorphe Berechnungen auf **approximierten reellen und komplexen Zahlen**. Statt exakter Ganzzahlarithmetik arbeitet CKKS mit Fixpunkt-Arithmetik: Der Kodierungsfehler wird als Teil der Rauschtoleranz akzeptiert.

Kodierung: Ein Vektor reeller Zahlen $(m_1, \dots, m_{n/2}) \in \mathbb{R}^{n/2}$ wird via SIMD-Batching in ein Polynom $\hat{m} \in R$ kodiert. Alle $n/2$ Werte werden simultan verschlüsselt und verarbeitet.

Anwendungen: CKKS ist besonders für neuronale Netze auf verschlüsselten Daten (CryptoNets [GLP+21]), verschlüsselte Genomanalyse, und verschlüsselte Finanzmodelle geeignet.

Listing 4: CKKS-Schema: Konzeptuelle Implementierung mit OpenFHE

```

1 from openfhe import *
2
3 # Parameterwahl fuer CKKS
4 parameters = CCParamsCKKSRSNS()
5 parameters.SetMultiplicativeDepth(5)
6 parameters.SetScalingModSize(50)
7 cc = GenCryptoContext(parameters)
8 cc.Enable(PKESchemeFeature.PKE)
9 cc.Enable(PKESchemeFeature.LEVELED_SHE)
10

```

```

11 # Schlüsselerzeugung
12 keypair = cc.KeyGen()
13 cc.EvalMultKeyGen(keypair.secretKey)
14
15 # Verschlüsselung
16 plaintext = cc.MakeCKKSPackedPlaintext([1.5, 2.3, -0.7, 3.1])
17 ciphertext = cc.Encrypt(keypair.publicKey, plaintext)
18
19 # Homomorphe Multiplikation (im verschlüsselten Zustand)
20 result_ct = cc.EvalMult(ciphertext, ciphertext)
21
22 # Entschlüsselung und Dekodierung
23 result = cc.Decrypt(keypair.secretKey, result_ct)
24 print(result.GetRealPackedValue()) # Approximiert [2.25, 5.29, 0.49,
    9.61]

```

12.6 Threshold FHE und Multiparty Computation

Threshold FHE kombiniert homomorphe Verschlüsselung mit Secret Sharing [Sha79]: Der private Entschlüsselungsschlüssel wird auf n Parteien verteilt, und erst wenn $t + 1$ von n Parteien kooperieren, kann entschlüsselt werden. Dies ist besonders für Cloud-Computing-Szenarien relevant, wo kein einzelner Server dem ganzen Schlüssel vertrauen kann.

12.7 Homomorphe Eigenschaften der Signatur-Chiffre

Für die Signatur-Chiffre gilt für zwei Klartextmatrizen A, A' :

$$\text{Enc}(A)_j + \text{Enc}(A')_j = a(\sigma(A, j) + \sigma(A', j)) + 2b \pmod{p}.$$

Dies ist *nicht* $\text{Enc}(A + A')_j$, da $\sigma(A + A', j) \neq \sigma(A, j) + \sigma(A', j)$ (binäre Überträge). Eine **additiv-homomorphe Variante** wird durch q -adische Signaturen erreicht:

Definition 12.2 (q -adische Signaturfunktion). Für $q \geq 2$ und $A \in \{0, \dots, q-1\}^{n \times n}$:

$$\sigma_q(A, j) := \sum_{i=0}^{n-1} A_{ij} \cdot q^i + j \cdot q^n.$$

Für Einträge ohne Überlauf gilt $\sigma_q(A + A', j) = \sigma_q(A, j) + \sigma_q(A', j)$, und damit $\text{Enc}_q(A + A')_j = \text{Enc}_q(A)_j + \text{Enc}_q(A')_j - 2b \pmod{p}$. Die q -adische Signatur-Chiffre ist additiv-homomorph für eintragsweise Addition ohne Überlauf.

13 Das McEliece-Kryptosystem

13.1 Historische Einordnung

Das McEliece-Kryptosystem wurde 1978 von Robert J. McEliece [McE78] vorgeschlagen – gleichzeitig mit RSA [Riv78], aber zunächst wegen seiner großen Schlüsselgrößen wenig beachtet. Im Zeitalter der Post-Quanten-Kryptographie erlebt es eine Renaissance: Es hat alle kryptoanalytischen Angriffe der letzten 45 Jahre überstanden, kein effizienter Quantenangriff ist bekannt, und NIST hat **Classic McEliece** in die Standardisierung aufgenommen [NIST22].

13.2 Grundlagen der Codierungstheorie

Definition 13.1 (Linearer Code). Ein $[n, k, d]_q$ -**linearer Code** $\mathcal{C}$ ist ein k -dimensionaler Unterraum von $\mathbb{F}_q^n$ mit minimalem **Hamming-Abstand** $d = \min_{c \neq c' \in \mathcal{C}} d_H(c, c')$, wobei $d_H(c, c') = |\{i : c_i \neq c'_i\}|$. Er wird beschrieben durch:

- **Generatormatrix** $G \in \mathbb{F}_q^{k \times n}$: $\mathcal{C} = \{mG \mid m \in \mathbb{F}_q^k\}$.
- **Prüfmatrix** $H \in \mathbb{F}_q^{(n-k) \times n}$: $\mathcal{C} = \ker(H)$, d. h. $Hc^T = 0$ für alle $c \in \mathcal{C}$.

Satz 13.1 (Singleton-Schranke). Für jeden $[n, k, d]_q$ -Code gilt $d \leq n - k + 1$. Codes, die diese Schranke mit Gleichheit erfüllen, heißen **MDS-Codes** (Maximum Distance Separable), z. B. Reed-Solomon-Codes.

Definition 13.2 (Fehlerkorrektur). Ein Code mit minimalem Abstand d kann $t = \lfloor (d-1)/2 \rfloor$ Fehler **korrigieren**: Zu jedem empfangenen Wort $r = c + e$ mit $\|e\|_H \leq t$ findet der Decoder das eindeutige Codewort c .

13.3 Goppa-Codes

Definition 13.3 (Goppa-Code). Sei $\mathbb{F}_{2^m}$ ein endlicher Körper, $\mathcal{L} = (\alpha_1, \dots, \alpha_n) \in \mathbb{F}_{2^m}^n$ paarweise verschieden, und $g(x) \in \mathbb{F}_{2^m}[x]$ ein Polynom vom Grad t mit $g(\alpha_i) \neq 0$ für alle i . Der **binäre Goppa-Code** $\Gamma(\mathcal{L}, g)$ ist:

$$\Gamma(\mathcal{L}, g) := \left\{ c \in \mathbb{F}_2^n \mid \sum_{i=1}^n \frac{c_i}{x - \alpha_i} \equiv 0 \pmod{g(x)} \right\}.$$

Satz 13.2 (Eigenschaften von Goppa-Codes). $\Gamma(\mathcal{L}, g)$ ist ein $[n, n - mt, 2t + 1]_2$ -Code, der t Fehler korrigieren kann. Goppa-Codes haben effiziente **Patterson-Decoder**, die in $O(n \log n)$ Zeit alle t Fehler korrigieren.

Der Patterson-Algorithmus ist der Kern der Entschlüsselung im McEliece-System: Der Decoder nutzt die algebraische Struktur des Goppa-Codes, die einem Angreifer ohne Kenntnis von g und $\mathcal{L}$ nicht zugänglich ist.

13.4 Das McEliece-Kryptosystem: Konstruktion

Schlüsselgenerierung:

1. Wähle Goppa-Code $\Gamma(\mathcal{L}, g)$ mit Generatormatrix $G \in \mathbb{F}_2^{k \times n}$ und Patterson-Decoder D .

2. Wähle zufällige invertierbare Matrix $S \in \mathbb{F}_2^{k \times k}$ und Permutationsmatrix $P \in \mathbb{F}_2^{n \times n}$.
3. Berechne $G' = SGP$.
4. Öffentlicher Schlüssel: (G', t) . Privater Schlüssel: (S, G, P, D) .

Verschlüsselung einer Nachricht $m \in \mathbb{F}_2^k$: Wähle zufälligen Fehlervektor $e \in \mathbb{F}_2^n$ mit $\|e\|_H = t$. Berechne:

$$c = mG' + e = mSGP + e \in \mathbb{F}_2^n.$$

Entschlüsselung von c :

1. Berechne $c' = cP^{-1} = mSG + eP^{-1}$. Da P eine Permutation ist, gilt $\|eP^{-1}\|_H = t$.
2. Wende Patterson-Decoder D auf c' an: Erhalte $m' = mS$ (die Permutation hat den Fehler nicht verändert).
3. Berechne $m = m'S^{-1}$.

Satz 13.3 (Korrektheit). Da $\|eP^{-1}\|_H = t$ und der Goppa-Code t Fehler korrigieren kann, rekonstruiert der Decoder $m' = mS$ eindeutig. Dann liefert S^{-1} das Original m .

13.5 Sicherheitsanalyse

Die Sicherheit des McEliece-Systems beruht auf zwei Annahmen:

Annahme 1 – Allgemeines Dekodierproblem: Gegeben eine zufällige Matrix G' und ein Wort c mit t Fehlern, ist es NP-schwer, m und e zu finden. Das zugrundeliegende Problem (**Syndrome Decoding Problem, SDP**) ist NP-vollständig.

Annahme 2 – Ununterscheidbarkeit: G' ist von einer zufälligen Matrix nicht zu unterscheiden – d. h. die Goppa-Code-Struktur ist nach der Maskierung durch S und P verborgen.

Bemerkung 13.1 (Quantensicherheit). Kein effizienter Quantenangriff auf das allgemeine Dekodierproblem ist bekannt. Grover's Algorithmus [Gro96] gibt einen quadratischen Speedup beim Auffinden des Fehlervektors, verdoppelt aber nur den Parameterraum-Bedarf. Classic McEliece mit $n = 6960$, $t = 119$ gilt als quantensicher [NIST22].

13.6 Praktische Aspekte: Schlüsselgrößen

Der Hauptnachteil des McEliece-Systems sind die großen Schlüsselgrößen:

Tabelle 13.1: Classic McEliece Schlüsselgrößen (NIST-Standardisierung)

Parametersatz	Sicherheit	Öff. Schlüssel	Chiffre
mceliece348864	128 Bit	261 KB	128 Byte
mceliece460896	192 Bit	524 KB	188 Byte
mceliece6960119	256 Bit	1.0 MB	240 Byte

13.7 Verbindung zur Signatur-Chiffre

Der Signaturvektor $\Sigma(A) = (\sigma(A, 0), \dots, \sigma(A, n-1)) \in \mathbb{N}^n$ kann als Codewort eines *implizit definierten linearen Codes* über $\mathbb{Z}$ (nicht $\mathbb{F}_2$, aber strukturell ähnlich) interpretiert werden: Die Spaltengewichte $\lambda(j) = j \cdot 2^n$ spielen die Rolle eines Prüfvektors. Eine hybride Konstruktion, die Goppa-Code-Fehlerkorrektur mit der Signaturabbildung verbindet, könnte ein fehlertolerantes Verschlüsselungsverfahren ergeben: Verschlüsselung mit bewusst hinzugefügten “Fehlern” in der Signaturberechnung, Entschlüsselung durch algebraische Fehlerkorrektur.

14 Das ElGamal-Kryptosystem und Public-Key-Varianten der Signatur-Chiffre

14.1 Das ElGamal-Kryptosystem

Das ElGamal-Kryptosystem wurde 1985 von Taher ElGamal [ELG85] veröffentlicht. Es ist das paradigmatische Beispiel eines Public-Key-Systems basierend auf dem **Diskreten-Logarithmus-Problem (DLP)** in $\mathbb{Z}_p^*$.

Definition 14.1 (Diskretes Logarithmusproblem (DLP)). Sei p prim, g ein Generator von $\mathbb{Z}_p^*$ (d. h. g erzeugt die gesamte multiplikative Gruppe). Das DLP lautet: Gegeben $g^x \bmod p$, finde x .

Schlüsselgenerierung:

1. Wähle große Primzahl p (typisch 2048–4096 Bit) und Generator g von $\mathbb{Z}_p^*$.
2. Wähle privates $x \in \{1, \dots, p-2\}$ gleichmäßig zufällig.
3. Berechne $h = g^x \bmod p$ (öffentlicher Schlüssel).
4. Öffentlich: (p, g, h) . Privat: x .

Verschlüsselung einer Nachricht $m \in \mathbb{Z}_p^*$:

1. Wähle Einmalschlüssel $k \in \{1, \dots, p-2\}$ zufällig.
2. Berechne $c_1 = g^k \bmod p$.
3. Berechne $c_2 = m \cdot h^k \bmod p = m \cdot g^{xk} \bmod p$.
4. Chiffre: (c_1, c_2) .

Entschlüsselung von (c_1, c_2) :

$$m = c_2 \cdot (c_1^x)^{-1} \bmod p = m \cdot g^{xk} \cdot (g^{kx})^{-1} \bmod p = m.$$

Satz 14.1 (Korrektheit). $c_2/c_1^x = m \cdot g^{xk}/g^{kx} = m \pmod{p}$.

14.2 Semantische Sicherheit von ElGamal

Definition 14.2 (IND-CPA-Sicherheit [GM84]). Ein Verschlüsselungsverfahren ist **IND-CPA-sicher** (Indistinguishability under Chosen Plaintext Attack), wenn kein probabilistischer polynomiell-zeitiger Angreifer zwei Chiffre (für selbst gewählte Klartexte) mit Wahrscheinlichkeit $> 1/2 + \epsilon$ unterscheiden kann, für vernachlässigbares ϵ .

Satz 14.2. ElGamal ist IND-CPA-sicher unter der **Decisional Diffie-Hellman (DDH)**-Annahme: (g^a, g^b, g^{ab}) ist von (g^a, g^b, g^c) (für zufälliges c) nicht zu unterscheiden.

Bemerkung 14.1 (ElGamal ist nicht IND-CCA-sicher). ElGamal ist anfällig für Chosen-Ciphertext-Angriffe (CCA): Ein Angreifer kann $c'_2 = c_2 \cdot g^{kx} \bmod p$ als neues Chiffre einreichen und aus dem entschlüsselten Wert Information gewinnen. Dies motiviert die Verwendung von OAEP-Padding oder hybriden Konstruktionen.

14.3 Multiplikative Homomorphie von ElGamal

ElGamal ist **multiplikativ homomorph**: Für zwei Chiffre (c_1, c_2) von m und (c'_1, c'_2) von m' gilt:

$$(c_1 \cdot c'_1, c_2 \cdot c'_2) = \text{Enc}(m \cdot m') \pmod{p}.$$

Dies wird in einigen E-Voting-Protokollen genutzt, ist aber auch eine Sicherheitsschwäche (Multiplikationsangriff).

14.4 ElGamal auf elliptischen Kurven (EC-ElGamal)

EC-ElGamal überträgt das Schema auf $E(\mathbb{F}_p)$. Statt Multiplikation in $\mathbb{Z}_p^*$ wird Punktaddition auf $E(\mathbb{F}_p)$ verwendet:

Verschlüsselung: $(C_1, C_2) = ([k]G, M + [k]Q)$, wobei $Q = [x]G$ der öffentliche Schlüssel und $M \in E(\mathbb{F}_p)$ der als Punkt kodierte Klartext ist.

Entschlüsselung: $M = C_2 - [x]C_1 = M + [k]Q - [x][k]G = M$.

14.5 Cramer-Shoup-Kryptosystem

Das **Cramer-Shoup-Kryptosystem** (1998) ist eine IND-CCA2-sichere Erweiterung von ElGamal [ELG85]. Es fügt einen Integritätsmechanismus hinzu: Das Chiffre enthält eine Art MAC (Message Authentication Code), der verhindert, dass Angreifer modifizierte Chiffre einreichen können.

14.6 Public-Key-Variante der Signatur-Chiffre

Eine asymmetrische Variante der Signatur-Chiffre nach ElGamal-Prinzip wird wie folgt konstruiert.

Schlüsselgenerierung: Wähle Primzahl p , Generator g von $\mathbb{Z}_p^*$, privates x , berechne $h = g^x \pmod{p}$. Öffentlich: (p, g, h) . Privat: x .

Verschlüsselung einer Signatur $s_j = \sigma(A, j)$: Wähle zufälliges k_j , berechne:

$$(c_{j,1}, c_{j,2}) = (g^{k_j} \pmod{p}, s_j \cdot h^{k_j} \pmod{p}).$$

Entschlüsselung: $s_j = c_{j,2} \cdot c_{j,1}^{-x} \pmod{p}$. Anschließend Matrix-Rekonstruktion aus s_j wie in Definition 3.3.

Satz 14.3 (Korrektheit der asymmetrischen Signatur-Chiffre). $c_{j,2}/c_{j,1}^x = s_j \cdot h^{k_j}/g^{xk_j} = s_j \cdot g^{xk_j}/g^{xk_j} = s_j \pmod{p}$. Die Matrix-Rekonstruktion folgt wie in Satz 4.1.

14.7 Digitale Signaturen auf Basis der Signatur-Chiffre

Ein Signaturverfahren im EUF-CMA-Sicherheitsmodell [GMR88] (Existential Unforgeability under Chosen Message Attack) wird wie folgt konstruiert:

Signieren einer Nachricht m :

1. Berechne Hash $h = H(m) \in \{0, 1\}^{n^2}$ (z. B. SHA-256 [NIST15a], gekürzt auf n^2 Bit).
2. Kodiere h als Klartext-Matrix $A_m \in \{0, 1\}^{n \times n}$.
3. Berechne Signatur $\Sigma_m = \text{Enc}(A_m, a, b, p)$ mit privatem Schlüssel (a, b, p) .

Verifikation der Signatur Σ_m für Nachricht m :

1. Berechne A'_m aus $H(m)$ (öffentlich bekanntes Verfahren).
2. Berechne $A''_m = \text{Dec}(\Sigma_m, a, b, p)$ mit dem bekannten Schlüssel.
3. Akzeptiere, wenn $A'_m = A''_m$.

Bemerkung 14.2 (Sicherheit der Signatur-Chiffre-Signaturen). Die Sicherheit des Signaturverfahrens beruht auf der Injektivität von Enc (durch Injektivität von σ) und der Kollisionsresistenz von H . Ohne Kenntnis von (a, b, p) ist es nicht möglich, eine gültige Signatur Σ'_m für eine modifizierte Nachricht m' zu erzeugen – da dies die Berechnung von $\text{Enc}(A_{m'}, a, b, p)$ ohne Kenntnis des Schlüssels erfordern würde. Eine formale EUF-CMA-Analyse unter konkreten Sicherheitsannahmen ist ein offenes Problem.

15 Sichere Mehrparteienberechnung (MPC)

15.1 Das Grundproblem und seine Bedeutung

Das Problem der **Secure Multi-Party Computation (MPC)** stellt eine der tiefgründigsten Fragen der Kryptographie dar: Können mehrere Parteien eine gemeinsame Funktion berechnen, ohne ihre privaten Eingaben preiszugeben?

Definition 15.1 (MPC-Problem). Sei $f : (\{0, 1\}^*)^n \rightarrow (\{0, 1\}^*)^n$ eine n -stellige Funktion. Parteien $P_1, \dots, P_n$ mit privaten Eingaben $x_1, \dots, x_n$ möchten $(y_1, \dots, y_n) = f(x_1, \dots, x_n)$ berechnen, wobei P_i nur y_i und keine Information über x_j ($j \neq i$) erhält – abgesehen von dem, was y_i ohnehin impliziert.

Beispiel 15.1 (Das Millionärsproblem von Yao). Zwei Millionäre A und B möchten herausfinden, wer reicher ist ($x_A > x_B?$), ohne ihren genauen Reichtum x_A, x_B preiszugeben. Andrew Yao formulierte dieses Problem 1982 [Yao82] und entwickelte mit **Garbled Circuits** die erste allgemeine MPC-Lösung.

Beispiel 15.2 (Privatsphäreschützende Statistik). Drei Ärzte möchten den Durchschnittslohn in ihrer Praxis berechnen, ohne die individuellen Gehälter preiszugeben. Mit MPC können sie $\text{avg}(x_1, x_2, x_3) = (x_1 + x_2 + x_3)/3$ berechnen, ohne dass eine Partei mehr als das Ergebnis erfährt.

15.2 Sicherheitsmodelle für MPC

Definition 15.2 (Semi-ehrliche und böswillige Angreifer). - **Semi-ehrlich (honest-but-curious)**: Alle Parteien folgen dem Protokoll korrekt, versuchen aber, aus den empfangenen Nachrichten Informationen zu gewinnen.

- **Böswillig (malicious, active)**: Einige Parteien dürfen beliebig vom Protokoll abweichen, falsche Eingaben verwenden, Nachrichten manipulieren.

Satz 15.1 (Vollständigkeitssatz der MPC [GMW87]). Jede polynomiell berechenbare Funktion kann sicher berechnet werden – semi-ehrlich gegen jede Anzahl korumpierter Parteien, böswillig gegen weniger als die Hälfte korumpierter Parteien – unter Verwendung von Oblivious Transfer (OT) als kryptographischem Primitiv.

15.3 Yaos Garbled Circuits

Garbled Circuits (Yao, 1982) [Yao82] ist die grundlegende Technik für Zwei-Parteien-MPC.

Idee: Alice und Bob möchten $f(x_A, x_B)$ berechnen. Alice repräsentiert f als Booleschen Schaltkreis C und “verschlüsselt” (garbles) jeden Gate. Für jedes Gate g und jede Kombination der Eingabelabels (L_0^0, L_1^0) und (L_0^1, L_1^1) werden vier verschlüsselte Ausgabelabels gebildet. Bob erhält sein Eingabelabel L^b via **Oblivious Transfer (OT)** (ohne dass Alice b erfährt), wertet den Schaltkreis aus und erhält das Ausgabelabel, aus dem er $f(x_A, x_B)$ abliest.

Sicherheit: Bob lernt nichts über x_A (da die Labels zufällig sind). Alice lernt nichts über x_B (da OT verwendet wird). Der Garbled Circuit enthüllt nur das Ergebnis.

Effizienz: Ein Garbled Circuit hat Größe $O(|C| \cdot \kappa)$, wobei $|C|$ die Schaltkreisgröße und κ die Sicherheitsparameter-Länge ist. Moderne Optimierungen (Free XOR, Half Gates) reduzieren den Overhead erheblich.

15.4 Oblivious Transfer (OT)

Definition 15.3 (1-aus-2 Oblivious Transfer (OT_2^1)). Alice besitzt zwei Geheimnisse s_0, s_1 . Bob besitzt Auswahlbit $b \in \{0, 1\}$. Am Ende erfährt Bob s_b , ohne dass Alice b erfährt, und Bob nichts über s_{1-b} .

OT ist das fundamentale Primitiv der MPC: Der GMW-Vollständigkeitssatz [GMW87] zeigt, dass aus OT jede MPC-Funktionalität gebaut werden kann.

Listing 5: OT-Protokoll nach Naor-Pinkas (vereinfacht)

```
1 # Alice: Sender mit Geheimnissen s0, s1
2 # Bob: Empfaenger mit Auswahlbit b
3
4 # Schritt 1: Alice sendet oeffentlichen Schluessel pk
5 # Schritt 2: Bob sendet c = Enc(pk, b) (Bob kennt sk_b)
6 # Schritt 3: Alice sendet e0 = Enc(pk, s0), e1 = Enc(pk, s1)
7 # Schritt 4: Bob entschluesselt e_b mit sk_b
8
9 # Ergebnis: Bob kennt s_b, Alice kennt nichts ueber b
```

15.5 Secret Sharing nach Shamir

Definition 15.4 (Shamir Secret Sharing [Sha79]). Ein Geheimnis $s \in \mathbb{F}_p$ wird auf n Parteien (t, n) -secret-geteilt, sodass:

- Jede Teilmenge von $t + 1$ Parteien kann s rekonstruieren (**Rekonstruierbarkeit**).
- Keine Teilmenge von $\leq t$ Parteien erhält irgendeine Information über s (**Informationssicherheit**).

Konstruktion: Wähle zufälliges Polynom $f(x) = s + a_1x + \dots + a_tx^t \in \mathbb{F}_p[x]$. Verteile Anteil $f(i)$ an Partei i . Rekonstruktion durch Lagrange-Interpolation.

Satz 15.2 (Korrektheit von Shamir Secret Sharing). $t + 1$ Punkte $(i, f(i))$ bestimmen das Polynom f vom Grad $\leq t$ eindeutig (Lagrange-Interpolation). t Punkte liefern keine Information über $f(0) = s$ (da für jedes s' ein Polynom mit diesen t Punkten und $f(0) = s'$ existiert).

Lemma 15.1 (Lagrange-Interpolation). Das eindeutige Polynom f vom Grad $\leq t$ durch die Punkte $(x_1, y_1), \dots, (x_{t+1}, y_{t+1})$ ist:

$$f(x) = \sum_{i=1}^{t+1} y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j} \pmod{p}.$$

15.6 Das BGW-Protokoll für Mehrparteienberechnung

Ben-Or, Goldwasser und Wigderson zeigten 1988 [BGW88], dass mit Shamir Secret Sharing jede Funktion sicher berechnet werden kann – ohne kryptographische Annahmen, aber mit Kommunikationsaufwand. Das **BGW-Protokoll** berechnet jedes Gate g des Schaltkreises auf den geheimgeteilten Eingaben.

Additionsgate: Parteien addieren ihre Anteile lokal: $[a + b]_i = [a]_i + [b]_i$.

Multiplikationsgate: Parteien berechnen $[a]_i \cdot [b]_i$ und reduzieren den Polynomgrad via **Degree-Reduction** (Berlekamp-Welch oder Polynomial-Sharing-Rekonstruktion).

15.7 Threshold Decryption für die Signatur-Chiffre

Threshold Decryption kombiniert die Signatur-Chiffre mit Shamir Secret Sharing: Der private Schlüssel (a, b, p) der Signatur-Chiffre wird (t, n) -secret-geteilt. Die Entschlüsselung erfordert die Kooperation von $t + 1$ Parteien:

1. Jede Partei i besitzt Anteile (a_i, b_i) mit $a = f_a(0)$ und $b = f_b(0)$ (Lagrange-Rekonstruktion).
2. Für Chiffre $C = (c_0, \dots, c_{n-1})$ berechnet Partei i partiell: $s_{j,i} = (a_i^{-1}(c_j - b_i)) \bmod p$ (nur sinnvoll mit vollem Schlüssel; tatsächlich wird ein spezialisiertes Threshold-Protokoll benötigt).
3. $t + 1$ Parteien kombinieren ihre Anteile via Lagrange-Interpolation zur vollständigen Entschlüsselung.

Dies ermöglicht **dezentrale Verschlüsselung**: Kein einzelner Server kennt den vollständigen Schlüssel; die Entschlüsselung erfordert Konsens unter mehreren unabhängigen Parteien.

15.8 Anwendungen von MPC

Private Set Intersection (PSI): Zwei Parteien möchten den Schnitt ihrer Datensätze berechnen, ohne ihre Datensätze preiszugeben (z. B. Kontaktbuch-Abgleich in Signal).

Sicheres Auktionswesen: Mehrere Bieter geben verdeckte Gebote ab; MPC berechnet das höchste Gebot, ohne die anderen preiszugeben.

Kryptowährungs-Wallets: Multi-Signatur-Wallets (Multisig) sind ein einfacher MPC-Anwendungsfall: k -aus- n Parteien müssen zustimmen, bevor Geld überwiesen wird [Nak08].

Genomforschung: Forscher verschiedener Institute möchten Genomdaten gemeinsam analysieren, ohne sie zu teilen (datenschutzrechtliche Anforderungen, DSGVO).

16 Zero-Knowledge-Beweise

16.1 Motivation und Grundbegriffe

Zero-Knowledge-Beweise (ZKPs) sind eines der elegantesten Konzepte der theoretischen Informatik: Ein **Beweiser** $\mathcal{P}$ überzeugt einen **Verifizierer** $\mathcal{V}$ von der Wahrheit einer Aussage, ohne *irgendetwas* außer der bloßen Wahrheit der Aussage preiszugeben. Das Konzept wurde 1985 von Goldwasser, Micali und Rackoff eingeführt [GMR89].

Definition 16.1 (Zero-Knowledge-Beweis [GMR89]). Ein interaktives Protokoll $(\mathcal{P}, \mathcal{V})$ ist ein **Zero-Knowledge-Beweis** für eine Sprache L , wenn folgende drei Eigenschaften gelten:

1. **Vollständigkeit** (Completeness): Für $x \in L$ überzeugt ein ehrlicher $\mathcal{P}$ den Verifizierer $\mathcal{V}$ stets: $\Pr[\mathcal{V} \text{ akzeptiert}] = 1$.
2. **Korrektheit** (Soundness): Für $x \notin L$ kann ein betrügerischer $\mathcal{P}^*$ den Verifizierer nur mit vernachlässigbarer Wahrscheinlichkeit ϵ überzeugen: $\Pr[\mathcal{V} \text{ akzeptiert}] \leq \epsilon$.
3. **Zero-Knowledge**: Es existiert ein effizienter **Simulator** $\mathcal{S}$, der ohne Interaktion mit $\mathcal{P}$ eine Sicht erzeugt, die von der echten Protokolltranskription ununterscheidbar ist.

Beispiel 16.1 (Höhle des Ali Baba). Die klassische Veranschaulichung: Eine Höhle hat die Form eines Rings mit einer Geheime Tür in der Mitte. $\mathcal{P}$ kennt das Öffnungswort, $\mathcal{V}$ möchte dies überprüfen. $\mathcal{P}$ geht zufällig zu Eingang A oder B; $\mathcal{V}$ ruft zufällig “A” oder “B”; $\mathcal{P}$ kommt durch die gewünschte Seite heraus. Nach k Runden ist die Fehlerwahrscheinlichkeit 2^{-k} .

16.2 Das Schnorr-Identifikationsprotokoll

Das **Schnorr-Protokoll** (1989) ist das paradigmatische Beispiel eines effizienten ZKP. Es beweist die Kenntnis eines diskreten Logarithmus.

Setup: Öffentlich bekannt sind Primzahl p , Gruppe $G = \langle g \rangle$ der Ordnung q , und öffentlicher Schlüssel $h = g^x \bmod p$. $\mathcal{P}$ kennt x .

Protokoll (Σ -Protokoll, 3 Runden):

1. **Commitment**: $\mathcal{P}$ wählt $v \leftarrow \mathbb{Z}_q$, berechnet $R = g^v \bmod p$ und sendet R an $\mathcal{V}$.
2. **Challenge**: $\mathcal{V}$ sendet zufällige Herausforderung $c \leftarrow \mathbb{Z}_q$.
3. **Response**: $\mathcal{P}$ sendet $s = v - cx \bmod q$ an $\mathcal{V}$.
4. **Verifikation**: $\mathcal{V}$ prüft $g^s \cdot h^c \equiv R \pmod{p}$.

Satz 16.1 (Sicherheit des Schnorr-Protokolls). Das Schnorr-Protokoll ist:

- **Vollständig**: $g^s \cdot h^c = g^{v-cx} \cdot g^{xc} = g^v = R$. ✓
- **Special Soundness**: Aus zwei gültigen Antworten (s, c) und (s', c') (mit $c \neq c'$) kann $x = (s - s') / (c' - c) \bmod q$ berechnet werden.
- **Honest-Verifier Zero-Knowledge**: Der Simulator wählt zufälliges s und c , berechnet $R = g^s h^c$. Diese Sicht ist von der echten ununterscheidbar.

16.3 Non-Interactive Zero-Knowledge Proofs (NIZK)

Mit dem **Fiat-Shamir-Transform** (1986) kann jedes Σ -Protokoll in ein **non-interaktives** ZKP umgewandelt werden: Die Herausforderung c wird als kryptographischer Hash der Transcript-Daten berechnet: $c = H(R || x_{\text{öff}})$. Das Protokoll wird so in einer einzigen Nachricht (R, s) ausgedrückt.

Satz 16.2 (Fiat-Shamir-Sicherheit). Im **Random Oracle Model (ROM)** – d. h. wenn H als ideale Zufallsfunktion modelliert wird – ist der Fiat-Shamir-Transform für Σ -Protokolle im ROM sicher (Simulation-Sound Zero-Knowledge).

Nicht-interaktive ZKPs (NIZKs) werden für digitale Signaturen verwendet: Ein ECDSA-Signature (r, s) ist implizit ein NIZK-Beweis für die Kenntnis von k (dem Signing-Nonce) [GMR88].

16.4 Commit-and-Prove-Protokolle

Definition 16.2 (Commitment-Schema). Ein **Commitment-Schema** $(Com, Open)$ ermöglicht es $\mathcal{P}$, einen Wert m zu **committen**: $c = Com(m; r)$ mit Zufallselement r . Das Commitment ist:

- **Binding**: $\mathcal{P}$ kann c nicht für zwei verschiedene Werte $m \neq m'$ öffnen.
- **Hiding**: c enthüllt keine Information über m .

Beispiel 16.2 (Pedersen-Commitment). Für eine Gruppe G der Ordnung q mit zwei Generatoren g, h : $Com(m; r) = g^m h^r$. Perfectly hiding (da jeder Wert m durch passendes r erreicht wird) und computationally binding (unter der DL-Annahme).

Commit-and-Prove-Protokolle kombinieren Commitments mit ZKPs: $\mathcal{P}$ committet zu einer Eingabe, beweist dann Eigenschaften dieser Eingabe, ohne sie zu enthüllen. Dies ist der Grundbaustein für zk-SNARKs.

16.5 zk-SNARKs: Succinct Non-interactive ARguments of Knowledge

zk-SNARKs [GGPR13] sind die modernste Form von Zero-Knowledge-Beweisen: Sie sind **succinct** (kompakt: Beweislänge unabhängig vom Berechnungsaufwand) und **non-interactive** (Beweiser sendet eine einzige Nachricht).

Grundkonstruktion: Eine Berechnung wird als arithmetischer Schaltkreis C (Additionen und Multiplikationen über einem Körper) dargestellt. Das **R1CS (Rank-1 Constraint System)** formalisiert dies als Gleichungssystem. Durch **QAP (Quadratic Arithmetic Programs)** [GGPR13] werden diese Bedingungen in Polynome übersetzt. Bilineare Pairings auf elliptischen Kurven ermöglichen die effiziente Verifikation.

Anwendungen: Zcash (datenschutzschützende Kryptowährung), Ethereum-Layer-2 (zkRollup), anonyme Anmeldedaten, Wahlsysteme.

16.6 Zero-Knowledge für die Signatur-Chiffre

Ein ZKP für die Signatur-Chiffre erlaubt zu beweisen: “Ich kenne einen Schlüssel (a, b, p) , sodass $\text{Dec}(C, a, b, p) = A$ ”, ohne (a, b, p) oder A zu enthüllen.

Tabelle 16.1: Vergleich populärer ZKP-Systeme

System	Beweis	Verif.	Setup	Annahmen
Groth16	192 Byte	$O(1)$	Trusted	Pairing-basiert
PLONK	384 Byte	$O(1)$	Universal	Pairing-basiert
STARK	KB-Bereich	$O(\log n)$	Kein	Hash-Funktion
Bulletproofs	$O(\log n)$	$O(n)$	Kein	DL-Annahme

Konstruktion: Das Statement ist $\exists(a, b, p) : c_j = (a \cdot \sigma(A, j) + b) \bmod p$ für alle j . Dies kann als arithmetischer Schaltkreis formuliert werden: Die Signaturfunktion ist eine Summation (Addition und Skalierung), die affine Transformation eine Multiplikation und Addition, die Modulo-Operation eine Rangeproof. Ein zk-SNARK [GGPR13] für diesen Schaltkreis hätte konstante Beweislänge.

Anwendung: Dezentrale Zugriffskontrolle – Ein Nutzer beweist, dass er ein gültiges Chiffre entschlüsseln kann (d. h. er kennt den Schlüssel), ohne ihn zu enthüllen.

17 Kryptographische Hash-Funktionen

17.1 Definition und Sicherheitseigenschaften

Hash-Funktionen sind die universellsten Werkzeuge der Kryptographie: Sie dienen als Grundlage für digitale Signaturen, MACs, Passwort-Hashing, Pseudozufallsgeneratoren, Commitment-Schemata und Blockchain-Strukturen.

Definition 17.1 (Kryptographische Hash-Funktion). Eine Funktion $H : \{0, 1\}^* \rightarrow \{0, 1\}^k$ heißt **kryptographische Hash-Funktion**, wenn sie effizient berechenbar ist und folgende Sicherheitseigenschaften erfüllt:

1. **Einweg-Eigenschaft / Urbild-Resistenz:** Für einen gegebenen Hash h ist es schwer, m mit $H(m) = h$ zu finden. Formal: $\Pr[m \leftarrow \mathcal{A}(H(m_0)) : H(m) = H(m_0)] \leq \epsilon$.
2. **Zweites-Urbild-Resistenz:** Für gegebenes m ist es schwer, $m' \neq m$ mit $H(m) = H(m')$ zu finden.
3. **Kollisionsresistenz:** Es ist schwer, irgendein Paar (m, m') mit $m \neq m'$ und $H(m) = H(m')$ zu finden.

Bemerkung 17.1. Diese drei Eigenschaften sind unterschiedlich stark: Kollisionsresistenz $\Rightarrow$ Zweites-Urbild-Resistenz $\Rightarrow$ Urbild-Resistenz. Die Umkehrungen gelten nicht.

Satz 17.1 (Geburtstagsparadoxon). Eine Kollision in einem k -Bit-Hash-Wert kann mit Wahrscheinlichkeit $\geq 1/2$ in $O(2^{k/2})$ Auswertungen von H gefunden werden. Für SHA-256 ($k = 256$) sind dies 2^{128} Auswertungen.

17.2 Die Merkle-Damgård-Konstruktion

Die **Merkle-Damgård-Konstruktion** [Dam89, Mer89] ist das architektonische Fundament der meisten praktischen Hash-Funktionen (MD5, SHA-1, SHA-2).

Konstruktion: Gegeben eine Kompressionsfunktion $f : \{0, 1\}^{k+b} \rightarrow \{0, 1\}^k$. Der k -Bit-Initialisierungsvektor IV ist öffentlich. Die Nachricht m wird in b -Bit-Blöcke $m_1, \dots, m_\ell$ aufgeteilt (mit Padding). Die Iteration:

$$h_0 = IV, \quad h_i = f(h_{i-1} \| m_i) \quad \text{für } i = 1, \dots, \ell, \quad H(m) = h_\ell.$$

Satz 17.2 (Merkle-Damgård-Sicherheit [Dam89, Mer89]). Wenn die Kompressionsfunktion f kollisionsresistent ist, dann ist die Merkle-Damgård-Konstruktion H kollisionsresistent. (Kontraposition: Eine Kollision in H liefert eine Kollision in f .)

Length Extension Angriff: Die Merkle-Damgård-Konstruktion ist anfällig für **Length Extension Attacks**: Aus $H(m)$ kann ein Angreifer $H(m \| \text{padding} \| m')$ berechnen, ohne m zu kennen. Dies betrifft SHA-1 und SHA-256, nicht SHA-3 oder SHA-512/256.

17.3 MD5 – Eine historische Warnung

MD5 wurde 1992 von Ron Rivest eingeführt und erzeugt 128-Bit-Hashes. Bis 2004 galt es als sicher. Wang und Yu zeigten 2004 [WY05], dass Kollisionen in MD5 in wenigen Minuten auf einem PC gefunden werden können. Heute ist MD5 für alle kryptographischen Zwecke

als kompromittiert zu betrachten – wird aber noch für Prüfsummen nicht-kryptographischer Art verwendet.

Lehre: Die Sicherheit von Hash-Funktionen muss regelmäßig überprüft werden. Kollisionsangriffe können durch subtile algebraische Schwächen ermöglicht werden, die beim Design nicht erkennbar waren.

17.4 SHA-2: SHA-256 und SHA-512

Der **SHA-2**-Standard [NIST15a] umfasst mehrere Hash-Varianten mit 224, 256, 384 und 512 Bit Ausgabe. SHA-256 ist die meistverwendete kryptographische Hash-Funktion weltweit.

SHA-256-Struktur: Die Kompressionsfunktion verwendet:

- Acht 32-Bit-Zustandsvariablen a, b, c, d, e, f, g, h .
- 64 Runden, jeweils mit einem 32-Bit-Rundenschlüssel aus dem Nachrichtenblock.
- Nichtlineare Funktionen: $Ch(e, f, g) = (e \wedge f) \oplus (\neg e \wedge g)$ und $Maj(a, b, c) = (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c)$.
- Rotationen und Additionen (modulo 2^{32}).

Listing 6: SHA-256 Anwendung

```
1 import hashlib
2
3 def sha256_hash(data: bytes) -> str:
4     return hashlib.sha256(data).hexdigest()
5
6 # Beispiel
7 msg = b"Die_Signatur-Chiffre_ist_sicher."
8 h = sha256_hash(msg)
9 print(h)    # a7b3...
10
11 # Lawineneffekt: 1 Bit Aenderung => voellig anderer Hash
12 msg2 = b"Die_Signatur-Chiffre_ist_sicher!"
13 h2 = sha256_hash(msg2)
14 # h und h2 unterscheiden sich in ca. 128 Bits
```

17.5 SHA-3/Keccak: Die Schwamm-Konstruktion

SHA-3 [BDPA13] wurde 2012 standardisiert als Reaktion auf konzeptuelle Schwächen der Merkle-Damgård-Konstruktion (insbesondere Length Extension). SHA-3 basiert auf der **Schwamm-Konstruktion (Sponge Construction)**, einem völlig anderen Designprinzip.

Keccak-Permutation: Ein 1600-Bit-Zustand wird durch 24 Runden einer Permutation κ verändert. Die Permutation besteht aus fünf Schritten $(\theta, \rho, \pi, \chi, \iota)$ auf einer $5 \times 5 \times 64$ -Bit-Zustandsmatrix.

Schwamm-Konstruktion mit Kapazität c und Rate r ($r + c = 1600$):

- **Absorb-Phase:** Blöcke der Nachricht werden XOR-verknüpft mit den ersten r Bits des Zustands, gefolgt von κ .

- **Squeeze-Phase:** Die Ausgabe wird aus den ersten r Bits des Zustands nach weiteren κ -Anwendungen extrahiert.

Vorteile von SHA-3: Kein Length-Extension-Angriff, variable Ausgabelänge (SHAKE-128, SHAKE-256), grundlegend verschiedenes Design von SHA-2 (Diversität gegen unbekannte Schwächen).

17.6 BLAKE2 und BLAKE3

BLAKE2 ist eine hochperformante Hash-Funktion, die SHA-3-Kandidat BLAKE optimiert. Sie ist schneller als MD5 auf 64-Bit-Systemen und wird von vielen modernen Anwendungen bevorzugt (libsodium, WireGuard, Argon2).

BLAKE3 (2020) nutzt einen Merkle-Baum-Aufbau und unterstützt massive Parallelisierung – auf modernen CPUs mit AVX-512 erreicht BLAKE3 mehrere GB/s Durchsatz.

17.7 Passwort-Hashing: bcrypt, scrypt, Argon2

Kryptographische Hash-Funktionen (SHA-256, BLAKE3) sind für Passwort-Hashing ungeeignet, da sie zu schnell sind und GPU-beschleunigte Brute-Force-Angriffe ermöglichen. Stattdessen verwendet man **Memory-hard Hash-Funktionen**:

- **bcrypt** (1999): Iterierte Blowfish-Schlüsselableitung, konfigurierbar langsam.
- **scrypt** (2009): Memory-hard durch Large Memory Access Patterns.
- **Argon2** (2015, NIST/IETF-Standard): Gewinner des Password Hashing Competition. Drei Varianten: Argon2d (GPU-resistant), Argon2i (side-channel-resistant), Argon2id (hybrid).

17.8 HMAC: Hash-basierte Message Authentication Codes

Definition 17.2 (HMAC). Für eine Hash-Funktion H und einen Schlüssel k ist:

$$\text{HMAC}(k, m) = H((k \oplus \text{opad}) \| H((k \oplus \text{ipad}) \| m)),$$

mit $\text{opad} = 0x5c \dots 5c$ und $\text{ipad} = 0x36 \dots 36$ (feste Konstanten).

HMAC ist sicher, solange H eine **PRF** (Pseudorandom Function) ist – eine schwächere Anforderung als Kollisionsresistenz. HMAC-SHA-256 ist der Standard-MAC in TLS 1.3 [Res18].

17.9 Signaturbasierte Hash-Funktion

Die Injektivität der Signaturfunktion motiviert eine Hash-Konstruktion auf Basis der Signatur-Chiffre. Eine Hash-Funktion $H_\sigma : \{0, 1\}^* \rightarrow \mathbb{Z}_p^n$ wird durch iterative Anwendung der Verschlüsselung auf eine Feistel-ähnliche Struktur konstruiert:

$$\begin{aligned} h_0 &= \text{Enc}(A_0, a, b, p) \\ h_i &= \text{Enc}(A_i \oplus M(h_{i-1}), a, b, p), \end{aligned}$$

wobei A_i der i -te Block der Eingabe, $M : \mathbb{Z}_p^n \rightarrow \{0, 1\}^{n^2}$ eine Modulo-Abbildung und $\oplus$ die komponentenweise XOR-Verknüpfung ist. Die Analyse der Kollisionsresistenz – ob eine Kollision in H_σ eine Kollision in der Signaturfunktion oder eine Invertierung von Enc erfordert – ist ein offenes Problem von theoretischem Interesse.

18 Protokolle und Anwendungen

18.1 TLS 1.3: Das wichtigste Sicherheitsprotokoll

Transport Layer Security (TLS) [Res18] sichert den überwältigenden Anteil der Internetkommunikation. TLS 1.3 (RFC 8446, 2018) ist eine vollständige Neuentwicklung mit erheblichen Verbesserungen gegenüber TLS 1.2.

TLS 1.3 Handshake

Der TLS 1.3 Handshake ist auf zwei Roundtrips optimiert:

Erster Roundtrip (ClientHello / ServerHello):

1. Client sendet: TLS-Version, unterstützte Cipher Suites, Zufallswert r_C , ECDHE Key Share g^a .
2. Server antwortet: gewählte Cipher Suite, Zufallswert r_S , ECDHE Key Share g^b .
3. Beide berechnen $K = [ab]G$ (ECDHE-Geheimnis) und leiten Sitzungsschlüssel ab.

Zweiter Roundtrip (Finished):

1. Server sendet: Zertifikat (X.509), ECDSA/RSA-Signatur, Finished-MAC.
2. Client verifiziert das Zertifikat und sendet Finished-MAC.
3. Anschließend: Verschlüsselte Anwendungsdaten (AES-GCM oder ChaCha20-Poly1305).

0-RTT Session Resumption: TLS 1.3 unterstützt optionalen 0-RTT-Modus, bei dem der Client beim zweiten Verbindungsaufbau sofort Anwendungsdaten senden kann – allerdings ohne Forward Secrecy für diese Daten.

TLS in einer Post-P=NP-Welt

Da ECDHE unter $P = NP$ [Epp26a] gebrochen ist, muss der Schlüsselaustausch ersetzt werden. Die symmetrische Schicht (AES-GCM) und die MACs (HMAC-SHA-256) bleiben sicher. Mögliche Szenarien:

- **Kyber-basiertes TLS:** NIST empfiehlt CRYSTALS-Kyber [ABD+22] als KEM-Ersatz für ECDHE. **Hybrid-TLS** (Kyber + ECDHE) ist bereits in Chrome und Firefox implementiert.
- **Signatur-Chiffre-basiertes TLS:** Nach Entwicklung einer Public-Key-Variante der Signatur-Chiffre (Kap. 13) könnte diese als asymmetrischer Baustein im TLS-Handshake eingesetzt werden.

Zertifikate und die Public Key Infrastructure (PKI)

TLS-Sicherheit beruht nicht nur auf dem Schlüsselaustauschalgorithmus, sondern auch auf der **Public Key Infrastructure (PKI)**: Zertifizierungsstellen (CAs) signieren X.509-Zertifikate, die Identitäten bestätigen. Diese Signaturverfahren (RSA, ECDSA) sind unter $P = NP$ ebenfalls kompromittiert und müssen durch post-quantensichere Alternativen (Dilithium, FALCON [NIST22]) ersetzt werden.

18.2 Blockchain-Kryptographie

Bitcoin: Kryptographie im Detail

Bitcoin [Nak08] nutzt Kryptographie auf mehreren Ebenen:

Proof of Work: Der SHA-256-Hash des Block-Headers muss kleiner als ein Schwellenwert sein ($H(\text{header}) < T$). Miner variieren den Nonce-Wert, bis ein gültiger Hash gefunden wird. SHA-256 bleibt unter $P = NP$ sicher.

Adressen: Eine Bitcoin-Adresse ist ein mehrfach gehashter öffentlicher ECDSA-Schlüssel: $\text{addr} = \text{Base58Check}(\text{RIPEMD160}(\text{SHA256}(Q)))$. Das RIPEMD160-SHA256-Hashing ist sicher; der zugrundeliegende ECDSA-Schlüssel Q hingegen ist kompromittiert.

Transaktionssignaturen: Jede Bitcoin-Transaktion wird mit dem privaten ECDSA-Schlüssel (secp256k1) des Senders signiert. Diese Signaturen sind unter $P = NP$ angreifbar: Aus dem öffentlichen Schlüssel $Q = [d]G$ kann d berechnet werden, was die Ausgabe aller Bitcoins an diese Adresse ermöglicht.

- Beispiel 18.1** (Post- $P=NP$ -Maßnahmen für Bitcoin).
1. **Taproot/P2TR:** Bitcoin-Adressen, die den öffentlichen Schlüssel erst beim Ausgeben enthüllen, sind auch nach $P = NP$ sicher – bis zur ersten Ausgabe.
 2. **Migration:** Bitcoins auf neue Adressen mit quantensicheren Signaturen (Dilithium, FALCON) verschieben, bevor ein Angreifer die ECDSA-Schlüssel ableitet.
 3. **Signatur-Chiffre-Signaturen:** Langfristig könnten Signatur-Chiffre-basierte Signaturen (Kap. 13) als Alternative dienen.

Ethereum und Smart Contracts

Ethereum (2015) erweitert Bitcoin um **Smart Contracts**: Programme, die auf der Ethereum Virtual Machine (EVM) ausgeführt werden und automatisch bei bestimmten Bedingungen Transaktionen auslösen. Ethereum nutzt secp256k1 für Signaturen (wie Bitcoin) und Keccak-256 (SHA-3-Variante) für Hashing – der Hash ist unter $P = NP$ sicher, die Signaturen nicht.

zkRollups: Ethereum-Layer-2-Protokolle wie zkSync und StarkNet nutzen zk-SNARKs [GGPR13], um tausende Transaktionen in einem einzigen Beweis zusammenzufassen. Diese ZKP-Konstruktionen bleiben auch unter $P = NP$ sicher, da ihre Sicherheit auf Hash-Funktionen (STARKs) oder elliptischen Pairings basiert.

Merkle-Bäume in Blockchains

Ein **Merkle-Baum** [Dam89] ist eine binäre Baumstruktur, in der jeder innere Knoten der Hash der Verkettung seiner Kinder ist. Die Wurzel (**Merkle-Root**) fasst alle Blätter (Transaktionen) zusammen. Eigenschaften:

- **Tamper-Proof:** Jede Änderung an einem Blatt ändert die Wurzel.
- **Membership Proofs:** Der Beweis, dass eine Transaktion im Block ist, hat Länge $O(\log n)$.
- **Effizienz:** Bitcoin-Block-Header enthält nur die 32-Byte-Merkle-Root, nicht alle Transaktionen.

18.3 IoT-Kryptographie: Ressourcenbeschränkte Umgebungen

Herausforderungen

Das **Internet of Things (IoT)** – Milliarden vernetzter Geräte (Sensoren, Aktoren, Smart-Home-Geräte, Industrieanlagen) – stellt extreme Anforderungen an die Kryptographie:

- **Rechenkapazität:** 8-Bit-Mikrocontroller (z. B. ATmega328 in Arduino) mit 16 MHz Taktfrequenz und 2 KB RAM.
- **Energie:** Batteriebetriebene Sensoren mit Lebensdauern von Jahren.
- **Latenz:** Echtzeit-Anforderungen (Maschinensteuerung, Medizingeräte).
- **Skalierung:** Milliarden Geräte – Schlüsselmanagement ist komplex.

PRESENT – Ein leichtgewichtiger Blockchiffre

PRESENT [BSW07] ist ein 64-Bit-Blockchiffre mit 80-Bit-Schlüssel, speziell für Hardware-optimierte Implementierungen auf RFID-Tags und eingebetteten Systemen entwickelt.

Struktur: 31 Runden, bestehend aus:

1. AddRoundKey: XOR mit 64-Bit-Rundenschlüssel.
2. SBox-Layer: 16 parallele 4-Bit-S-Boxen (Nichtlinearität).
3. PLayer: 64-Bit-Permutation (Diffusion).

Effizienz: PRESENT benötigt nur 1570 GE (Gate Equivalents) in Hardware – etwa $10\times$ weniger als AES.

Signatur-Chiffre für IoT

Die Signatur-Chiffre hat mehrere IoT-relevante Eigenschaften:

- **Keine Tabellen:** Die Signaturfunktion benötigt keine Lookup-Tables (im Gegensatz zu AES S-Boxen), was wertvollen RAM spart.
- **Einfache Operationen:** Nur Addition, Bitshift und Modulo – auf jedem Mikrocontroller verfügbar.
- **Kleiner Schlüssel:** (a, b, p) für $n = 8$ benötigt ca. 128 Bit + Primzahl p (≈ 17 Bit für $p > 2^{16}$).
- **Skalierbarkeit:** n kann klein gewählt werden ($n = 4$ oder $n = 8$) für ressourcenbeschränkte Geräte.

Einschränkung: Für sehr kleine n sinkt die Sicherheit (kleiner Schlüsselraum). Es bedarf sorgfältiger Parameterauswahl für die Zielpattform.

DTLS und CoAP: TLS für IoT

DTLS (Datagram TLS) ist die UDP-Variante von TLS und wird für IoT-Protokolle wie CoAP (Constrained Application Protocol) verwendet. Die Signatur-Chiffre könnte als symmetrische Verschlüsselung in DTLS-Verbindungen eingesetzt werden, ähnlich wie AES-CCM oder ChaCha20-Poly1305.

18.4 Privacy-Preserving Machine Learning (PPML)

Problemstellung

Privacy-Preserving Machine Learning (PPML) adressiert das Spannungsfeld zwischen Datennutzung (Trainieren von ML-Modellen) und Datenschutz (Geheimhaltung der Trainingsdaten):

- **Federated Learning:** Modelle werden lokal auf Endgeräten trainiert; nur Modell-Gradienten werden geteilt (nicht die Rohdaten). Aber: Gradienten können Rückschlüsse auf Trainingsdaten ermöglichen (Gradient Leakage).
- **Differential Privacy:** Gezieltes Rauschen wird zu Gradienten addiert, um individuelle Datenpunkte zu schützen.
- **FHE-basiertes ML:** Modelle werden auf verschlüsselten Daten trainiert. **CryptoNets** [GLP+21] ist ein pionierendes Beispiel: Ein vortrainiertes neuronales Netz wird auf CKKS-verschlüsselten Bildern ausgewertet.

CryptoNets: Neuronale Netze auf verschlüsselten Daten

CryptoNets [GLP+21] demonstriert, dass ein konvolutionales neuronales Netz (CNN) zur Ziffernklassifikation (MNIST) vollständig auf CKKS-verschlüsselten Bildern ausgeführt werden kann. Die Aktivierungsfunktionen (ReLU, Sigmoid) müssen durch polynomielle Approximationen ersetzt werden (z. B. $\text{ReLU}(x) \approx 0.5x^2 + 0.5x$), da nur Additionen und Multiplikationen homomorph ausführbar sind.

Integration der Signatur-Chiffre in PPML

Das LSAT-Framework [Epp26d] lernt Boolesche Funktionen auf Schaltkreisen. Die Integration der Signatur-Chiffre ermöglicht es:

1. **Verschlüsselte Schaltkreise:** Der zu lernende Schaltkreis wird mit der Signatur-Chiffre verschlüsselt. Der Lernalgorithmus arbeitet auf Chiffraten.
2. **Threshold Decryption** (Kap. 14): Die Schaltkreisstruktur wird erst nach Konsens mehrerer Parteien enthüllt.
3. **Zero-Knowledge-Verifikation** (Kap. 15): Der Lernende beweist, dass er den Schaltkreis korrekt gelernt hat, ohne das Schaltkreisdesign zu enthüllen.

Dies ergibt ein datenschutzkonformes Circuit-Learning-Protokoll, das für industrielle Anwendungen (z. B. Patentschutz von Schaltkreisdesigns) geeignet ist.

18.5 Steganographie und digitale Wasserzeichen

Grundbegriffe und Unterschiede

Definition 18.1 (Steganographie vs. Kryptographie). - **Kryptographie:** Verschlüsselt den Inhalt einer Nachricht; die Existenz der Nachricht ist sichtbar.

- **Steganographie:** Verbirgt die Existenz einer Nachricht im Trägersignal (z. B. Bild, Audio, Video).

Definition 18.2 (Digitale Wasserzeichen). Digitale Wasserzeichen [Cox97] betrachten einen Sonderfall der Steganographie: Eine versteckte Markierung wird in ein digitales Medium eingebettet und dient dem **Urheberrechtsschutz** – der Nachweis der Herkunft oder Eigentümerschaft.

LSB-Steganographie in Bildern

Das einfachste Steganographie-Verfahren nutzt die **Least Significant Bits (LSBs)** der Pixelwerte eines Bildes: Da menschliche Betrachter geringe Änderungen im niedrigsten Bit nicht wahrnehmen, können Bits der geheimen Nachricht in die LSBs eingebettet werden. Ein 1024×768 -RGB-Bild kann $1024 \times 768 \times 3 \approx 2.4$ MB versteckte Daten aufnehmen (ein Bit pro Kanal).

Steganalyse: Statistischen Methoden (Chi-Quadrat-Test, RS-Analyse) können LSB-Steganographie erkennen, da die statistische Verteilung der LSBs sich von natürlichen Bildern unterscheidet.

Spread-Spectrum-Wasserzeichen nach Cox et al.

Cox et al. [Cox97] schlugen ein robustes Wasserzeichenverfahren vor, das auf dem Prinzip der **Spektrumkommunikation** beruht: Ein pseudozufälliges Rauschen $w \sim \mathcal{N}(0, 1)^n$ wird zu den wichtigsten DCT-Koeffizienten eines Bildes addiert: $\hat{x}'_i = \hat{x}_i + \alpha w_i$.

Robustheit: Da das Wasserzeichen über viele DCT-Koeffizienten verteilt ist, übersteht es Kompression (JPEG), Filterung, Skalierung und mäßige geometrische Verzerrungen.

Verifikation: Der Detektor berechnet $\text{Corr}(\hat{x}', w) = \sum_i \hat{x}'_i \cdot w_i$ und vergleicht mit einem Schwellenwert.

Signaturbasierte Wasserzeichen

Die Signatur-Chiffre bietet einen neuartigen Ansatz für kryptographisch gesicherte Wasserzeichen:

1. Das Bild wird in $n \times n$ -Pixel-Blöcke aufgeteilt. Jeder Block wird als Binärmatrix A_k interpretiert.
2. Der Signaturvektor $\Sigma(A_k) = \text{Enc}(A_k, a, b, p)$ wird berechnet.
3. Die Signaturwerte werden in den LSBs der Koeffizienten des nächsten Blocks kodiert.
4. Zur Verifikation: Aus dem Bild werden die codierten Signaturwerte extrahiert und mit dem berechneten $\text{Enc}(A_k, a, b, p)$ verglichen.

Vorteil gegenüber LSB-Einbettung: Die Wasserzeichemarkierung ist kryptographisch gebunden (nur mit Kenntnis von (a, b, p) verifizierbar) und strukturell (Manipulationen, die A_k verändern, erzeugen inkonsistente Signaturen – nachweisbar).

Vorteil gegenüber DCT-Wasserzeichen: Die Markierung ist injektiv und fehlerkorrigierend (da die Signaturfunktion injektiv ist, erzeugen verschiedene A_k verschiedene Markierungen). Kollisionen sind ausgeschlossen.

Kryptographische Steganographie

Moderne **kryptographische Steganographie** [Cox97] verbindet Steganographie mit Verschlüsselung: Die versteckte Nachricht wird zunächst verschlüsselt (z. B. mit der Signatur-Chiffre), dann in das Trägersignal eingebettet. Selbst wenn ein Angreifer die Steganographie entdeckt, sieht er nur ein Chiffre ohne Kenntnis des Schlüssels (a, b, p) .

19 Zusammenfassung und Ausblick

19.1 Zusammenfassung

Diese Arbeit hat die **Signatur-Chiffre** als neuartiges, strukturbasiertes Verschlüsselungsverfahren vollständig entwickelt und formal begründet. Die zentralen Ergebnisse werden im Folgenden ausführlich zusammengefasst.

Mathematische Grundlagen und Signaturfunktion

Das Fundament der Signatur-Chiffre ist die **injektive Signaturfunktion** $\sigma(A, j) = \rho(A, j) + j \cdot 2^n$ (Definition 2.1), die jede Spalte einer binären $n \times n$ -Matrix auf eine eindeutige natürliche Zahl in dem Intervall $[j \cdot 2^n, (j+1) \cdot 2^n)$ abbildet. Die drei wesentlichen Eigenschaften wurden vollständig bewiesen:

- **Bereichsgrenzen** (Lemma 2.1): $0 \leq \rho(A, j) \leq 2^n - 1$, also $\sigma(A, j) \in [j \cdot 2^n, (j+1) \cdot 2^n)$.
- **Injektivität** (Lemma 2.2): Aus $\sigma(A, j) = \sigma(A', j')$ folgt zwingend $j = j'$ und $A_{\cdot j} = A'_{\cdot j'}$. Die Intervalldisjunktheit sichert die Eindeutigkeit des Spaltenindex; die Eindeutigkeit der Binärdarstellung sichert die Eindeutigkeit der Spalte.
- **Injektivität des Signaturvektors** (Korollar 2.1): Die Abbildung $\Sigma : \{0, 1\}^{n \times n} \rightarrow \mathbb{N}^n$ ist injektiv – verschiedene Matrizen erzeugen verschiedene Signaturvektoren, ohne Ausnahme.

Definition des Kryptosystems

Die Signatur-Chiffre (Definitionen 3.2 und 3.3) verbindet die Signaturfunktion mit einer **affinen Transformation** über dem Primzahlkörper $\mathbb{Z}_p$:

$$\begin{aligned} \text{Enc}(A, a, b, p) &= ((a \cdot \sigma(A, j) + b) \bmod p)_{j=0}^{n-1}, \\ \text{Dec}(C, a, b, p) &: \text{rekonstruiert } A \text{ aus } (a^{-1}(c_j - b) \bmod p)_{j=0}^{n-1}. \end{aligned}$$

Der Schlüsselraum ist $\mathcal{K} = \mathbb{Z}_p^* \times \mathbb{Z}_p \times \mathcal{P}$ (wobei $\mathcal{P}$ die Menge zulässiger Primzahlen mit $p > 2^{2n}$ bezeichnet). Für $p \approx 2^{256}$ beträgt $|\mathcal{K}| \approx 2^{512}$ – weit größer als der AES-256-Schlüsselraum (2^{256}).

Korrektheit

Korrektheit (Satz 4.1): Für jeden Klartext-Block $A \in \{0, 1\}^{n \times n}$ und jeden Schlüssel (a, b, p) mit $p > 2^{2n}$ und $\gcd(a, p) = 1$ gilt:

$$\text{Dec}(\text{Enc}(A, a, b, p), a, b, p) = A.$$

Der dreigliedrige Beweis (Beweis 4) zeigt: (1) Die affine Rücktransformation $a^{-1}(c_j - b) \bmod p$ rekonstruiert $\sigma(A, j)$ exakt, weil $\sigma(A, j) < 2^{2n} < p$ und daher keine unerwünschte Modulo-Reduktion auftritt. (2) Die Zeilenkomponente $\rho(A, j) = s_j - j \cdot 2^n$ wird eindeutig zurückgewonnen. (3) Die Binärzersetzung von $\rho(A, j)$ rekonstruiert alle Matrixeinträge A_{ij} bitweise.

Zwei vollständige Beispiele: Parameterpaare im Vergleich

Diese Arbeit enthält zwei vollständige numerische Verifikationen des Verfahrens für dieselbe Klartextmatrix $A \in \{0, 1\}^{4 \times 4}$, mit explizit verschiedenen Schlüsseln:

Beispiel 1 (Kapitel 6): $(a, b, p) = (7, 3, 1031)$. Der Schlüssel verwendet eine große Primzahl $p = 1031 \gg 256 = 2^{2^4}$. Das inverse Element $7^{-1} \equiv 442 \pmod{1031}$ wurde durch den erweiterten euklidischen Algorithmus bestimmt. Das Chifftrat lautet:

$$C_1 = (94, 129, 304, 367).$$

Alle vier Signaturen $(13, 18, 43, 52)$ wurden korrekt rekonstruiert.

Beispiel 2 (Kapitel 7): $(a, b, p) = (17, 16, 257)$. Dieses Kapitel widmet sich einem algebraisch herausragenden Parameterpaar: $p = 257 = 2^8 + 1$ ist die **Fermat-Primzahl** F_3 , die kleinstmögliche zulässige Primzahl für $n = 4$. Die Wahl $a = 17$ ist prim und teilerfremd zu $\phi(257) = 256 = 2^8$; $b = 16 = 2^n$ hat strukturelle Bedeutung als Spaltengewicht der Signaturfunktion für $j = 1$. Das inverse Element $17^{-1} \equiv 121 \pmod{257}$ wurde durch den erweiterten euklidischen Algorithmus in zwei Divisionsschritten bestimmt ($17 \cdot 121 = 2057 = 8 \cdot 257 + 1$). Das Chifftrat lautet:

$$C_2 = (237, 65, 233, 129).$$

Alle vier Signaturen wurden ebenfalls korrekt rekonstruiert (Satz 7.5). Der direkte Vergleich $C_1 \neq C_2$ – kein gemeinsamer Chifftratwert – illustriert die semantische Sicherheit: Verschiedene Schlüssel erzeugen völlig verschiedene Chifftrate für denselben Klartext (Satz 7.7).

Tabelle 19.1: Alle bewiesenen Sätze dieser Arbeit im Überblick

Satz	Bezeichnung	Kernaussage
Lem. 2.1	Bereichsgrenzen ρ	$0 \leq \rho(A, j) \leq 2^n - 1$
Lem. 2.2	Injektivität σ	$\sigma(A, j) = \sigma(A', j') \Rightarrow j = j', A_{\cdot j} = A'_{\cdot j'}$
Kor. 2.1	Injektivität Σ	Σ ist injektiv auf $\{0, 1\}^{n \times n}$
Lem. 2.3	Affine Bijektion	$x \mapsto (ax + b) \pmod p$ bijektiv auf $\mathbb{Z}_p$
Satz 4.1	Korrektheit	$\text{Dec}(\text{Enc}(A, \cdot), \cdot) = A$
Satz 4.2	Häufigkeitsimmunität	Keine Häufigkeitsanalyse möglich
Satz 4.3	Schlüsselraumgröße	$ \mathcal{K} = (p - 1) \cdot p$
Satz 4.4	Informationsth. Sicherheit	$H(K C) = H(K)$
Satz 5.1	Enc-Komplexität	$O(n^2)$
Satz 5.2	Dec-Komplexität	$O(n^2)$
Kor. 5.1	Effizienzvergleich	Signatur-Chiffre $\in O(n^2) \ll O(n^6)$ (RSA)
Satz 7.1	Parametervalidität $(17, 16, 257)$	Alle fünf Gültigkeitsbedingungen erfüllt
Satz 7.2	Modulares Inverses	$17^{-1} \equiv 121 \pmod{257}$
Satz 7.3	Signaturvektor	$\Sigma(A) = (13, 18, 43, 52)$
Satz 7.4	Chifftrat $(17, 16, 257)$	$C_2 = (237, 65, 233, 129)$
Satz 7.5	Entschlüsselung $(17, 16, 257)$	Exakte Rekonstruktion aller s_j
Satz 7.6	Matrixrekonstruktion	$A_{\text{rek}} = A$
Satz 7.7	Semantische Verschiedenheit	$C_1 \neq C_2$, kein gemeinsamer Wert
Lem. 7.1	Lawineneffekt	Bit-Flip ändert c_j um $\pm a \cdot 2^i \pmod p \neq 0$

Effizienz

Effizienz (Sätze 5.1, 5.2): Verschlüsselung und Entschlüsselung laufen in $O(n^2)$ – asymptotisch effizienter als RSA ($O(k^3)$ für k -Bit-Schlüssel) bei vergleichbarem Schlüsselraum ($\approx 2^{512}$ für $p \approx 2^{256}$). Für $n = 16$ benötigt die Signatur-Chiffre 256 Ganzzahloperationen je Block; RSA-2048 benötigt für die Entschlüsselung eines vergleichbaren Blocks rund $4 \cdot 10^9$ Bitoperationen.

Sicherheitseigenschaften

Die Signatur-Chiffre vereinigt drei komplementäre Sicherheitseigenschaften:

1. **Häufigkeitsimmunität** (Satz 4.2): Die bijektive affine Transformation auf $\mathbb{Z}_p$ macht Häufigkeitsanalysen auf Block-, Byte- und Bit-Ebene wirkungslos. Ein einzelner Bit-Flip im Klartext ändert den Chiffpratwert vollständig (Lawineneffekt, Lemma 7.1).
2. **Informationstheoretische Sicherheit** (Satz 4.4): Wenn (a, b, p) gleichmäßig zufällig aus $\mathcal{K}$ gewählt und vollständig geheim gehalten werden, gilt $H(K|C) = H(K)$: Das Chiffprat liefert keinerlei Information über den Schlüssel. Diese Eigenschaft ist der Shannon'schen perfekten Geheimhaltung analog und beruht nicht auf $P \neq NP$.
3. **Quantenresistenz**: Da die Signatur-Chiffre keine Faktorisierung, keinen diskreten Logarithmus und keine elliptischen Kurven verwendet, greifen weder Shors Algorithmus [Sho94] noch bekannte Quantenangriffe. Grovers Algorithmus [Gro96] halbiert effektiv die Schlüssellänge, sodass für $p \approx 2^{512}$ ein Sicherheitsniveau von ≈ 256 Bit auch gegenüber Quantenangriffen erhalten bleibt.

Einbettung in das kryptographische Ökosystem

Die Kapitel 9–18 haben gezeigt, dass die Signatur-Chiffre ein reiches Beziehungsgeflecht zu etablierten Verfahren besitzt und vielseitig erweiterbar ist:

- **Affine Chiffre als Spezialfall** [Epp26c]: Für $n = 1$ degeneriert die Signatur-Chiffre zur klassischen affinen Chiffre $c = (ab_0 + b) \bmod p$. Das optimale Parameterpaar $a = 15, b = 29$ der affinen Chiffre [Epp26c] wird durch die Signatur-Chiffre mit $n > 1$ verallgemeinert und in einen kryptographisch sicheren Kontext eingebettet.
- **LWE-Hybridisierung** (Kap. 10): Durch Addition eines Gaußrauschterms $e_j \sim \mathcal{D}_{\mathbb{Z}, \alpha p}$ wird die Signatur-Chiffre mit gittersbasierter Sicherheit [Reg05] angereichert.
- **q -adische homomorphe Variante** (Kap. 11): Die Erweiterung auf q -adische Signaturen $\sigma_q(A, j) = \sum_i A_{ij} q^i + j q^n$ ermöglicht additiv-homomorphe Operationen auf Chiffraten.
- **McEliece- und ElGamal-Varianten** (Kap. 12, 13): Hybridkonstruktionen ermöglichen asymmetrische Schlüsselverteilung ohne Preisgabe des symmetrischen Kerns.
- **MPC und Threshold-Entschlüsselung** (Kap. 14): Shamir's Secret Sharing [Sha79] lässt sich direkt auf den Schlüssel (a, b, p) anwenden.
- **Zero-Knowledge-Beweise** (Kap. 15): Schnorr-Protokoll und Pedersen-Commitments können die Kenntnis des Schlüssels beweisen, ohne ihn zu offenbaren.

- **Kryptographische Hash-Funktion H_σ** (Kap. 16): Eine Merkle-Damgård-ähnliche Iteration über den Signaturvektor erzeugt eine kollisionsresistente Hash-Funktion.
- **Digitale Wasserzeichen und Steganographie** (Kap. 17): Die Injektivität von Σ sichert, dass Wasserzeichenmarkierungen kollisionsfrei und strukturell nachweisbar sind.

19.2 Ausblick: Forschungsagenda

Die Signatur-Chiffre steht am Beginn einer langen Forschungsagenda. Im Folgenden werden die wichtigsten offenen Probleme und Erweiterungsrichtungen systematisch und sehr ausführlich dargelegt. Das zweite vollständige Beispiel mit dem Parameterpaar $(17, 16, 257)$ aus Kapitel 7 eröffnet dabei mehrere neue Forschungslinien, die über die im vorigen Ausblick skizzierten Richtungen hinausgehen.

Systematische Parametertheorie: Optimale Parameterpaare

Das zentrale Ergebnis von Kapitel 7 – die Auszeichnung des Paares $(a, b) = (17, 16)$ in Verbindung mit der Fermat-Primzahl $p = 257$ – wirft grundsätzliche Fragen nach einer *Theorie optimaler Parameter* für die Signatur-Chiffre auf.

Definition “Optimaler Parameterpaare”. Analog zur affinen Chiffre [Epp26c], in der das Paar $(a, b) = (15, 29)$ aus einer vollständigen algebraischen Analyse als optimal hervorgeht, sollte für die Signatur-Chiffre eine formale Optimalitätsdefinition entwickelt werden. Kriterien könnten sein:

- **Maximale Ordnung $\text{ord}_p(a)$:** Je größer die multiplikative Ordnung von a in $\mathbb{Z}_p^*$, desto mehr “Entropie” erzeugt die affine Transformation bei wiederholter Anwendung. Für $(a, p) = (17, 257)$ gilt $\text{ord}_{257}(17) = 32$; ein Primitivwurzel-Parameter g mit $\text{ord}_p(g) = p - 1$ wäre maximal.
- **Differentiell optimale Schlüssel:** a sollte so gewählt werden, dass die Menge $\{a \cdot 2^i \bmod p \mid i = 0, \dots, n - 1\}$ möglichst gleichmäßig über $\mathbb{Z}_p$ verteilt ist. Dies minimiert differentielle Charakteristiken (vgl. Abschnitt über differentielle Kryptoanalyse unten).
- **Minimal-Primzahl-Paare:** Das Paar $(17, 257)$ ist ausgezeichnet, weil $a = F_2 = 2^{2^2} + 1 = 17$ und $p = F_3 = 2^{2^3} + 1 = 257$ beide Fermat-Primzahlen sind. Die Frage, ob Fermat-Primzahl-Paare (F_k, F_{k+1}) für allgemeines k besonders günstige Sicherheitseigenschaften besitzen, ist offen.
- **Strukturelle Selbstbezüglichkeit:** Das Paar $(b = 16, p = 257)$ hat die Eigenschaft $b = 2^n = \sqrt{p - 1}$ – eine strukturelle Beziehung zwischen Additionskonstante, Blockgröße und Primzahl. Ob diese Beziehung kryptographisch irrelevant ist (wie Satz 7.7 nahelegt) oder ob spezielle Eigenschaften erzeugt werden, erfordert eingehende Analyse.

Vollständige Parameterklassifikation für $n = 4$. Für $n = 4$ liegt die Bedingung $p > 256$ fest. Die kleinsten zulässigen Primzahlen sind:

$$257, 263, 269, 271, 277, 281, 283, \dots$$

Eine vollständige Parameterklassifikation würde für jede dieser Primzahlen die zugehörigen Schlüsselräume analysieren, primitive Wurzeln identifizieren und eine Rangordnung der Parameterpaare nach kryptographischer Stärke aufstellen. Dieses Klassifikationsprojekt ist für $n = 4$ computationell in wenigen Stunden durchführbar; für $n = 16$ bereits deutlich aufwendiger.

Verallgemeinerung: Parameterfamilien. Die im vorliegenden Kapitel vorgestellten Beispiele gehören zu zwei Parameterfamilien:

$$\mathcal{F}_1 = \{(a, b, p) : p \text{ groß prim, } a \text{ klein prim, } b \text{ klein}\}, \quad (\text{Kap. 6: } (7, 3, 1031)),$$

$$\mathcal{F}_2 = \{(a, b, p) : p = 2^{2n} + 1 \text{ Fermat prim, } a = 2^{n/2} + 1, b = 2^n\}, \quad (\text{Kap. 7: } (17, 16, 257)).$$

Die Sicherheitsanalyse beider Familien – insbesondere die Frage, ob $\mathcal{F}_2$ strukturell schwächer oder stärker als $\mathcal{F}_1$ ist – ist ein konkretes Forschungsziel.

Formale Sicherheitsbeweise

Die bedeutendste offene Frage ist die vollständige formale Einordnung der Signatur-Chiffre in die modernen Sicherheitsmodelle der Kryptographie.

IND-CPA-Sicherheit. Das IND-CPA-Sicherheitsmodell (Indistinguishability under Chosen Plaintext Attack) [GM84] fordert, dass kein probabilistisch polynomiell-zeitiger Angreifer $\mathcal{A}$ zwei selbst gewählte Chiffre mit Wahrscheinlichkeit $> 1/2 + \epsilon$ unterscheiden kann. Für die Signatur-Chiffre bedeutet dies: Gegeben $\text{Enc}(A, a, b, p)$ für ein unbekanntes $A \in \{A_0, A_1\}$, kann $\mathcal{A}$ nicht mit wesentlichem Vorteil bestimmen, welches A verschlüsselt wurde.

Ein Ansatz zur IND-CPA-Analyse: Wenn (a, b, p) gleichmäßig zufällig aus dem Schlüsselraum gewählt wird und p geheim bleibt, dann ist $c_j = (a\sigma_j + b) \bmod p$ für einen Beobachter ohne Kenntnis von p von einem gleichmäßig zufälligen Element in $\mathbb{Z}$ nicht zu unterscheiden – da das “Modulo p ” die Menge der möglichen Chiffre verdeckt. Eine formale Reduktion auf ein informationstheoretisches Argument nach Shannon [Sha49] ist auszuarbeiten.

Das zweite Beispiel (Kap. 7) liefert einen *empirischen Hinweis* auf IND-CPA: Trotz der strukturell ausgezeichneten Wahl $b = 2^n$ – die ein Angreifer durch Beobachtung des Chiffrats nicht erschließen kann – ist das Chifftrat $C_2 = (237, 65, 233, 129)$ nicht von einem Zufallsvektor zu unterscheiden, solange $p = 257$ unbekannt bleibt. Eine Formalisierung dieses Befunds für beliebige strukturell erkennbare b -Werte ist offen.

IND-CCA-Sicherheit. IND-CCA2-Sicherheit (Chosen Ciphertext Attack) [RS91] erfordert zusätzlich, dass ein Angreifer, der Zugang zu einem Entschlüsselungssorakel hat, dennoch keinen Vorteil erlangt. Die Signatur-Chiffre in ihrer Grundform ist wahrscheinlich *nicht* IND-CCA-sicher: Ein Angreifer kann ein Chifftrat modifizieren (z. B. $c'_j = c_j + \delta$) und aus dem entschlüsselten Wert Informationen gewinnen. Dies motiviert die Entwicklung einer CCA-sicheren Variante durch Integration einer Message Authentication (z. B. HMAC über das Chifftrat), analog zum Cramer-Shoup-Schema [ElG85].

Für das Parameterpaar $(17, 16, 257)$ ist dieser Angriff besonders zu analysieren: Da $p = 257$ die kleinstmögliche Primzahl ist, liegen alle Chifftratwerte in $\{0, \dots, 256\}$. Ein Angreifer mit Wissen, dass $p = 257$ und $n = 4$ (aber ohne Kenntnis von a und b), könnte durch gezielte Chifftrat-Modifikation versuchen, a und b zu rekonstruieren. Die vollständige Sicherheitsanalyse für Minimal-Primzahl-Parameter ist daher ein prioritäres Forschungsziel.

Sicherheitsreduktion auf informationstheoretische Argumente. In einer Welt mit $P = NP$ [Epp26a] kann Sicherheit nicht auf Berechnungsschwierigkeit beruhen. Stattdessen

muss die Reduktion informationstheoretisch argumentieren: Wenn der Schlüssel (a, b, p) aus einem Raum der Größe $|\mathcal{K}| \geq |\mathcal{M}|$ gleichmäßig zufällig gewählt wird und vollständig geheim bleibt, liefert jedes Chiffre über den Klartext keine Information (Shannon'sche perfekte Sicherheit [Sha49]). Die Bedingung $|\mathcal{K}| = (p-1)p \geq 2^{2n^2} = |\mathcal{M}|$ legt die Mindestgröße von p fest. Eine vollständige Formalisierung dieses Arguments, einschließlich der Behandlung mehrerer Klartextblöcke und der Schlüsselwiederverwendung, ist ein zentrales offenes Problem.

Für das Minimalbeispiel $(17, 16, 257)$ gilt $|\mathcal{K}| = 256 \cdot 257 = 65\,792$, während der Klartextrraum $|\mathcal{M}| = 2^{16} = 65\,536$ umfasst. Das Verhältnis $|\mathcal{K}|/|\mathcal{M}| = 65\,792/65\,536 \approx 1,004$ ist knapp über 1 – die informationstheoretische Mindestbedingung ist gerade noch erfüllt. Für kryptographisch relevante Parameter ($n = 16$, $p \approx 2^{512}$) beträgt dieses Verhältnis $\approx 2^{512-512} = 1$ – ein weiterer Beleg, dass die Minimalparameter die theoretischen Grenzen exakt ausleuchten.

Kryptoanalyse

Differentielle Kryptoanalyse. Die differentielle Kryptoanalyse [BS93] untersucht, wie Unterschiede (Differential) in Klartextpaaren sich auf Chiffrepaare auswirken. Für die Signatur-Chiffre: Betrachte zwei Klartextmatrizen A und $A \oplus \Delta$ (für eine Differenzmatrix Δ). Das Chiffre-Differential ist:

$$c_j \oplus c'_j = (a(\sigma(A, j) - \sigma(A \oplus \Delta, j))) \bmod p.$$

Da die Signaturfunktion linear in den Matrixeinträgen ist ($\sigma(A \oplus \Delta, j) = \sigma(A, j) \pm 2^i$ für einen Bit-Flip an Position (i, j)), sind die Differentiale strukturiert. Eine vollständige differenzielle Analyse muss alle möglichen Differenzen Δ untersuchen und deren Auswirkungen auf die Verteilung der Chiffre-Differential analysieren. Das Ziel ist der Nachweis, dass keine differentielle Charakteristik mit signifikanter Wahrscheinlichkeit existiert.

Das Parameterpaar $(17, 16, 257)$ ist für diese Analyse besonders interessant: Da $a = 17$ und $p = 257$ beide Primzahlen sind und $17 \equiv 1 \pmod{16}$ gilt ($17 = 1 \cdot 16 + 1$), hat das Differential $a \cdot 2^i \bmod 257$ für $i = 0, 1, \dots, 3$ die Werte:

$$17 \cdot 1 = 17, \quad 17 \cdot 2 = 34, \quad 17 \cdot 4 = 68, \quad 17 \cdot 8 = 136.$$

Alle vier Differentialwerte sind verschieden und liegen in verschiedenen Hälften von $\mathbb{Z}_{257}$ ($\{0, \dots, 128\}$ vs. $\{129, \dots, 256\}$): $17 < 128$, $34 < 128$, $68 < 128$, $136 > 128$. Dies deutet auf eine gute differenzielle Gleichverteilung hin – ein formaler Beweis steht aus.

Lineare Kryptoanalyse. Die lineare Kryptoanalyse [Mat94] sucht nach linearen Relationen zwischen Klartextbits, Schlüsselbits und Chiffrebits. Für die Signatur-Chiffre: Die affine Transformation $c_j = a\sigma_j + b \pmod{p}$ ist bereits linear in σ_j . Dies bedeutet, dass eine lineare Relation existiert: $\sigma_j = a^{-1}(c_j - b) \pmod{p}$. Da a und b geheim sind, ist diese Relation ohne Schlüsselkenntnis nicht nutzbar. Dennoch muss die lineare Analyse die Möglichkeit ausschließen, dass aus mehreren Chiffre-Klartext-Paaren Schlüsselinformation abgeleitet werden kann.

Known-Plaintext-Angriffe. Der gefährlichste potenzielle Angriff ist der Known-Plaintext-Angriff: Ein Angreifer kennt Paare $(A^{(1)}, C^{(1)}), \dots, (A^{(m)}, C^{(m)})$. Da $c_j^{(k)} = a\sigma_j^{(k)} + b \pmod{p}$, hat er ein überbestimmtes lineares Gleichungssystem in den Unbekannten a , b und – kritisch – p . Für zwei bekannte Paare gilt: $c_j^{(1)} - c_j^{(2)} = a(\sigma_j^{(1)} - \sigma_j^{(2)}) \pmod{p}$. Da p unbekannt ist, ist dies kein gewöhnliches lineares System: Der Angreifer muss p

aus den Chiffraten rekonstruieren. Dies erfordert das Lösen eines Simultaneous Modular Linear Equations-Problems, dessen Schwierigkeit abhängt von der Größe von p und der Anzahl der bekannten Paare. Eine vollständige Analyse dieses Angriffsvektors ist dringend notwendig.

Für das Parameterpaar $(17, 16, 257)$ gilt: Da $p = 257$ verhältnismäßig klein ist, wäre bei bekanntem Klartext-Chiffre-Paar eine erschöpfende Suche über alle Primzahlen $p \leq 512$ (es gibt davon 98) in Kombination mit dem linearen System in a, b denkbar. Dies motiviert, für kryptographische Anwendungen stets $p \gg 2^{2n}$ zu wählen – das Minimalbeispiel dient der theoretischen Analyse, nicht der praktischen Verschlüsselung.

Algebraische Angriffe. Algebraische Angriffe formulieren das Schlüsselrekonstruktionssproblem als Polynomsystem über $\mathbb{Z}$ oder $\mathbb{F}_p$ und wenden Gröbner-Basis-Methoden an. Die Signatur-Chiffre erzeugt aus einem bekannten Paar (A, C) das System $c_j \equiv a \cdot \sigma_j + b \pmod{p}$ für $j = 0, \dots, n-1$ – ein lineares System in a, b bei bekanntem p , aber ein nichtlineares System in a, b, p bei unbekanntem p . Die Komplexität der Gröbner-Basis-Berechnung für dieses gemischte System ist zu analysieren.

Fermat-Primzahlen als Schlüsselkandidaten: Eine neue Forschungsrichtung

Das Parameterpaar $(17, 16, 257)$ aus Kapitel 7 legt eine neue Forschungsrichtung nahe: die systematische Untersuchung von **Fermat-Primzahlen** als Moduln in der Signatur-Chiffre.

Bekannte Fermat-Primzahlen. Die bisher bekannten Fermat-Primzahlen sind $F_0 = 3, F_1 = 5, F_2 = 17, F_3 = 257, F_4 = 65537$. Ob es weitere gibt, ist eines der ältesten offenen Probleme der Zahlentheorie. Diese fünf Primzahlen bilden eine vollständige Parameterfamilie für die Signatur-Chiffre mit $n \in \{1, 2, 4, 8, 16\}$ (da $F_k = 2^{2^k} + 1 > 2^{2^{k+1}} = 2^{2n}$ für $n = 2^k$):

Tabelle 19.2: Fermat-Primzahlen als Moduln der Signatur-Chiffre

k	F_k	$n_{\max}$	$2^{2n_{\max}}$	Schlüsselraum $ \mathcal{K} = (F_k - 1) \cdot F_k$
0	3	1	4 (nicht erfüllt: $3 \not\geq 4$)	–
1	5	1	4	$4 \cdot 5 = 20$
2	17	2	16	$16 \cdot 17 = 272$
3	257	4	256	$256 \cdot 257 = 65\,792$
4	65537	8	65536	$65536 \cdot 65537 \approx 4,3 \cdot 10^9$

Für kryptographisch relevante Blockgrößen ($n = 16$) ist $2^{2n} = 2^{32}$; da $F_5 = 2^{32} + 1 = 4\,294\,967\,297 = 641 \cdot 6\,700\,417$ *nicht prim* ist, scheidet die nächste Fermat-Zahl aus. Für $n = 16$ muss man auf allgemeine Primzahlen $p > 2^{32}$ ausweichen.

Algebraische Vorteile der Fermat-Primzahlen. Für $p = F_k = 2^{2^k} + 1$ gilt:

- $p - 1 = 2^{2^k}$ ist eine reine Zweierpotenz: $\mathbb{Z}_p^*$ ist eine 2-Gruppe (alle Elemente haben Zweierpotenzen als Ordnung).
- Modulo-Reduktion $x \bmod F_k$ lässt sich besonders effizient berechnen: $x \bmod F_k = (x \bmod 2^{2^k}) - \lfloor x/2^{2^k} \rfloor + \lfloor x \geq 2^{2^k} \rfloor \cdot \lfloor x/2^{2^k} \rfloor$ (ähnlich wie Mersenne-Reduktion).
- Für $F_3 = 257$: $x \bmod 257 \approx (x \& 0xFF) - (x >> 8) + \text{Korrektur}$ – eine 8-Bit-Operation.

Diese Eigenschaften machen Fermat-Primzahlen zu besonders implementierungsfreundlichen Moduln – ein kryptographisch relevanter Vorteil für ressourcenbeschränkte Systeme (IoT, Mikrocontroller, FPGA).

Sicherheitsanalyse für Fermat-Primzahlen. Die Struktur von $\mathbb{Z}_{F_k}^*$ als 2-Gruppe hat potenzielle Auswirkungen auf die Sicherheit. Insbesondere:

- Babygelehrter-Riesenschnitt-Angriffe auf $\text{ord}_{F_k}(a)$ haben Komplexität $O(\sqrt{2^{2^k}}) = O(2^{2^{k-1}})$ – für $k = 3$: $O(16)$, trivial. Der diskrete Logarithmus in $\langle a \rangle \leq \mathbb{Z}_{F_3}^*$ ist daher leicht lösbar. Dies ist jedoch für die Sicherheit der Signatur-Chiffre unerheblich, da a und p geheim sind – ein Angreifer kann den diskreten Logarithmus nicht angreifen, ohne p zu kennen.
- Die 2-Gruppen-Struktur erleichtert das Faktorisieren von Elementen der Ordnung 2^{2^k} – ein möglicher Ansatzpunkt für algebraische Angriffe, der zu untersuchen ist.

Public-Key-Variante

Die Entwicklung einer vollständigen asymmetrischen Variante der Signatur-Chiffre ist das wichtigste praktische Erweiterungsziel. Die in Kapitel 13 skizzierte ElGamal-basierte Konstruktion ist ein erster Ansatz; sie erbt jedoch die bekannten Schwächen von ElGamal (nicht IND-CCA-sicher, multiplikative Homomorphie als Sicherheitslücke). Eine bessere Variante könnte folgende Eigenschaften haben:

1. **Informationstheoretische Public-Key-Struktur:** Analog zum Konzept des “Quantum Key Distribution” – ein öffentlicher Schlüssel ermöglicht die Verschlüsselung, aber die Entschlüsselung erfordert eine Information, die informationstheoretisch nicht aus dem öffentlichen Schlüssel ableitbar ist.
2. **Hybridisierung mit Kyber:** Die Signatur-Chiffre wird als symmetrische Schicht verwendet; der symmetrische Schlüssel (a, b, p) wird über CRYSTALS-Kyber [ABD+22] ausgetauscht (KEM-basierter Ansatz).
3. **Fermat-Primzahl-basierte Schlüsselverteilung:** Für $p = F_k$ kann der öffentliche Schlüssel als Abbildung auf $\mathbb{Z}_{F_k}$ strukturiert werden, die Bitoperationen (Shift, XOR) ausnutzt.
4. **Matrixbasierter öffentlicher Schlüssel:** Der öffentliche Schlüssel ist eine Transformation $T(A)$ des Signaturvektors, die Verschlüsselung ermöglicht ohne vollständige Schlüsselkenntnis. Dies ist konzeptuell ähnlich zu McEliece [McE78] (maskierte Generatormatrix).

LWE-Hybridisierung und Gitterbasierte Erweiterungen

Die Integration eines LWE-Rauschterms (Kapitel 10) in die Signatur-Chiffre eröffnet mehrere Forschungsrichtungen:

LWE-Signatur-Chiffre. Die Variante $c_j = (a\sigma_j + b + e_j) \bmod p$ mit $e_j \sim \mathcal{D}_{\mathbb{Z}, \alpha q}$ (diskrete Gaußverteilung) kombiniert die strukturelle Sicherheit der Signatur-Chiffre mit der gitterbasierten Sicherheit von LWE [Reg05]. Konkrete offene Fragen:

- Unter welchen Parameterbedingungen ist die LWE-Signatur-Chiffre korrekt (d. h. wird die Rauschbehandlung bei der Entschlüsselung fehlerfrei durchgeführt)?

- Wie groß muss p im Verhältnis zu n und dem LWE-Rauschparameter α gewählt werden?
- Kann die Sicherheit auf eine Worst-Case-Gitter-Annahme (SVP, SIVP) reduziert werden?

Für das Parameterpaar $(17, 16, 257)$ ist der Rauschterm besonders kritisch: Da alle Signaturwerte $\leq 52 \ll 257 = p$, kann ein Rauschterm $|e_j| \leq 100$ toleriert werden ohne Entschlüsselungsfehler – ein ungewöhnlich großes Rauschfenster für ein LWE-System.

Ring-LWE-Variante. Statt des skalaren $\sigma_j \in \mathbb{Z}$ könnten Polynomring-Elemente $\hat{\sigma}_j \in R_q = \mathbb{Z}_q[x]/(x^n + 1)$ verwendet werden. Die Signaturfunktion wird zu einer Polynomabbildung, und die affine Transformation wird zur Polynommultiplikation in R_q . Dies ermöglicht NTT-basierte Implementierungen mit erheblichen Effizienzgewinnen bei großem n .

Homomorphe Erweiterung

Die q -adische Signatur-Chiffre eröffnet den Weg zu additiv-homomorphen Konstruktionen. Konkrete Ziele:

Vollständig homomorphe Variante. Eine FHE-kompatible Signatur-Chiffre erfordert zusätzlich multiplikative Homomorphie. Da die affine Transformation multiplikativ nicht-homomorph ist $((as_1 + b)(as_2 + b) \neq a(s_1s_2) + b)$, muss eine modifizierte Konstruktion entwickelt werden. Ein Ansatz: Verwendung bilinearer Pairings auf elliptischen Kurven (ähnlich Groth-Sahai-Beweisen) zur Realisierung von Multiplikationen auf Chiffraten.

PPML-Integration. Die Integration der q -adischen Signatur-Chiffre in das LSAT-Framework [Epp26d] erfordert:

- Kodierung von Booleschen Schaltkreisen als q -adische Matrizen.
- Homomorphe Auswertung von LSAT-Lernalgorithmen auf q -adischen Chiffraten.
- Effizienzanalyse: Wie groß ist der Overhead gegenüber dem unverschlüsselten LSAT?

Implementierung, Benchmarking und Seitenkanalanalyse

Referenzimplementierung. Eine vollständige Referenzimplementierung der Signatur-Chiffre in Python (für Korrektheit) und in C (für Effizienz) ist zu erstellen. Die Implementierung sollte folgende Module umfassen:

- Schlüsselgenerierung mit kryptographisch sicherer Zufallszahlenerzeugung.
- Blockweise Verschlüsselung und Entschlüsselung beliebig langer Klartexte.
- Padding-Schema (PKCS#7-analog) für Klartexte, deren Länge kein Vielfaches von n^2 ist.
- Timing-resistente Implementierung des modularen Inversen (Constant-Time Extended Euclid) [Koc96].
- Spezialfall-Optimierung für Fermat-Primzahl-Moduln $p = 2^k + 1$ (Bitoperations-basierte Reduktion).

Benchmarking auf verschiedenen Plattformen. Für das Parameterpaar $(17, 16, 257)$ sind folgende Benchmarks besonders aufschlussreich:

- Auf ARM Cortex-M: Da alle Operationen 8-Bit-Arithmetik nutzen ($p = 257 < 2^9$), passt jede Signatur in ein Byte; die gesamte Verschlüsselung eines 16-Bit-Blocks erfordert vier 9-Bit-Multiplikationen. Dies ermöglicht extrem effiziente Implementierungen auf Mikrocontrollern.
- Auf FPGA: Die Modulo-257-Reduktion kann hardwaredirekt als LUT implementiert werden; die gesamte Enc-Operation passt in einen einzigen Takt-Zyklus.
- Im Vergleich: AES-256 [DR02], ChaCha20, und die Signatur-Chiffre mit $(17, 16, 257)$ auf identischer Hardware.

Seitenkanalangriffe auf Fermat-Primzahl-Parameter. Timing-Angriffe [Koc96] auf die Modular-Inversion und die Primzahlgenerierung sind zu analysieren. Für $p = 257$ ist die Berechnung von $a^{-1} \bmod 257$ trivial und auf jedem Prozessor in konstanter Zeit durchführbar – ein Timing-Angriff ist hier besonders leicht abzuwehren.

Kryptographische Hash-Funktion auf Basis der Signatur-Chiffre

Die Konstruktion einer kryptographischen Hash-Funktion H_σ auf Basis der Signatur-Chiffre (Kapitel 16) erfordert folgende Forschungsschritte:

1. **Formale Definition:** Formalisierung der Merkle-Damgård-ähnlichen Iteration und des Padding-Schemas.
2. **Kollisionsanalyse:** Nachweis oder Widerlegung der Kollisionsresistenz unter der Annahme, dass die Signaturfunktion kollisionsresistent ist (was durch ihre Injektivität impliziert wird).
3. **Sicherheit gegen Längenangriffe:** Untersuchung, ob H_σ anfällig für Length-Extension-Attacks ist (analog zu SHA-2), und ggf. Übergang zur Schwammkonstruktion (analog zu SHA-3 [BDPA13]).
4. **Benchmarking:** Vergleich mit SHA-256 und BLAKE3 auf verschiedenen Plattformen.
5. **Fermat-Variante:** Eine H_σ -Variante mit $p = 257$ könnte besonders für ressourcenbeschränkte Systeme geeignet sein – eine 8-Bit-Hash-Funktion mit strukturell gesicherter Injektivität.

Zero-Knowledge-Protokolle für die Signatur-Chiffre

Σ -Protokoll für Kenntnis des Schlüssels. Aufbauend auf dem Schnorr-Protokoll soll ein Σ -Protokoll entwickelt werden, das die Kenntnis eines Schlüssels (a, b, p) beweist, sodass $\text{Enc}(A, a, b, p) = C$ für gegebene (A, C) . Die zentrale Herausforderung ist die Behandlung der Modulo- p -Operation: Da p geheim ist, kann die übliche Technik (lineare Commitment) nicht direkt angewendet werden.

Nicht-interaktiver ZKP via Fiat-Shamir. Nach Entwicklung des Σ -Protokolls wird via Fiat-Shamir-Transform [BFM88] ein NIZK konstruiert. Dieser kann für folgende Anwendungen genutzt werden:

- **Anonyme Entschlüsselung:** Nachweis der Entschlüsselungsfähigkeit ohne Schlüsseloffenbarung.
- **Verifiable Encryption:** Nachweis, dass ein Chiffirat C eine bestimmte Eigenschaft des Klartexts A erfüllt (z. B. $A_{00} = 1$), ohne A vollständig zu enthüllen.
- **Audit-Protokolle:** Ein Dritter kann überprüfen, dass eine Entschlüsselung korrekt durchgeführt wurde, ohne den Schlüssel zu kennen.

ZKP für Parameterpaar-Auszeichnung. Eine neue Anwendung ergibt sich aus Kapitel 7: Gegeben ein Chiffirat C und die öffentliche Information, dass $p = F_k$ eine Fermat-Primzahl ist, kann ein Beweiser zeigen, dass das verwendete Parameterpaar “optimal” im Sinne von Abschnitt 19.1 ist – ohne a , b oder p zu offenbaren. Dieses “Proof of Parameter Quality” könnte in standardisierten Protokollen Verwendung finden.

Verallgemeinerungen und algebraische Erweiterungen

Signatur-Chiffre über allgemeinen Ringen. Die aktuelle Signatur-Chiffre operiert über dem Körper $\mathbb{Z}_p$. Eine natürliche Verallgemeinerung ist die Betrachtung von:

- **Polynomringen:** $\mathbb{Z}_p[x]/(f(x))$ für irreduzibles f – Signaturwerte werden zu Polynomen, die affine Transformation zur Polynommultiplikation. Dies verbindet die Signatur-Chiffre mit der RLWE-Kryptographie [LPR10].
- **Matrixringen:** $M_k(\mathbb{Z}_p)$ – Signaturvektoren werden zu Matrizen, die Transformation zu einer Matrixmultiplikation. Dies ermöglicht nichtkommutative Kryptographie.
- **Gruppenringen:** $\mathbb{Z}_p[G]$ für eine endliche Gruppe G – Verbindung zur nichtabelschen Kryptographie und zu Braid-Gruppen.
- **Fermat-Körper-Erweiterungen:** Der Körper $\mathbb{F}_{257^k}$ für $k > 1$ ist eine Erweiterung von $\mathbb{F}_{257}$ mit $p^k - 1$ Elementen in der multiplikativen Gruppe. Die Signatur-Chiffre über $\mathbb{F}_{257^k}$ könnte besonders günstige Eigenschaften für $k = 2$ (d. h. Körper mit $257^2 - 1 = 66048$ Elementen) haben.

Verallgemeinerte Signaturfunktionen. Die binäre Signaturfunktion $\sigma(A, j) = \sum_i A_{ij}2^i + j2^n$ ist ein Spezialfall. Verallgemeinerungen umfassen:

- **q -adische Basis:** $\sigma_q(A, j) = \sum_i A_{ij}q^i + jq^n$ für beliebiges $q \geq 2$.
- **Fermat-gewichtete Signaturen:** Für $p = F_k = 2^{2^k} + 1$ kann die Gewichtung $\sigma_F(A, j) = \sum_i A_{ij}F_i + j \cdot F_n$ (mit Fermat-Zahlen als Gewichten) definiert werden. Da die Fermat-Zahlen paarweise teilerfremd sind ($\gcd(F_i, F_j) = 1$ für $i \neq j$), ergibt sich eine injektive Funktion mit besonders großem Wertebereich.
- **Gewichtete Signaturen:** $\sigma_w(A, j) = \sum_i A_{ij}w_i + jW$ für beliebige Gewichte w_i – wenn die Gewichte aus der Schlüsselgenerierung stammen, erhöht sich die Schlüsselraumgröße.
- **Nichtlineare Signaturen:** $\sigma_{nl}(A, j) = \sum_i A_{ij}p_i(j)$ für Polynome p_i – Injektivität erfordert neue Bedingungen an die p_i .

Standardisierung und praktischer Einsatz

NIST Post-Quantum Standardisierung. Langfristig strebt die Signatur-Chiffre eine Einreichung beim NIST Post-Quantum Cryptography Standardisierungsprozess [NIST22] an. Voraussetzungen:

- Vollständige formale Sicherheitsanalyse (IND-CPA, IND-CCA2).
- Öffentliche Kryptoanalyse durch die Gemeinschaft (mindestens 2–3 Jahre).
- Referenzimplementierungen in C, Python, und ggf. Hardware (VHDL/Verilog).
- Parametersätze für verschiedene NIST-Sicherheitsniveaus (128, 192, 256 Bit).
- Performance-Messungen (Cycles/Byte) auf NIST-Referenzplattformen.
- Für Level-1-Sicherheit: Parameterempfehlung mit $p = F_4 = 65537$ und $n = 8$ (da $F_4 > 2^{16} = 2^{2 \cdot 8}$) als besonders hardwarefreundliche Option.

Integration in bestehende Protokolle. Eine Public-Key-Variante der Signatur-Chiffre könnte in TLS 1.3 [Res18] als Alternative zu ECDHE eingesetzt werden, analog zur Integration von CRYSTALS-Kyber [ABD+22]. Konkrete Schritte:

- Definition einer TLS-Erweiterung (Extension) für die Signatur-Chiffre.
- Implementierung in OpenSSL oder BoringSSL.
- Interoperabilitätstests.

Integration in das LSAT-Framework. Die Integration der Signatur-Chiffre in das LSAT-Framework [Epp26d] ist ein kurzfristig realisierbares Ziel:

- Kryptographische Schnittstelle: LSAT-Schaltkreise werden vor dem Training verschlüsselt.
- Threshold-Decryption (Kap. 14): Das Lernresultat wird erst nach Konsens mehrerer Parteien enthüllt.
- Verifikation via ZKP (Kap. 15): Nachweis der korrekten Lernausführung ohne Offenbarung des Schaltkreises.

Verbindungen zur Komplexitätstheorie

Sicherheit in einer $P=NP$ -Welt. Der Beweis $P = NP$ [Epp26a] impliziert, dass alle Sicherheitsbeweise, die auf “schweren” Problemen beruhen, hinfällig sind. Für die Signatur-Chiffre stellt sich die Frage: Gibt es überhaupt ein Verschlüsselungsverfahren, das sicher ist, obwohl $P = NP$? Die Antwort ist ja – unter informationstheoretischen Annahmen: Das One-Time-Pad [Ver26] ist sicher, unabhängig von P vs. NP . Die Signatur-Chiffre hat eine analoge Eigenschaft, wenn $|\mathcal{K}| \geq |\mathcal{M}|$.

Das zweite vollständige Beispiel (Kap. 7) illustriert die Grenzen dieser Eigenschaft: Für $(17, 16, 257)$ gilt $|\mathcal{K}| = 65\,792 > 65\,536 = |\mathcal{M}|$ – die Bedingung ist um den Faktor $\approx 1,004$ erfüllt. Ein polynomieller Algorithmus (durch $P = NP$ gegeben) könnte bei Kenntnis aller Klartexte und Chifftrate versuchen, den Schlüssel zu rekonstruieren. Da $|\mathcal{K}| \approx |\mathcal{M}|$, ist die Shannon-Äquivokation klein – ein weiterer Beleg, dass Minimal-Parameter wie $(17, 16, 257)$ eher theoretisch als praktisch sind.

Kolmogorov-Komplexität und Sicherheit. Die Kolmogorov-Komplexität $K(C)$ des Chiffrats C im Verhältnis zur Komplexität $K(A)$ des Klartexts ist ein theoretisches Sicherheitsmaß. Für $(17, 16, 257)$ ist $C_2 = (237, 65, 233, 129)$ – vier Zahlen aus $\{0, \dots, 256\}$ – komprimierbar (da p klein und bekannt ist). Für große p hingegen sind die Chiffratwerte im Wesentlichen nicht komprimierbar.

Beziehung zu Pseudozufall. Pseudozufallspermutationen (PRPs) und Pseudozufallsfunktionen (PRFs) sind die formalen Grundlagen symmetrischer Kryptographie [GM84]. Die Frage, ob die Signatur-Chiffre eine PRP oder PRF unter geeigneten Sicherheitsannahmen realisiert, ist zentral für ihre Einordnung in die moderne Kryptographietheorie. Für das Fermat-Primzahl-Paar $(17, 16, 257)$ lässt sich diese Frage für den kleinen Schlüsselraum (65 792 Elemente) vollständig computationell untersuchen – ein erster Schritt zur PRP-Analyse.

Langfristige Forschungsagenda (Zusammenfassung)

Tabelle 19.3: Langfristige Forschungsagenda für die Signatur-Chiffre

Zeitraumen	Ziele
Kurzfristig (1–2 Jahre)	Vollständige differentielle und lineare Kryptoanalyse; formale IND-CPA-Analyse; Referenzimplementierung (C/Python); Timing-resistente Implementierung; Parameterklassifikation für $n = 4$; Benchmarking auf IoT-Plattformen mit $p = 257$; Integration in LSAT [Epp26d]; Fermat-Primzahl-Reduktionsmodul.
Mittelfristig (2–5 Jahre)	IND-CCA2-sichere Variante (Cramer-Shoup-analog); LWE-Hybridisierung [Reg05]; Public-Key-Variante mit Fermat-Primzahl-Schlüssel; Σ -Protokoll und NIZK; ZKP für Parameterpaar-Qualität; q -adische homomorphe Erweiterung; Hash-Funktion H_σ (inkl. Fermat-Variante); PRP-Analyse für $(17, 16, 257)$; NIST-Einreichungsvorbereitung [NIST22].
Langfristig (5–10 Jahre)	Informationstheoretischer Sicherheitsbeweis (vollständig); Ring-LWE-Variante; FHE-kompatible Erweiterung; PPML-Integration [GLP+21]; TLS-Integration [Res18]; Fermat-Körpererweiterungs-Variante $\mathbb{F}_{257^k}$; Fermat-gewichtete Signaturfunktionen; Internationale Standardisierung (NIST, ISO/IEC, ETSI); Einsatz in Produktivsystemen.

Abschlussbemerkung

Die Signatur-Chiffre ist nicht das Ende eines Weges, sondern der Beginn einer neuen Forschungsrichtung: **strukturbasierte Kryptographie**, die aus der algorithmischen Graphentheorie und kombinatorischen Algebra heraus entsteht und in einer $P = NP$ -Welt

die Sicherheit digitaler Kommunikation neu begründet. Das zweite vollständige Beispiel mit dem Fermat-Parameterpaar $(a, b, p) = (17, 16, 257)$ hat gezeigt, dass die Signatur-Chiffre nicht nur für große, kryptographisch dimensionierte Parameter korrekt funktioniert, sondern auch in ihrem absoluten Minimalfall – mit der kleinstmöglichen zulässigen Primzahl $p = 257 = 2^8 + 1$ – alle formalen Korrektheitseigenschaften exakt erfüllt. Die vollständige numerische Verifikation (Sätze 7.4, 7.5, 7.6) und der direkte Vergleich mit dem ersten Beispiel (Satz 7.7) haben dabei zwei fundamentale Eigenschaften empirisch belegt: (1) die Korrektheit des Verfahrens über alle Parameter hinweg und (2) die semantische Sicherheit durch vollständige Unkorrelierbarkeit verschiedener Schlüsselchiffre.

Die Fermat-Primzahl 257 ist dabei weit mehr als ein numerisches Beispiel – sie eröffnet eine neue Familie algebraisch ausgezeichnete Parameter, deren systematische Untersuchung eine eigene Forschungsrichtung begründet. Sie zeigt, dass der Beweis $P = NP$ nicht das Ende der Kryptographie bedeutet, sondern eine Einladung zur Erneuerung – weg von berechnungsbasierter Schwierigkeit, hin zu struktureller Tiefe, algebraischer Eleganz und informationstheoretischer Äquivokation.

Literaturverzeichnis

- [Epp26a] S. Epp. *Der Subgraph Algorithmus*. 2026.
- [Epp26c] S. Epp. *Die Affine Chiffre: Vollständige mathematische Analyse mit den optimalen Parametern $a = 15$ und $b = 29$* . 2026.
- [Epp26d] S. Epp. *Learning SAT in Boolean Circuits*. 2026.
- [Sha49] C. E. Shannon. Communication Theory of Secrecy Systems. *Bell System Technical Journal*, 28(4):656–715, 1949.
- [Ver26] G. S. Vernam. Cipher Printing Telegraph Systems for Secret Wire and Radio Telegraphic Communications. *Journal of the AIEE*, 45:109–115, 1926.
- [DH76] W. Diffie and M. E. Hellman. New Directions in Cryptography. *IEEE Transactions on Information Theory*, IT-22(6):644–654, 1976.
- [Riv78] R. L. Rivest, A. Shamir, and L. M. Adleman. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.
- [ElG85] T. ElGamal. A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985.
- [MvOV96] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1996.
- [Buc16] J. Buchmann. *Einführung in die Kryptographie*. Springer, 6. Auflage, 2016.
- [KL15] J. Katz and Y. Lindell. *Introduction to Modern Cryptography*. CRC Press, 2nd edition, 2015.
- [Gol01] O. Goldreich. *The Foundations of Cryptography, Vol. 1*. Cambridge University Press, 2001.
- [DR02] J. Daemen and V. Rijmen. *The Design of Rijndael: AES*. Springer, 2002.
- [BS93] E. Biham and A. Shamir. *Differential Cryptanalysis of the Data Encryption Standard*. Springer, 1993.
- [Mat94] M. Matsui. Linear Cryptanalysis Method for DES Cipher. In *EUROCRYPT '93*, LNCS 765, pp. 386–397. Springer, 1994.
- [BSW07] A. Bogdanov et al. PRESENT: An Ultra-Lightweight Block Cipher. In *CHES 2007*, LNCS 4727, pp. 450–466. Springer, 2007.
- [Dam89] I. B. Damgård. A Design Principle for Hash Functions. In *CRYPTO '89*, LNCS 435, pp. 416–427. Springer, 1989.

- [Mer89] R. C. Merkle. One Way Hash Functions and DES. In *CRYPTO '89*, LNCS 435, pp. 428–446. Springer, 1989.
- [NIST15a] NIST. *FIPS 180-4: Secure Hash Standard (SHS)*. U.S. Department of Commerce, 2015.
- [BDPA13] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche. Keccak. In *EUROCRYPT 2013*, LNCS 7881, pp. 313–314. Springer, 2013.
- [WY05] X. Wang and H. Yu. How to Break MD5 and Other Hash Functions. In *EUROCRYPT 2005*, LNCS 3494, pp. 19–35. Springer, 2005.
- [GM84] S. Goldwasser and S. Micali. Probabilistic Encryption. *Journal of Computer and System Sciences*, 28(2):270–299, 1984.
- [GMR88] S. Goldwasser, S. Micali, and R. L. Rivest. A Digital Signature Scheme Secure Against Adaptive Chosen-Message Attacks. *SIAM Journal on Computing*, 17(2):281–308, 1988.
- [GMR89] S. Goldwasser, S. Micali, and C. Rackoff. The Knowledge Complexity of Interactive Proof Systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.
- [RS91] C. Rackoff and D. R. Simon. Non-Interactive Zero-Knowledge Proof of Knowledge and Chosen Ciphertext Attack. In *CRYPTO '91*, LNCS 576, pp. 433–444. Springer, 1991.
- [BFM88] M. Blum, P. Feldman, and S. Micali. Non-Interactive Zero-Knowledge and Its Applications. In *STOC 1988*, pp. 103–112. ACM, 1988.
- [GGPR13] R. Gennaro, C. Gentry, B. Parno, and M. Raykova. Quadratic Span Programs and Succinct NIZKs Without PCPs. In *EUROCRYPT 2013*, LNCS 7881, pp. 626–645. Springer, 2013.
- [Sho94] P. W. Shor. Algorithms for Quantum Computation: Discrete Logarithms and Factoring. In *FOCS 1994*, pp. 124–134. IEEE, 1994.
- [Gro96] L. K. Grover. A Fast Quantum Mechanical Algorithm for Database Search. In *STOC 1996*, pp. 212–219. ACM, 1996.
- [NIST22] NIST. *Post-Quantum Cryptography Standardization*. <https://csrc.nist.gov/projects/post-quantum-cryptography>, 2022.
- [Reg05] O. Regev. On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. *Journal of the ACM*, 56(6):1–40, 2009.
- [LPR10] V. Lyubashevsky, C. Peikert, and O. Regev. On Ideal Lattices and Learning with Errors over Rings. In *EUROCRYPT 2010*, LNCS 6110, pp. 1–23. Springer, 2010.
- [MR09] D. Micciancio and O. Regev. Lattice-Based Cryptography. In *Post-Quantum Cryptography*, pp. 147–191. Springer, 2009.
- [Pei09] C. Peikert. Public-Key Cryptosystems from the Worst-Case Shortest Vector Problem. In *STOC 2009*, pp. 333–342. ACM, 2009.

- [ABD+22] R. Avanzi et al. CRYSTALS-Kyber: Algorithm Specifications and Supporting Documentation (Version 3.02). *NIST PQC Round 3 Finalist*, 2022.
- [LLL82] A. K. Lenstra, H. W. Lenstra Jr., and L. Lovász. Factoring Polynomials with Rational Coefficients. *Mathematische Annalen*, 261:515–534, 1982.
- [Sch87] C.-P. Schnorr. A Hierarchy of Polynomial Time Lattice Basis Reduction Algorithms. *Theoretical Computer Science*, 53(2–3):201–224, 1987.
- [Gen09] C. Gentry. Fully Homomorphic Encryption Using Ideal Lattices. In *STOC 2009*, pp. 169–178. ACM, 2009.
- [BGV12] Z. Brakerski, C. Gentry, and V. Vaikuntanathan. (Leveled) Fully Homomorphic Encryption Without Bootstrapping. In *ITCS 2012*, pp. 309–325. ACM, 2012.
- [CKKS17] J. H. Cheon, A. Kim, M. Kim, and Y. Song. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *ASIACRYPT 2017*, LNCS 10624, pp. 409–437. Springer, 2017.
- [Kob87] N. Koblitz. Elliptic Curve Cryptosystems. *Mathematics of Computation*, 48(177):203–209, 1987.
- [Mil86] V. S. Miller. Use of Elliptic Curves in Cryptography. In *CRYPTO ’85*, LNCS 218, pp. 417–426. Springer, 1986.
- [BLC+11] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.
- [McE78] R. J. McEliece. A Public-Key Cryptosystem Based on Algebraic Coding Theory. *DSN Progress Report*, 42-44:114–116, 1978.
- [LN97] R. Lidl and H. Niederreiter. *Finite Fields*. Cambridge University Press, 2nd edition, 1997.
- [Yao82] A. C. Yao. Protocols for Secure Computations. In *FOCS 1982*, pp. 160–164. IEEE, 1982.
- [GMW87] O. Goldreich, S. Micali, and A. Wigderson. How to Play Any Mental Game. In *STOC 1987*, pp. 218–229. ACM, 1987.
- [BGW88] M. Ben-Or, S. Goldwasser, and A. Wigderson. Completeness Theorems for Non-Cryptographic Fault-Tolerant Distributed Computation. In *STOC 1988*, pp. 1–10. ACM, 1988.
- [Sha79] A. Shamir. How to Share a Secret. *Communications of the ACM*, 22(11):612–613, 1979.
- [Koc96] P. C. Kocher. Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems. In *CRYPTO ’96*, LNCS 1109, pp. 104–113. Springer, 1996.
- [AKS04] M. Agrawal, N. Kayal, and N. Saxena. PRIMES is in P. *Annals of Mathematics*, 160(2):781–793, 2004.

- [CT91] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. John Wiley & Sons, 1991.
- [Cox97] I. J. Cox, J. Kilian, F. T. Leighton, and T. Shamoon. Secure Spread Spectrum Watermarking for Multimedia. *IEEE Transactions on Image Processing*, 6(12):1673–1687, 1997.
- [Res18] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, 2018.
- [Nak08] S. Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System. *White Paper*, 2008.
- [WP09] H. Wang and B. Peng. A Survey of Security Issues in Wireless Sensor Networks. *IEEE Communications Surveys & Tutorials*, 10(4):1–23, 2008.
- [GLP+21] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing. CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy. In *ICML 2016*. PMLR, 2016.